



华南师范大学

本科学生实验（实践）报告

院 系：计算机学院

实验课程：编译原理课程设计

实验项目：编译原理课程设计

指导老师：黄煜廉

开课时间：2025 ~ 2026 年度第 1 学期

专 业：计算机科学与技术

班 级：计科 2 班

学生姓名：袁知本

华南师范大学教务处

华南师范大学实验报告

学生姓名 袁知本 学 号 20232131006
专 业 计算机科学与技术 年级、班级 2023 级 计科 2 班
课程名称 编译原理课程设计 实验项目 编译原理课程设目
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2025 年 12 月 5 日
实验指导老师 黄煜廉 实验评分 _____

一、课程设计内容

任务一内容：

- (1) 以文本文件的方式输入某一高级程序设计语言的所有单词对应的正则表达式，系统需要提供一个操作界面，让用户打开某一语言的所有单词对应正则表达式文本文件，该文本文件的具体格式可根据自己实际的需要进行定义。
- (2) 正则表达式应该可以支持命名操作，运算符号有：转移符号 (\)、连接、选择 (|)、闭包 (*)、正闭包 (+)、[]、可选 (?)、括号()
- (3) 要提供一个源程序编辑界面，让用户输入一行（一个）或多行（多个）正则表达式（可保存、打开正则表达式文件）
- (4) 需要提供窗口以使用户可以查看转换得到的 NFA（用状态转换表呈现即可）
- (5) 需要提供窗口以使用户可以查看转换得到的 DFA（用状态转换表呈现即可）
- (6) 需要提供窗口以使用户可以查看转换得到的最小化 DFA（用状态转换表呈现即可）
- (7) 将最小化得到 DFA 图转换得到的词法分析源程序（该分析程序需要用 C/C++ 语言描述，而且只能采用讲稿中的转换方法一或转换方法二来生成源程序），系统需提供窗口以使用户可以查看转换得到的词法分析程序
- (8) 对要求（7）得到的源程序进行编译生成一个可执行程序，并以该高级程序设计语言的一个源程序进行测试，输出该源程序的单词编

码。需要提供窗口以便用户可以查看该单词编码。

- (9) 对系统进行测试：(A) 先以 TINY 语言的所有单词的正则表达式作为文本来测试，生成一个 TINY 语言的词法分析源程序；(B) 接着对这个词法分析源程序利用 C/C++编译器进行编译，并生成可执行程序；(C) 以 sample.tny 来测试，输出该 TINY 语言源程序的单词编码文件 sample.lex。

- (10) 要求应用程序为窗口式图形界面

- (11) 书写完善的软件文档

输入格式的要求及说明：

digit=[0-9]

letter=[A-Za-z]

_num100=(\+|-)?digit+

_ID101=letter(letter|digit)*

_specail200S=\+|*|/|=|<|<=<<

说明：

- (1)通过命名中加下划线()来表示该正则表达式需要生成 DFA 图。
- (2)命名中的名字后的数值为对应单词的编码
- (3)命名中的名字中数值后加上 S 表示后面有多个单词，但对应的单词编码从这个数值开始编码。
- (4)由于整数中的正号(+),在系统已经被用作正比闭包运算符，所以需要通过引入转义符号\来区分。同理，如算术运算符(*),在系统已经被用作闭包运算符，所以也需要通过引入转义符号\来区分。

任务二内容：

- (1) 以文本文件的方式输入某一高级程序设计语言的所有文法对应的 BNF 文法，因此系统需要提供一个操作界面，让用户打开某一语言的所有语法 BNF 文法的文本文件，该文本文件的具体格式可根据自己实际的需要进行定义。

- (2) 求出文法的每个非终结符号 First 集合和 Follow 集合，并提供窗口以

使用户可以查看该表结果（可用两张表格的形式分别进行呈现）

（3） 需要提供窗口以使用户可以查看文法对应的 LR(0)DFA 图。（可以用画图的方式呈现，也可用表格方式呈现该图点与边数据）

（4） 判断该文法是否为 SLR(1)文法。（应提供窗口呈现判断的结果，如果不是 SLR(1)文法，需要在窗口中显示其原因）

（5） 需要提供窗口以使用户可以查看文法对应的 LR(1)DFA 图。（可以画图的方式呈现，也可用表格方式呈现该图点与边数据）

（6） 需要提供窗口以使用户可以查看文法对应的 LR(1)分析表。（LR(1)分析表采用表格的形式呈现）

（7） 打开一个与上述所输入 BNF 文法对应的程序设计语言的源程序（即任务一的输出文件），并采用 LR(1)语法分析方法对这个源程序中的每个语句进行语法分析，需要提供窗口以使用户可以查看对应的语法分析过程。【可以使用表格的形式逐行显示分析过程】

（8） 在（7）的分析过程中同时生成相应的语法树，需要提供窗口以使用户可以查看对应的语法分析过程（语法树需要采用树状形式进行呈现）。【每个语句的语法树结构可根据实际的需要定义。】

（9） 以 TINY 用语言的所有语法以及第一项任务的测试结果 sample.lex 作为测试，并生成对应的语法树并呈现出来。

（10） 要求应用程序为窗口式图形界面

（11） 书写完善的软件文档

（12） 选做内容：可以生成某种中间代码【具体的中间代码形式可以自定】

（13） 输入格式的说明：输入的文法均为 2 型文法（上下文无关文法）。文法规则为了处理上的简单，输入时均默认输入的第一个非终结符就是文法的开始符号，用#表示空串 ϵ 。因此下面的文法均为正确的输入：

例 1：

$E \rightarrow E + T$

$T \rightarrow a \mid \#$

由于第一个非终结符为 E，因此 E 是文法的开始符号。

#表示空串 ϵ

例 2:

$\text{exp} \rightarrow \text{exp addop term} \mid \text{term}$

$\text{addop} \rightarrow + \mid -$

$\text{term} \rightarrow \text{term mulop factor} \mid \text{factor}$

$\text{mulop} \rightarrow * \mid /$

$\text{factor} \rightarrow (\text{exp}) \mid n$

由于第一个非终结符为 `exp`，因此 `exp` 是文法的开始符号。

任务三内容:

mini-c 语言作为测试

(1) 以 mini-c 的词法进行测试，并以至少一个 mini-c 源程序进行词法分析的测试（该 mini-c 源程序需要自己根据 mini-c 词法和语法编写出来，类似于 `sample.tny`）。

(2) 以 mini-c 的语法进行测试，并以测试步骤（1）的源程序所生成的单词编码文件进行语法分析，生成对应的语法树。

选做内容:

- (1) 采用 LR(1)语法分析方法进行语法分析并生成相应的中间代码，中间代码可以自选（可以是四元组、三元组、伪代码等中间代码），中间代码生产结果可以在屏幕上显示或以文件的形式保存

二、课程设计目的

任务一目的:

本任务主要目标是实现一个带有图形化窗口界面、支持解析特定格式文件的词法分析器生成工具，通过分析用户输入或导入的指定格式的正则表达式，生成 NFA、DFA、最小化 DFA 图，并生成词法分析代码（能识别并分类源代码中的诸如数字、标识符、关键字、符号等各类单词）。本任务一项目旨在实现程序编译过程中最重要的第一步，实践词法分析的全流程，在正则表达式处理、自动机转换、代码生成与测试验证中强化提高对编译原理技术

的理解与实际工程能力。

任务二目的：

本任务主要目标是实现一个基于 LR(1)分析法的语法分析器生成工具，要求具备图形化窗口界面，实现 FIRST 集、FOLLOW 集的计算与展示，LR(0)、LR(1)自动机的构建与展示，SLR(1)文法判断以及 LR(1)分析表的生成，此外还要完成语法对应源程序代码的语法分析，生成分析表并可视化语法树和生成中间代码，在实现 FIRST 集、FOLLOW 集的计算、LR(0)、LR(1)的 DFA 图构建、SLR(1)语法判断、源程序语法分析、语法树可视化、中间代码生成过程中，深入 LR 分析算法的实践，并提升对编译原理中语法分析阶段的设计与实践能力。

任务三目的：

本任务旨在综合前两个任务的成果，用 mini-c 语言完成对任务一、任务二所实现的项目的整体测试，测试此前完成的编译器的完整度和准确性。通过设计符合 mini-c 词法和语法的测试文本（正则表达式、2 型文法、测试程序源代码等），进一步巩固词法分析、语法分析及中间代码生成等技术，全面提升编译原理设计与实现能力。

三、课程设计文档：

（1）任务一：

1. 系统总体设计：

在进行任务一的项目设计时，我将其分为两个大层次——图形化用户接口层和数据处理层。

图形化用户接口层包括两个模块：I/O 模块（正则表达式的输入输出，导入导出）、展示模块（显示 NFA、DFA、最小化 DFA、词法分析程序代码、lex 文件），其中展示模块中 NFA 图、DFA 图、最小化 DFA 均通过表格形式进行展示。

数据处理层包含五个模块：正则表达式处理模块、NFA 处理构建模块、DFA 处理构建模块、DFA 最小化处理构建模块、词法分析程序生成模块。

各层次和功能模块的如下图所示：

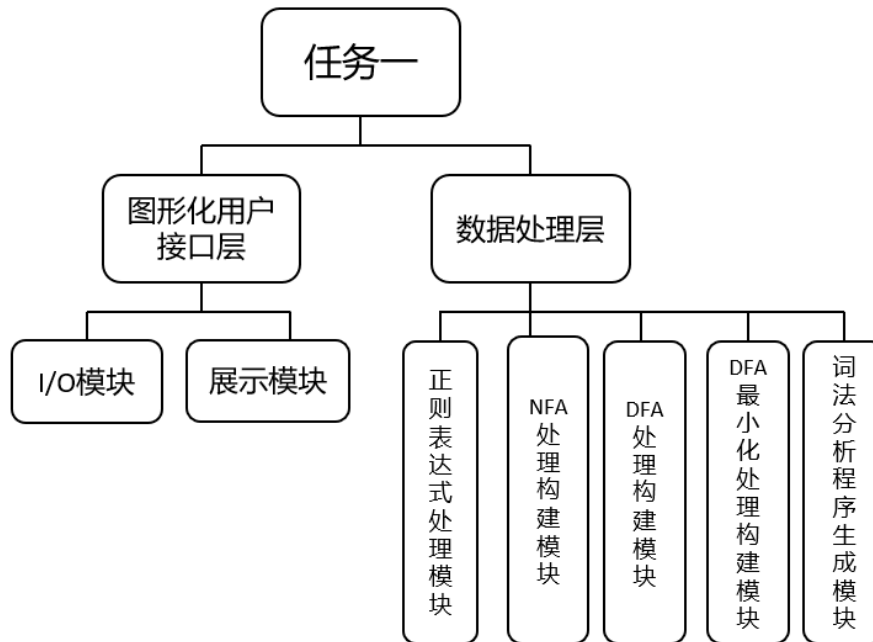


图 1-1 任务一项目的层次架构

2. 功能模块设计与实现：

2.1. 图形化用户接口层各模块

（1）图形化窗口实现

任务一项目通过 QT 实现了图形化窗口界面，通过控件 `plainTextEdit` 提供了正则表达式的输入/导入框，通过控件 `plainTextEdit` 和 `tableWidget` 实现了正则表达式分析后的相关展示功能（`plainTextEdit` 负责展示词法分析程序和 `lex` 文件内容；`tableWidget` 负责以表格形式展示 NFA、DFA、最小化 DFA）。

对于用户的交互操作，我通过 QT 的 `pushButton` 按钮控件予以实现，整个系统共拥有 10 个按钮控件，分别实现了输入规则和使用说明查看、正则表达式的导入、正则表达式的导出、正则表达式分析、NFA 图生成、DFA 图生成、DFA 最小化、词法分析程序生成、`lex` 文件内容查看这九项功能，并将处理的结果通过上述的 `plainTextEdit` 和 `tableWidget` 控件进行呈现。第十个按钮实现了任务一和任务二的操作界面切换。

(2) UI 设计

我通过 QT 自带的 UI 设计面板进行任务一项目的图形化窗口界面设计，具体的 UI 设计图如下图所示：



图 1-2 任务一项目 UI 设计图

项目系统启动运行后，其界面如下图所示：



图 1-3 任务一项目系统运行界面图

(3) 各模块功能实现

图形化窗口界面的展示模块通过 `pushButton` 按钮控件的槽函数

触发，并通过 `plainTextEdit` 和 `tableWidget` 控件进行展示，具体的各槽函数定义如下：

```

/*****

* @Copyright (c) 2025 LDL. All rights reserved.

*

* @FileName: widget.h

* @Brief: 应用程序窗口界面声明文件，包含主窗口类，负责交互操作控件定义

* @Module: 主界面模块

*

* @Current Version: 2.0.0

* @Author: 袁知本

* @Created Date: 2025/10/5

*

* @Revision History:

*   Version   Date       Author    Description
*   -----   -
*   1.0.0     2025/10/5   袁知本    初始版本创建
*   2.0.0     2026/1/16   袁知本    整合任务一和任务二的交互控件函数

*****/

#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
#include <QGraphicsScene>
#include <QGraphicsProxyWidget>

QT_BEGIN_NAMESPACE
namespace Ui { class Widget; }
```

```
QT_END_NAMESPACE
```

```
class Widget : public QWidget
```

```
{
```

```
    Q_OBJECT
```

```
public:
```

```
    Widget(QWidget *parent = nullptr);
```

```
    void wrapPage(QWidget *page, QGraphicsView *view);
```

```
    bool eventFilter(QObject *watched, QEvent *event);
```

```
    ~Widget();
```

```
private slots:
```

```
    // 查看正则表达式的输入规则说明
```

```
    void on_pushButton_Help_clicked();
```

```
    // 导入正则表达式
```

```
    void on_pushButton_LoadRegex_clicked();
```

```
    // 导出正则表达式
```

```
    void on_pushButton_SaveRegex_clicked();
```

```
    // 正则表达式分析
```

```
    void on_pushButton_Analyse_clicked();
```

```
    // 生成 NFA
```

```
    void on_pushButton_ShowNFA_clicked();
```

```
// 生成 DFA

void on_pushButton_ShowDFA_clicked();


// DFA 最小化

void on_pushButton_ShowMinDFA_clicked();


// 生成词法分析程序

void on_pushButton_GenerateCode_clicked();


// 查看 LEX 文件

void on_pushButton_LoadLEXFile_clicked();


// 查看文法输入规则说明

void on_pushButton_Help_1_clicked();


// 载入文法规则

void on_pushButton_LoadGrammar_clicked();


// 保存文法规则

void on_pushButton_SaveGrammar_clicked();


// 载入源程序

void on_pushButton_LoadProgram_clicked();


// 保存源程序

void on_pushButton_SaveProgram_clicked();


// 求 First 集

void on_pushButton_GetFirstSet_clicked();
```

// 求 Follow 集

void on_pushButton_GetFollowSet_clicked();

// 分析结果说明

void on_pushButton_Help_2_clicked();

// LR(0)DFA 生成

void on_pushButton_GenLR0DFA_clicked();

// SLR(1) 文法分析

void on_pushButton_SLR1Analyse_clicked();

// LR(1)DFA 生成

void on_pushButton_GenLR1DFA_clicked();

// LR(1) 分析表生成

void on_pushButton_LR1Analyse_clicked();

// 源程序分析

void on_pushButton_AnalyseSentence_clicked();

// 中间代码生成

void on_pushButton_Interlanguage_clicked();

// 可视化显示语法树

void on_pushButton_ShowSyntaxTree_clicked();

// 转到任务二界面

```
void on_commandLinkButton_clicked();

// 转到任务一界面

void on_commandLinkButton_2_clicked();

private:
    Ui::Widget *ui;
};

#endif // WIDGET_H
```

2.2. 正则表达式输入处理

该模块负责正则表达式的输入、预处理和转换，为后续的 NFA 构建做准备，其主要功能包括文件操作（指通过文件导入和导出正则表达式）、正则表达式解析、特殊符号处理和格式规范化，为后续的分析处理做好相应的初期准备。

2.2.1 存储结构

输入：符合格式要求的单行或多行正则表达式（字符串）

输出：预处理后的正则表达式（finalRegex 变量存储）

主要存储结构如下表所示：

表 1-1 正则表达式主要存储结构

对象或变量名称	功能	存储结构
RegexEncoding	编码映射结构，用于存储正则表达式的编码映射关系	struct regexEncoding
regexEncodings	存储用户输入的所有正则表达式编码	Vector<RegexEncoding> regexEncodings
encodingMap	显式名称到编码的映射	map<string, int>
sequenceEncodings	存储序列编码映射，处理多个单词顺序编码	map<string, vector<pair<string, int>>
m1	符号到字符串的映射表，用于显示	map<char, string>
m2	字符串到符号的映射表，用于数据处理层的后续处理	map<string, char>
finalRegex	存储处理后的正则表达式	string

keyWords	存储关键词集合	set<string>
onMap	存储操作符到操作符名称的映射	map<string, string>
commentSymbol	存储注释符号（开始符号和结束符号）	string[2]
isLowerCase	大小写是否考虑的布尔变量	bool
regexStd	存储转换为标准字符串格式的正则表达式	string
result	分析结果判断，用于分析正则表达式是否符合输入格式要求	string
allRegex	存储输入的所有原始正则表达式文本	QString

2.2.2 算法实现过程文字描述

实现正则表达式处理可以分为输入、预处理、组件提取、表达式构建、输出这五个阶段：

（1）输入阶段：

用户通过文件载入或手动输入符合输入格式要求的正则表达式，点击“开始分析”按钮后，系统将读取原始输入/载入的正则表达式文本，进行初步的格式合规性验证。

（2）预处理阶段：

在用户点击“开始分析”按钮后，系统移除所有的空白字符，简化后续处理，按行存储表达式并记录各表达式的编码（根据规则，有数字尾缀且带S尾缀的进行多个单词顺序编码，不带S尾缀但有数字尾缀的按数字作为编码值），之后再将易冲突的操作符替换为内部表示字符（通过m2映射实现），此阶段通过核心处理函数handleAllRegex实现。

（3）组件提取阶段：

提取关键词并存入集合，建立操作符到名称的对应关系，解析注释符号并生成注释模式，对部分冗长表达式展开（如[0-9]、[a-zA-Z]等）进行选择替代（如替代为digit、letter，前提是有诸如digit=[0-9]、letter=[a-zA-Z]）。

（4）表达式构建阶段：

将各组件用括号包围并通过“|”符号进行连接，并处理中括号、“+”号等特殊符号语法，此外还有在需要隐式连接的位置添加显示连接符（本次课

"(\u0002@\u0002*@\u0002@\u0002*)(\u0004@(\u0004|\u0002)*)"

使用 TINY 的正则表达式进行测试，经过对输入格式适配性处理后，对应的输入内容如下所示：

```
digit=[0-9]
letter=[a-zA-Z]
_key100S=if|then|else|end|repeat|until|read|write
_special200S=\+\\|\\*\\|\\%|<|<>|<=|>|=|=:|\\(|\\)\\
_specialName300S=PLUS|MINUS|MULTIPLY|DIVIDE|MOD|LT|NE|LTEQ|RTEQ|RT|EQ|SEMI|ASSIGN|LPAN|RPAN
_ID400=letter(letter|digit)*
_num500=digitdigit*
_annotation600={~}
```

处理后的正则表达式如下所示（采用@符号作为显示连接符，其中诸如\u0002 等是由于对冗长的展开式进行替代后的结果，不影响正则表达式的预处理）：

```
"(i@f|t@h@e@n|e@l@s@e@n@n@d|r@e@p@e@a@t|u@n@t@i@l|r@e@a
@d|w@r@i@t@e)|(\u000B|
|\u000F|\\%|<|<@>|<@=|>@=|>|=|:|@=|r|\u000E)|(\u0004@(\u0004|\u0002)*
)|(\u0002@\u0002*)({ @~@})"
```

2.3. NFA 构造

该模块负责将预处理后的正则表达式转换为 NFA 图（非确定有限自动机，本次项目中采用表格形式展示），其主要功能包括正则表达式的解析、基本 NFA 单元的构建、运算符（连接、选择、闭包等）的处理、NFA 图的组合以及状态转换表的生成，为后续的 DFA 转换和语法分析提供结构化的自动机基础。

2.3.1 存储结构

输入：之前预处理完成的正则表达式字符串

输出：完整的 NFA 图结构和状态转换表

主要存储结构如下表所示：

表 1-2 NFA 图的主要存储结构		
对象或变量名称	功能	存储结构
nodeCount	全局节点 ID 计数器，确保每个节点都有唯一的标识	int
nfaCharSet	存储 NFA 中出现的所有字符	set<char>
statusTableNode	状态转换表的单个节点	struct statusTableNode
nfaNode	NFA 状态节点，包含状态信息和出边	struct nfaNode
nfaEdge	NFA 转移边，包含字符和目标节点	struct nfaEdge
NFA	完整的 NFA 图包含起始和结束节点	struct NFA
statusTableNode	状态转换表条目，记录状态转移关系	struct statusTableNode
statusTable	全局状态转换表，映射状态 ID 到状态信息	unordered_map<int, statusTableNode>
insertionOrder	记录状态插入顺序，用于进行有序的展示	vector<int>
startNFAsatus	存储所有初态节点的 ID	set<int>
endNFAsatus	存储所有终态节点的 ID	set<int>
MappedCharTable	映射字符表，保存映射字符到显示名称的关系，辅助 NFA 图表的构建	unordered_map<char, string> mappedCharTable
final_nfa	构建完成后的 NFA，用于后续 DFA 构建	NFA final_nfa

2.3.2 算法实现过程文字描述

正则表达式转 NFA 的核心算法采用双栈递归下降解析策略，通过将复杂的正则表达式分解为基本构建块，然后按照运算符优先级组合成完整的 NFA 图。整个处理流程从接收预处理后的正则表达式字符串开始，首先初始化运算符栈(opStack)和 NFA 栈(nfaStack)，然后逐个字符扫描表达式，根据字符类型采取不同的处理策略。

NFA 图生成的算法基本原理如下：

- 基本 NFA 图的生成：

对于预处理后的正则表达式转为 NFA 图，本项目中我采用了

参考逆波兰表达式的**双栈法**，对正则表达式进行扫描，用两个栈进行存储（一个存储运算符，一个存储 NFA 图），并做好正则表达式的运算符优先级定义：

表 1-3 正则表达式运算符优先级

符号	含义	编码（数字越大优先级越高）
(your_regex) [your_regex] \ ? + *	括号、字符集/数字集、转义符 可选、正闭包、闭包	4 3
@	连接符（本项目中用@符号替代隐式连接为显式连接符）	2
	或运算	1
(	左括号（具有最高优先级）	0

如表 1-3 所示，入栈前左括号有最高优先级，让其他运算符入栈，但入栈后左括号的优先级变为最低，在扫描正则表达式遇到右括号之前阻止其他运算符出栈，故将其编码值设置为 0，而其他的运算符则按照其优先级设置对应的编码值

当扫描遇到一个字符时，需要创建一个局部的 NFA 图，示例如下：

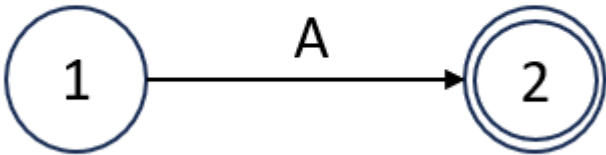


图 1-4 构建基本 NFA 图

如图 1-4 所示，遇到一个普通字符的时候构造局部 NFA 图，同时初态和

终态都为 true，并将该字符作为两个状态的转移字符。

若遇到的是运算符，则需要先通过优先级判断进行处理，具体如下：

- 或运算符处理：

形如 $a|b$ (a 和 b 为局部 NFA 图)，在扫描时系统已将 a 和 b 的 NFA 图 push 到 NFA 栈中，而在 pop 出来后，需要对 a 和 b 的 NFA 图进行连接——即创建一个新初态和一个新终态，并通过 ϵ 边进行连接，如下图所示：

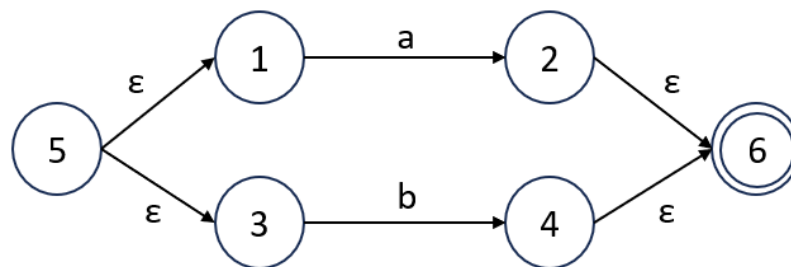


图 1-5 处理或运算符

同理可得，当进行其他运算符的连接操作时，NFA 的构建方式都遵循新建一个新初态和一个新终态，并使用 ϵ 边进行连接的方式：

- 处理闭包：

如下图所示：

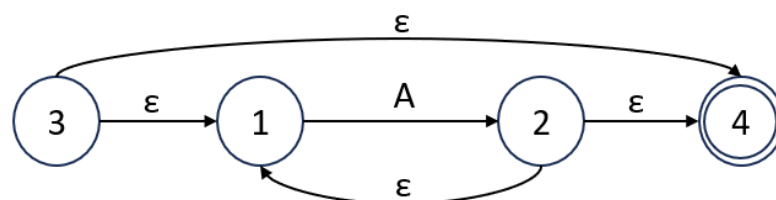


图 1-6 处理闭包运算符

- 处理可选：

如下图所示：

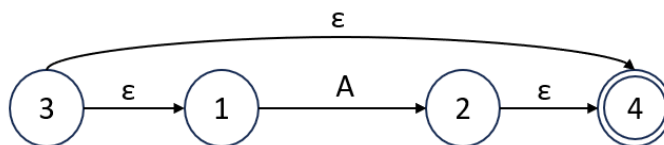


图 1-6 处理可选运算符

实现 NFA 图的构建过程可以分为输入处理与字符分类、运算符处理与 NFA 组合构建、后续处理与状态表生成这三个阶段：

(1) 输入处理与字符分类：

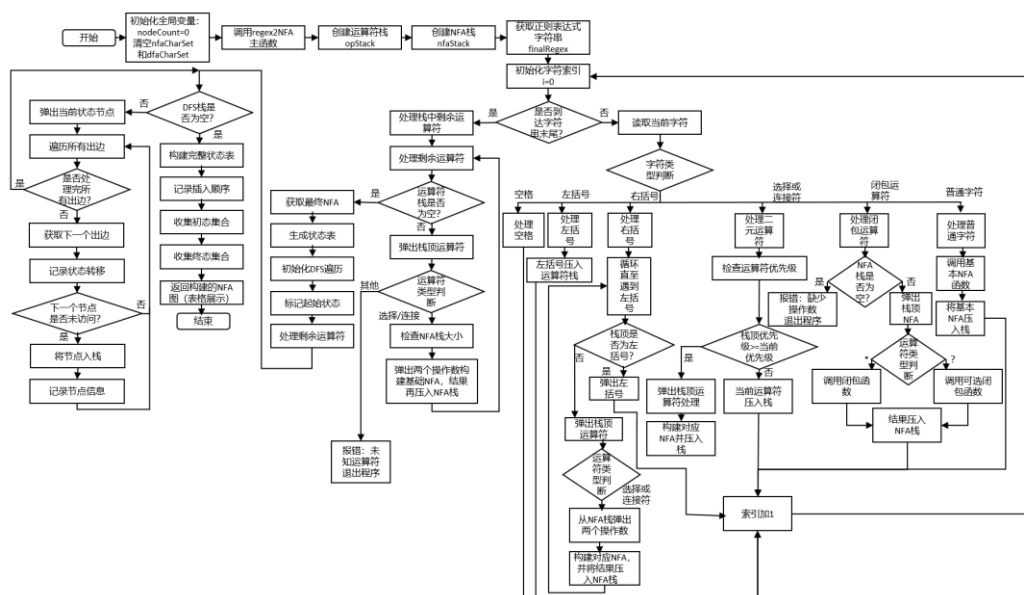
系统接收预处理后的正则表达式字符串，并初始化运算符栈 `opStack` 和 NFA 栈 `nfaStack`。算法逐个字符扫描输入表达式，根据字符类型进行差异化处理：遇到空格字符时直接跳过；遇到左括号 '(' 时压入 `opStack` 栈；遇到右括号 ')' 时循环处理栈内运算符直到匹配的左括号；遇到普通字符时，首先通过 `isMappedChar` 函数判断其是否为映射字符，如果是映射字符则调用 `CreateMappedCharNFA` 创建保留映射名称的 NFA 单元，否则调用 `CreateBasicNFA` 创建基本 NFA 单元，并将生成的 NFA 压入 `nfaStack`。在处理映射字符的过程中，系统会通过 `registerMappedChar` 函数记录映射字符与显示名称的对应关系，并更新 `mappedCharTable`、`nfaCharSet` 和 `dfaCharSet`。

(2) 运算符处理与 NFA 组合构建：

当遇到选择运算符 '|' 或连接运算符 '@' 时，系统通过 `Precedence` 函数比较运算符优先级，确保高优先级运算优先处理。处理过程中从 `nfaStack` 弹出相应数量的 NFA 子图，分别调用 `CreateUnionNFA` 或 `CreateConcatenationNFA` 函数构建新的组合 NFA。对于闭包运算符 '*' 和 '?', 直接弹出栈顶 NFA 并调用 `CreateZeroOrMoreNFA` 或 `CreateOptionalNFA` 函数进行处理，构建包含循环或可选语义的 NFA 结构。所有 NFA 组合操作均通过创建新的 `nfaNode` 节点、设置 `isStart` 和 `isEnd` 标志、并添加带有 EPSILON 或指定字符的 `nfaEdge` 边来完成。

(3) 后续处理与状态表生成：

2.3.3 算法实现流程图



继续基于此前预处理完成的简单的测试用正则表达式进行 NFA 图构建的测试，具体结果如下：



图 1-5 任务一 NFA 图构建的单元测试

继续基于此前预处理完成的 TINY 的正则表达式进行 NFA 图构建的测试，具体结果如下：

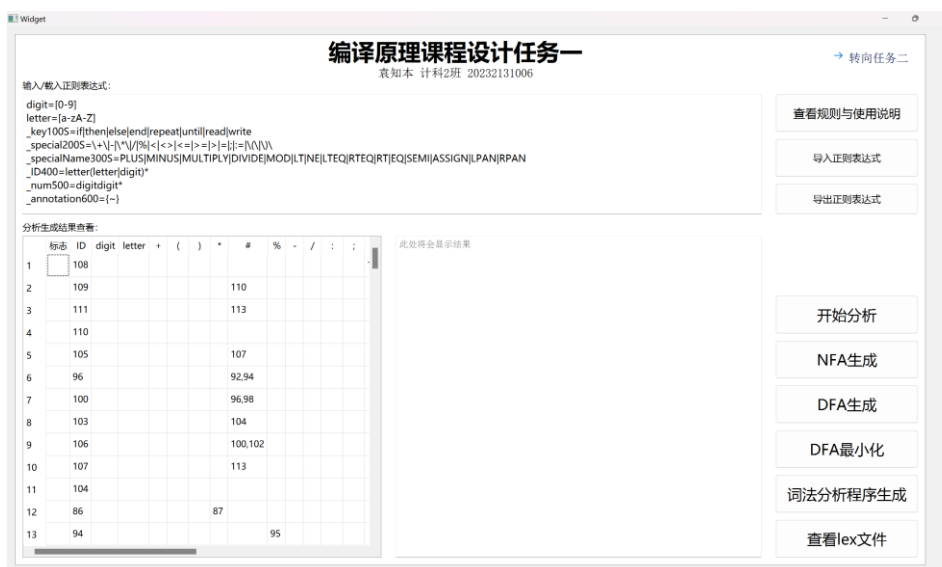


图 1-6 任务一 NFA 图构建的单元测试

需注意的是，由于分析结果的 NFA 表大小以及展示框的大小问题，无法完整展示所有的结果，但可以通过滑动条查看，处理得到的 NFA 表完全正确，并且其中的 digit 和 letter 分别作为了[0-9]和[a-zA-Z]冗长的展开式的替代，简化了 NFA 的展示，并未影响到内在的逻辑。

2.3.5 关键源程序代码段

NFA 核心数据结构定义，定义了 NFA 的底层数据结构，为整个构建过程提供了基础支撑：

```
// 结构体，NFA 图的边

struct nfaEdge
{
    char c;                // 转移字符
    nfaNode* next;         // 指向目标节点的指针
};

// 结构体，NFA 图的结点

struct nfaNode
{
    int id;                // 结点唯一编号
    bool isStart;          // 初态标识
    bool isEnd;            // 终态标识
    vector<nfaEdge> edges;  // 边，用 vector 因为有可能一个结点有多条边可走

    nfaNode() {
        id = nodeCount++;
        isStart = false;
        isEnd = false;
    }
};

// 结构体，NFA 图

struct NFA
{
    nfaNode* start;
    nfaNode* end;
```

```

NFA() {}

NFA(nfaNode* s, nfaNode* e)

{
    start = s;

    end = e;

}

};

// 结构体，状态转换表单个结点

struct statusTableNode

{

    string flag;           // 标记初态还是终态

    int id;                // 唯一 id 值

    map<char, set<int>> m; // 对应字符能到达的状态

    statusTableNode()

    {

        flag = "";        // 默认为空

    }

};

```

双栈算法核心逻辑（实现正则表达式的递归下降解析）：

```

// 主处理循环

for (char c : regex)

{

    switch (c)

    {

        case ' ': // 空格跳过

            break;

        case '(':

```

```

        opStack.push(c);

        break;
    case ')':
        while (!opStack.empty() && opStack.top() != '(')
        {
            char op = opStack.top();
            opStack.pop();

            if (op == '|') {
                if (nfaStack.size() < 2) {
                    qDebug() << "正则表达式语法错误：不足以处理运算符
|";

                    exit(-1);
                }
                NFA nfa2 = nfaStack.top();
                nfaStack.pop();
                NFA nfa1 = nfaStack.top();
                nfaStack.pop();

                NFA resultNFA = CreateUnionNFA(nfa1, nfa2);
                nfaStack.push(resultNFA);
            }
            else if (op == '@') {
                if (nfaStack.size() < 2) {
                    qDebug() << "正则表达式语法错误：不足以处理运算符
@";

                    exit(-1);
                }
                NFA nfa2 = nfaStack.top();

```

```

        nfaStack.pop();

        NFA nfa1 = nfaStack.top();

        nfaStack.pop();

        NFA resultNFA = CreateConcatenationNFA(nfa1, nfa2);

        nfaStack.push(resultNFA);

    }

}

if (opStack.empty())

{

    qDebug() << "括号未闭合，请检查正则表达式！";

    exit(-1);

}

else

{

    opStack.pop(); // 弹出(

}

break;

case '|':

case '@':

    while (!opStack.empty() && (opStack.top() == '|' || opStack.top() == '@'))

    &&

        Precedence(opStack.top()) >= Precedence(c))

    {

        char op = opStack.top();

        opStack.pop();

        if (op == '|') {

            if (nfaStack.size() < 2) {

```

```

        qDebug() << "正则表达式语法错误：不足以处理运算符
|";

        exit(-1);
    }
    NFA nfa2 = nfaStack.top();
    nfaStack.pop();
    NFA nfa1 = nfaStack.top();
    nfaStack.pop();

    NFA resultNFA = CreateUnionNFA(nfa1, nfa2);
    nfaStack.push(resultNFA);
}
else if (op == '@') {
    if (nfaStack.size() < 2) {
        qDebug() << "正则表达式语法错误：不足以处理运算符
@";

        exit(-1);
    }
    NFA nfa2 = nfaStack.top();
    nfaStack.pop();
    NFA nfa1 = nfaStack.top();
    nfaStack.pop();

    NFA resultNFA = CreateConcatenationNFA(nfa1, nfa2);
    nfaStack.push(resultNFA);
}
}
opStack.push(c);
break;

```

```
case '?':

case '*':

    if (!nfaStack.empty()) {

        NFA nfa = nfaStack.top();

        nfaStack.pop();

        if (c == '?') {

            NFA resultNFA = CreateOptionalNFA(nfa);

            nfaStack.push(resultNFA);

        }

        else if (c == '*') {

            NFA resultNFA = CreateZeroOrMoreNFA(nfa);

            nfaStack.push(resultNFA);

        }

    }

    else {

        qDebug() << "正则表达式语法错误： 闭包操作没有 NFA 可用！ ";

        exit(-1);

    }

    break;

default:

    if (isMappedChar(c)) {

        // 处理映射字符（保留映射名称）

        NFA nfa = CreateMappedCharNFA(c);

        nfaStack.push(nfa);

    } else {

        // 处理普通字符

        NFA nfa = CreateBasicNFA(c);

        nfaStack.push(nfa);

    }

}
```

```
        break;

    }

}
```

基本 NFA 构造函数，创建最基本的 NFA 单元，是所有复杂 NFA 的构建基础：

```
// 创建基本字符的 NFA
NFA GenerateNFA::CreateBasicNFA(char character) {

    nfaNode* start = new nfaNode();

    nfaNode* end = new nfaNode();

    start->isStart = true;

    end->isEnd = true;

    nfaEdge edge;

    edge.c = character;

    edge.next = end;

    start->edges.push_back(edge);

    // 如果是普通可打印字符，添加到字符集
    if (character >= 32 && character <= 126) {

        nfaCharSet.insert(character);

        dfaCharSet.insert(character);

    }

    NFA nfa(start, end);

    return nfa;

}
```

组合运算构造函数，实现 NFA 的组合运算，连接基本（局部）NFA 图，支持复杂 NFA 图的构建：

```
// 创建可选运算 (?) 的 NFA
NFA GenerateNFA::CreateOptionalNFA(NFA nfa1) {
    nfaNode* start = new nfaNode();
    nfaNode* end = new nfaNode();

    start->isStart = true;
    end->isEnd = true;

    nfaEdge edge1;
    edge1.c = EPSILON;
    edge1.next = nfa1.start;
    start->edges.push_back(edge1);
    nfa1.start->isStart = false;

    nfaEdge edge2;
    edge2.c = EPSILON;
    edge2.next = end;
    start->edges.push_back(edge2);

    nfa1.end->isEnd = false;

    nfaEdge edge3;
    edge3.c = EPSILON;
    edge3.next = end;
    nfa1.end->edges.push_back(edge3);
}
```

```
NFA nfa(start, end);

return nfa;
}

// 创建闭包运算（*）的 NFA
NFA GenerateNFA::CreateZeroOrMoreNFA(NFA nfa1) {
    nfaNode* start = new nfaNode();
    nfaNode* end = new nfaNode();

    start->isStart = true;
    end->isEnd = true;

    nfaEdge edge1;
    edge1.c = EPSILON;
    edge1.next = nfa1.start;
    start->edges.push_back(edge1);
    nfa1.start->isStart = false;

    nfaEdge edge2;
    edge2.c = EPSILON;
    edge2.next = end;
    start->edges.push_back(edge2);

    nfa1.end->isEnd = false;

    nfaEdge edge3;
    edge3.c = EPSILON;
    edge3.next = nfa1.start;
```

```

nfa1.end->edges.push_back(edge3);

nfaEdge edge4;
edge4.c = EPSILON;
edge4.next = end;
nfa1.end->edges.push_back(edge4);

NFA nfa{ start,end };

return nfa;
}

// 创建选择运算 (|) 的 NFA
NFA GenerateNFA::CreateUnionNFA(NFA nfa1, NFA nfa2) {

    nfaNode* start = new nfaNode();
    nfaNode* end = new nfaNode();

    start->isStart = true;
    end->isEnd = true;

    nfaEdge edge1;
    edge1.c = EPSILON;
    edge1.next = nfa1.start;
    start->edges.push_back(edge1);
    nfa1.start->isStart = false;

    nfaEdge edge2;
    edge2.c = EPSILON;
    edge2.next = nfa2.start;

```

```

    start->edges.push_back(edge2);

    nfa2.start->isStart = false;

    nfa1.end->isEnd = false;
    nfa2.end->isEnd = false;

    nfaEdge edge3;
    edge3.c = EPSILON;
    edge3.next = end;
    nfa1.end->edges.push_back(edge3);

    nfaEdge edge4;
    edge4.c = EPSILON;
    edge4.next = end;
    nfa2.end->edges.push_back(edge4);

    NFA nfa{start, end};

    return nfa;
}

// 创建连接运算（@）的 NFA
NFA GenerateNFA::CreateConcatenationNFA(NFA nfa1, NFA nfa2) {
    nfa1.end->isEnd = false;
    nfa2.start->isStart = false;

    nfaEdge edge;
    edge.c = EPSILON;
    edge.next = nfa2.start;

```

```

nfa1.end->edges.push_back(edge);

NFA nfa;

nfa.start = nfa1.start;

nfa.end = nfa2.end;

return nfa;
}

```

优先级判断函数，确保运算符按照正确的顺序定义，避免了语法歧义：

```

// 辅助函数，获取运算符优先级
int GenerateNFA::Precedence(char op) {

    switch(op) {

        case '|': return 1;

        case '@': return 2;

        case '*': return 3;

        case '?': return 3;

        default: return 0;

    }

}

```

状态表生成函数，将 NFA 图转换为可查询的状态转换表，再通过 Qt 控件展示在图形化窗口页面上供用户查看 NFA 构建结果：

```

// 生成 NFA 状态转换表，使用 DFS 算法
void GenerateNFA::createNFAStatusTable(NFA& nfa) {

    stack<nfaNode*> nfaStack;

    set<nfaNode*> visitedNodes;

    // 创建节点 ID 到节点的映射

```

```
unordered_map<int, nfaNode*> idToNode;

// 清空现有数据

statusTable.clear();

insertionOrder.clear();

startNFAsatus.clear();

endNFAsatus.clear();

nfaNode* startNode = nfa.start;

// 深度优先遍历，收集所有节点

nfaStack.push(startNode);

while (!nfaStack.empty()) {

    nfaNode* currentNode = nfaStack.top();

    nfaStack.pop();

    if (visitedNodes.find(currentNode) != visitedNodes.end()) {

        continue;

    }

    visitedNodes.insert(currentNode);

    idToNode[currentNode->id] = currentNode;

    for (const nfaEdge& edge : currentNode->edges) {

        if (visitedNodes.find(edge.next) == visitedNodes.end()) {

            nfaStack.push(edge.next);

        }

    }

}
```

```
// 创建状态表条目

for (nfaNode* node : visitedNodes) {

    int nodeId = node->id;

    // 确保每个节点都有状态表条目

    if (statusTable.find(nodeId) == statusTable.end()) {

        statusTableNode newNode;

        newNode.id = nodeId;

        // 设置标志

        if (node->isStart) {

            newNode.flag = "-";

            startNFAsatus.insert(nodeId);

        }

        if (node->isEnd) {

            if (!newNode.flag.empty()) newNode.flag += ",";

            newNode.flag += "+";

            endNFAsatus.insert(nodeId);

        }

        if (newNode.flag.empty()) {

            newNode.flag = " ";

        }

        statusTable[nodeId] = newNode;

        // 添加到插入顺序（除了终态在最后）

        if (!node->isEnd) {

            insertionOrder.push_back(nodeId);

        }

    }

}
```

```

        }

    }

}

// 添加终态到插入顺序的末尾
for (nfaNode* node : visitedNodes) {
    if (node->isEnd) {
        insertionOrder.push_back(node->id);
    }
}

// 再次遍历，填充转移关系
for (nfaNode* node : visitedNodes) {
    int nodeId = node->id;

    for (const nfaEdge& edge : node->edges) {
        char transitionChar = edge.c;
        int targetId = edge.next->id;

        // 确保目标节点在状态表中
        if (statusTable.find(targetId) == statusTable.end()) {
            qDebug() << "警告：目标状态" << targetId << "不在状态表中，正在添加...";

            statusTableNode targetNode;
            targetNode.id = targetId;

            if (edge.next->isStart) {
                targetNode.flag = "-";
                startNFAsatus.insert(targetId);
            }
        }
    }
}

```

```

    }

    if (edge.next->isEnd) {

        if (!targetNode.flag.empty()) targetNode.flag += ",";

        targetNode.flag += "+";

        endNFAsatus.insert(targetId);

    }

    if (targetNode.flag.empty()) {

        targetNode.flag = " ";

    }

    statusTable[targetId] = targetNode;

    // 添加到插入顺序

    if (!edge.next->isEnd) {

        insertionOrder.push_back(targetId);

    } else {

        // 终态添加到末尾

        insertionOrder.push_back(targetId);

    }

}

// 添加转移

statusTable[nodeId].m[transitionChar].insert(targetId);

}

}

// 确保终态被正确标记

for (nfaNode* node : visitedNodes) {

    if (node->isEnd && endNFAsatus.find(node->id) == endNFAsatus.end()) {

```

```

        endNFAsatus.insert(node->id);

        if (statusTable.find(node->id) != statusTable.end()) {

            statusTableNode& nodeEntry = statusTable[node->id];

            if (nodeEntry.flag.find('+') == string::npos) {

                if (nodeEntry.flag != " ") {

                    nodeEntry.flag += ",";

                } else {

                    nodeEntry.flag = "+";

                }

            }

        }

    }

}

qDebug() << "NFA 状态表构建完成";

qDebug() << "共收集到" << visitedNodes.size() << "个节点";

qDebug() << "状态表中有" << statusTable.size() << "个状态";

qDebug() << "初态:" << startNFAsatus.size() << "个";

qDebug() << "终态:" << endNFAsatus.size() << "个";

}

```

2.4. DFA 构造

通过子集构造法实现了从 NFA 到 DFA 的转换，使用 ϵ 闭包和字符闭包计算来构建等价的 DFA 图（本项目使用表格的形式展示），为后续的 DFA 最小化提供了基础。

2.4.1 存储结构

输入：NFA 图结构（完整的 NFA 状态转换表 statusTable、起始节点 start、种植状态集合 endNFAsatus 等）、全局字符集 dfaCharSet、状态映射信息等

输出：完整的 DFA 状态转换表

主要存储结构如下表所示：

表 1-4 DFA 图的主要存储结构

对象或变量名称	功能	存储结构
dfaNode	DFA 状态节点结构	struct dfaNode
dfaTable	存储完整的 DFA 状态表	Vector<dfaNode>
dfaStatusSet	用于 DFA 状态去重，避免重复状态	set<set<int>>
dfaEndStatusSet	存储所有 DFA 终态编号	set<int>
dfaNotEndStatusSet	存储所有 DFA 非终态编号	set<int>
dfa2numberMap	状态集合到 DFA 状态编号的映射	map<set<int>, int>
startStatus	记录 DFA 起始状态编号	int
dfaCharSet	存储 DFA 字符集（从 NFA 继承）	set<char>

2.4.2 算法实现过程文字描述

NFA 转 DFA 的算法采用了**子集构造法**，从 NFA 的起始状态开始，通过计算 ϵ 闭包和字符转移闭包逐步构建等价的 DFA。

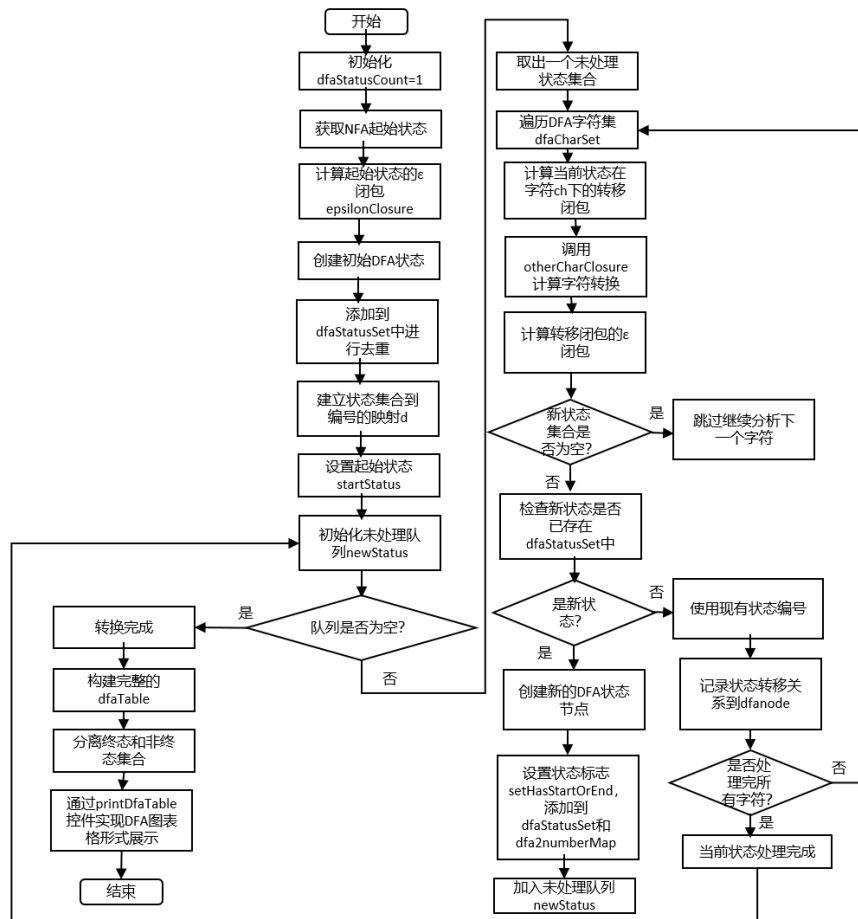
该算法首先初始化 DFA 状态计数器 dfaStatusCount 和起始状态处理：获取 NFA 的起始节点 start 后，调用 epsilonClosure 函数计算起始状态的 ϵ 闭包，得到 DFA 的初始状态集合，然后使用 setHasStartOrEnd 函数判断该状态集合是否包含初态或终态，并设置相应的 flag 标记，接着算法维护全局状态映射关系，通过 dfa2numberMap 记录状态集合到编号的映射，并根据状态属性将状态分类存入 dfaEndStatusSet 或 dfaNotEndStatusSet。

之后算法进入字符处理循环过程，遍历 dfaCharSet 中的每个字符，对当前状态中的每个 NFA 状态，调用 otherCharClosure 函数计算在字符下的转移闭包，将这些闭包合并形成新的状态集合，若新集合不为空，则检查 dfaStatusSet 判断是否为新状态，若是新状态，则分配新编号并加入处理队列 newStatus，同时更新状态分类，无论是否新状态，都会记录转移关系到当前 DFA 节点的 transitions 映射中。

完成初始状态处理后，算法使用**广度优先搜索（BFS）策略**，循环处理 newStatus 队列中的每个待处理状态，对每个状态集合，同样遍历所有字符计

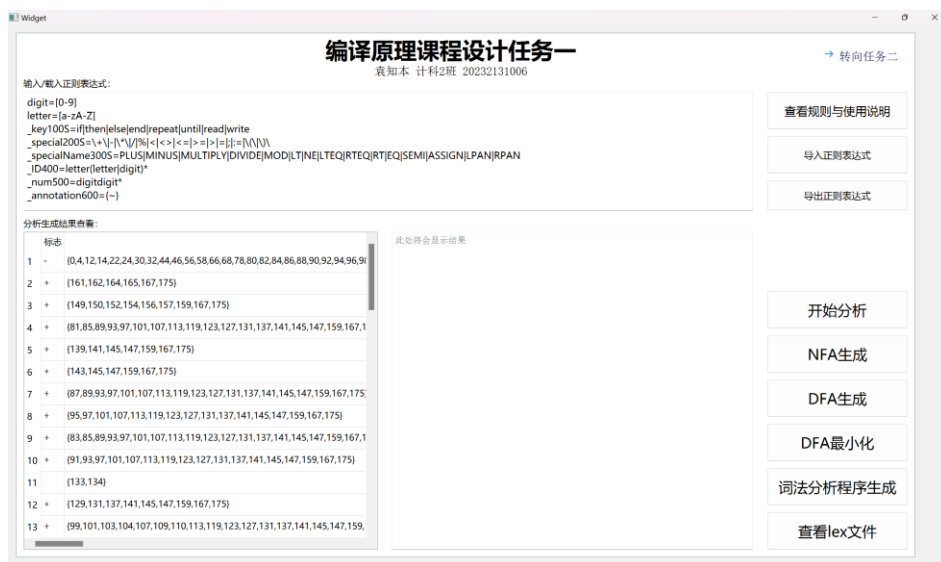
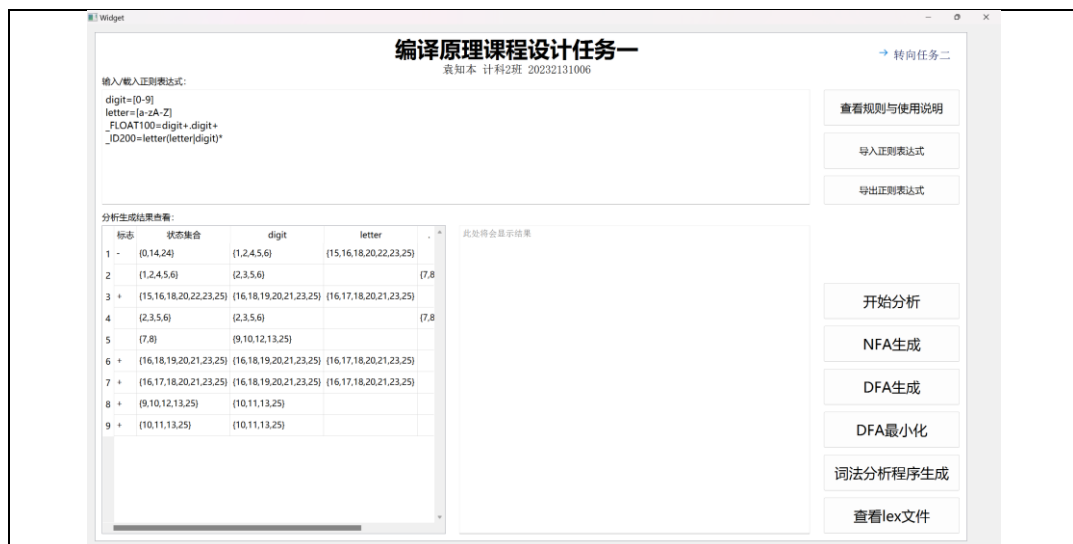
算转移闭包，生成新状态并更新 DFA 表。整个过程持续到没有新状态产生为止，最终 dfaTable 中存储了完整的 DFA 状态转换表，其中每个 dfaNode 包含状态标记、对应的 NFA 状态集合以及所有字符的转移关系。

2.4.3 算法实现流程图



2.4.4 单元测试

继续基于此前构建的 NFA 图进行 NFA 转 DFA 图的测试，具体结果如下：



任务一 DFA 图构建的单元测试 (TINY 语言)

需注意的是，由于分析结果的 DFA 表大小以及展示框的大小问题，无法完整展示所有的结果，但可以通过滑动条查看，处理得到的 DFA 表完全正确，并且其中的 digit 和 letter 分别作为了[0-9]和[a-zA-Z]冗长的展开式的替代，简化了 DFA 的展示，并未影响到内在的逻辑。

2.4.5 关键源程序代码段

核心数据结构定义，定义了 DFA 状态的基本结构，包含状态标记、对

应的 NFA 状态集合和转移关系

```
// dfa 节点

struct dfaNode
{
    string flag;           // 是否包含终态 (+) 或初态 (-)
    set<int> nfaStates;     // 该 DFA 状态包含的 NFA 状态的集合
    map<char, set<int>> transitions; // 字符到下一状态的映射

    dfaNode() {
        flag = "";
    }
};
```

ϵ 闭包计算核心代码 `epsilonClosure` 函数，计算给定 NFA 状态的 ϵ 闭包，使用深度优先搜索遍历所有可通过 ϵ 转移到达的状态：

```
//计算 NFA 状态的  $\epsilon$  闭包，确保正确处理映射字符

set<int> GenerateDFA::epsilonClosure(int id) {

    set<int> eResult{ id };

    stack<int> stack;

    stack.push(id);

    while (!stack.empty())
    {
        int current = stack.top();

        stack.pop();

        // 获取  $\epsilon$  转换

        set<int> eClosure = statusTable[current].m[EPSILON];

        for (auto t : eClosure)
        {
```

```

        if (eResult.find(t) == eResult.end())
        {
            eResult.insert(t);
            stack.push(t);
        }
    }
}

return eResult;
}

```

字符转移闭包计算函数 `otherCharClosure`，计算特定字符下的转移闭包，包括直接字符转移和后续的 ϵ 闭包：

```

// 计算字符转换闭包，确保正确处理映射字符
set<int> GenerateDFA::otherCharClosure(int id, char ch) {

    set<int> otherResult{};

    set<int> processed;

    stack<int> stack;

    stack.push(id);

    while (!stack.empty())
    {
        int current = stack.top();

        stack.pop();

        if (processed.find(current) != processed.end())

            continue;

        processed.insert(current);
    }
}

```

```

        // 获取字符 ch 的转换

        set<int> otherClosure = statusTable[current].m[ch];

        for (auto o : otherClosure)
        {
            auto tmp = epsilonClosure(o);

            otherResult.insert(tmp.begin(), tmp.end());

            stack.push(o);
        }
    }

    return otherResult;
}

```

DFA 构建算法核心代码，通过 NFA2DFA 函数实现，实现了完整的子集构造算法，通过 ϵ 闭包和字符转移闭包计算，将 NFA 转换为等价的 DFA：

```

// NFA 转 DFA 函数（格式化输出）
void GenerateDFA::NFA2DFA(NFA& nfa) {

    // 清空之前的 DFA 数据

    dfaStatusSet.clear();

    dfaTable.clear();

    dfaEndStatusSet.clear();

    dfaNotEndStatusSet.clear();

    dfa2numberMap.clear();

    int dfaStatusCount = 1;

    auto start = nfa.start;                                // 获得 NFA
图的起始位置
    auto startId = start->id;                                // 获得起始

```

编号

```
    dfaNode startDFANode;

    startDFANode.nfaStates = epsilonClosure(startId);           // 初始闭包
    startDFANode.flag = setHasStartOrEnd(startDFANode.nfaStates); // 判断初态终
    态

    deque<set<int>> newStatus {};

    dfa2numberMap[startDFANode.nfaStates] = dfaStatusCount;

    startStaus = dfaStatusCount;

    if (setHasStartOrEnd(startDFANode.nfaStates).find("+") != string::npos) {
        dfaEndStatusSet.insert(dfaStatusCount++);
    }
    else
    {
        dfaNotEndStatusSet.insert(dfaStatusCount++);
    }

    // 对每个字符进行遍历
    for (auto ch : dfaCharSet)
    {
        set<int> thisChClosure {};

        for (auto c : startDFANode.nfaStates)
        {
            set<int> tmp = otherCharClosure(c, ch);

            thisChClosure.insert(tmp.begin(), tmp.end());
        }

        if (thisChClosure.empty())           // 如果这个闭包是空集没必要继续下去了
        {
            continue;
        }
    }
}
```

```

    }

    int presize = dfaStatusSet.size();

    dfaStatusSet.insert(thisChClosure);

    int lastsize = dfaStatusSet.size();

    // 不管一不一样都是该节点这个字符的状态
    startDFANode.transitions[ch] = thisChClosure;

    // 若大小不一样，则证明是新状态
    if (lastsize > presize)
    {
        dfa2numberMap[thisChClosure] = dfaStatusCount;

        newStatus.push_back(thisChClosure);

        if (setHasStartOrEnd(thisChClosure).find("+") != string::npos) {
            dfaEndStatusSet.insert(dfaStatusCount++);
        }
        else
        {
            dfaNotEndStatusSet.insert(dfaStatusCount++);
        }
    }
}

dfaTable.push_back(startDFANode);

// 对后面的新状态进行不停遍历
while (!newStatus.empty())
{
    // 拿出一个新状态
    set<int> ns = newStatus.front();

    newStatus.pop_front();

    dfaNode DFANode;

```

```

DFANode.nfaStates = ns; // 该节点状态集合

DFANode.flag = setHasStartOrEnd(ns);

for (auto ch : dfaCharSet)
{
    set<int> thisChClosure {};

    for (auto c : ns)
    {
        set<int> tmp = otherCharClosure(c, ch);

        thisChClosure.insert(tmp.begin(), tmp.end());
    }

    if (thisChClosure.empty()) // 如果这个闭包是空集没必要继续下去了
    {
        continue;
    }

    int presize = dfaStatusSet.size();

    dfaStatusSet.insert(thisChClosure);

    int lastsize = dfaStatusSet.size();

    // 不管一不一样都是该节点这个字符的状态
    DFANode.transitions[ch] = thisChClosure;

    // 如果大小不一样，证明是新状态
    if (lastsize > presize)
    {
        dfa2numberMap[thisChClosure] = dfaStatusCount;

        newStatus.push_back(thisChClosure);

        if (setHasStartOrEnd(thisChClosure).find("+") != string::npos) {
            dfaEndStatusSet.insert(dfaStatusCount++);
        }
    }
    else

```

```

        {
            dfaNotEndStatusSet.insert(dfaStatusCount++);
        }
    }
    dfaTable.push_back(DFANode);
}

// 输出 DFA 表
printDfaTable(dfaTable);

qDebug() << "NFA 转 DFA 完成！";
}

```

状态属性判断通过 `setHasStartOrEnd` 函数实现，判断状态集合是否包含初态或终态，生成相应的标记字符串：

```

// 辅助函数，判断是否含有初态、终态，含有则返回对应字符串
string GenerateDFA::setHasStartOrEnd(set<int>& statusSet)
{
    string result = "";

    for (const int& element : startNFAsatus)
        if (statusSet.count(element) > 0)
            result += "-";

    for (const int& element : endNFAsatus)
        if (statusSet.count(element) > 0)
            result += "+";

    return result;
}

```

2.5. DFA 最小化

通过等价类划分，将 DFA 状态数最小化，同时保持其语言识别能力不变。

2.5.1 存储结构

输入：DFA 状态转换表、字符集、状态分类集合等

输出：最小化 DFA 状态表

主要存储结构如下表所示：

表 1-5 最小化 DFA 图的主要存储结构

对象或变量名称	功能	存储结构
dfaMinNode	最小化 DFA 节点，包含状态标记、ID 和转移关系	struct dfaMinNode
dfaMinTable	存储最小化后的 DFA 状态表	vector<dfaMinNode>
divideVector	在最小化过程中存储分割的状态集合	vector<set<int>>
dfaMinMap	记录每个状态在 divideVector 中的下标位置	map<int, int>
dfaEndStatusSet	存储 DFA 的终态编号（从 DFA 构建继承）	set<int>
dfaNotEndStatusSet	存储 DFA 的非终态编号（从 DFA 构建继承）	set<int>
dfa2numberMap	状态集合到 DFA 状态编号的映射（从 DFA 构建继承）	map<set<int>, int>
startStatus	记录 DFA 的起始状态编号（从 DFA 构建继承）	int
dfaCharSet	字符集，用于遍历所有可能的输入字符（从 DFA 构建继承）	set<char>

2.5.2 算法实现过程文字描述

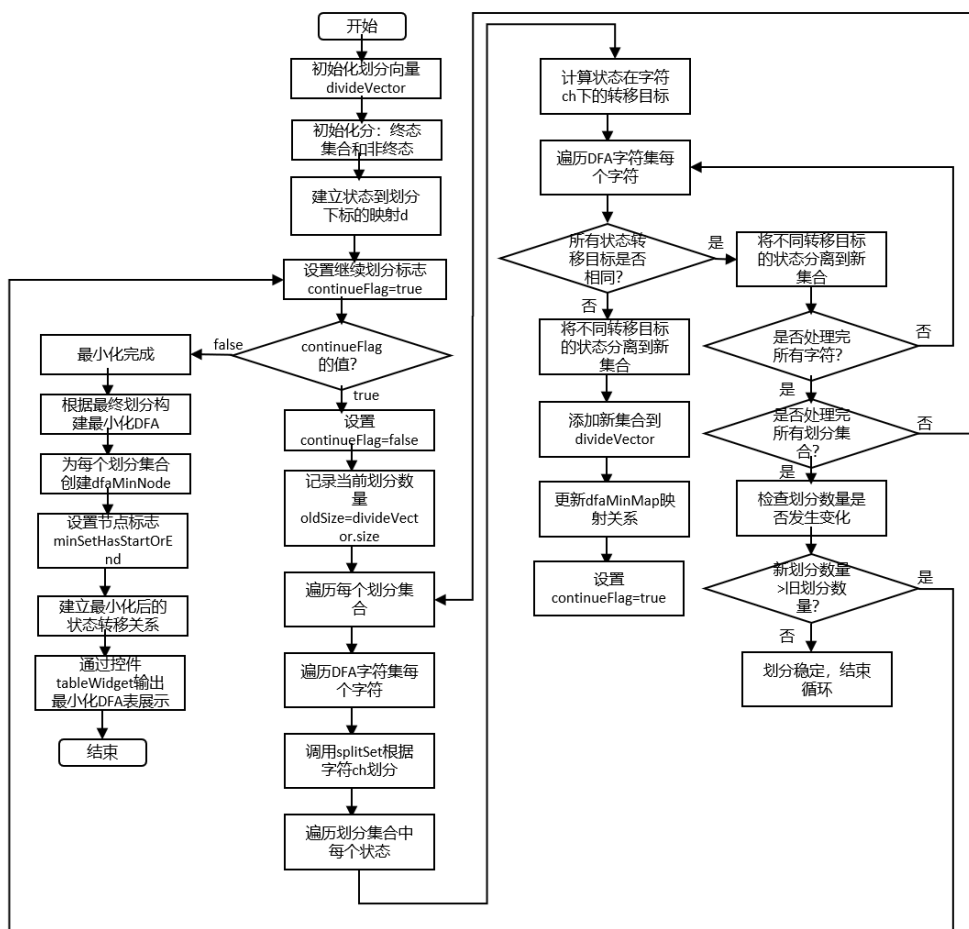
DFA 最小化算法采用等价类划分方法，通过不断分割状态集合直到无法继续分割为止。

算法首先初始化 divideVector，将原始 DFA 的状态划分为两个初始等价类——非终态集合 dfaNotEndStatusSet 和终态集合 dfaEndStatusSet，并建立

状态到分割下标的映射 `dfaMinMap`。然后进入主循环，设置 `continueFlag` 标志位控制循环继续。

在每次循环中，算法遍历当前 `divideVector` 中的每个状态集合和 `dfaCharSet` 中的每个字符，调用 `splitSet` 函数尝试根据字符转移进行分割。`splitSet` 函数检查集合中每个状态在给定字符下的转移目标所在的分组，如果发现转移目标不一致，则将不一致的状态分割出来形成新的等价类。每次循环完成后，检查分组数量是否增加，如果增加则继续循环，否则结束分割过程。最后，根据最终的分割结果构建最小化 DFA 状态表 `dfaMinTable`，其中每个等价类对应一个最小化状态，计算状态标记并建立简化后的转移关系。

2.5.3 算法实现流程图



2.5.4 单元测试

继续基于此前构建的 DFA 图进行最小化 DFA 构建的测试，具体结果如下：



图 1-8 任务一最小化 DFA 图构建的单元测试（简单正则表达式）

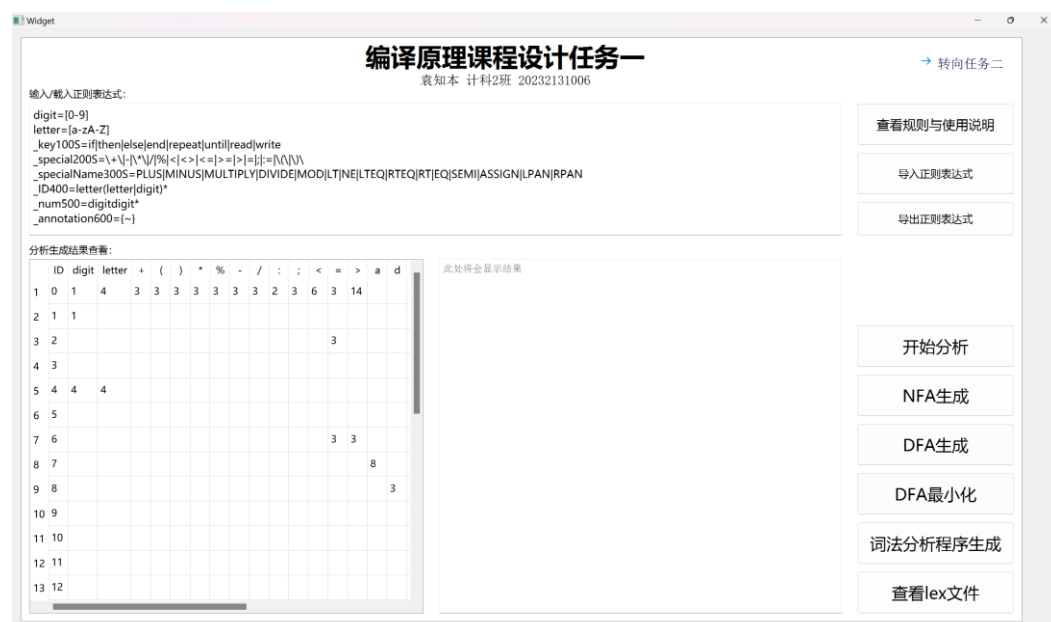


图 1-9 任务一最小化 DFA 图构建的单元测试（TINY 语言）

需注意的是，由于分析结果的最小化 DFA 表大小以及展示框的大小问题，无法完整展示所有的结果，但可以通过滑动条查看，处理得到的最小化

DFA 表完全正确，并且其中的 `digit` 和 `letter` 分别作为了 `[0-9]` 和 `[a-zA-Z]` 冗长的展开式的替代，简化了最小化 DFA 的展示，并未影响到内在的逻辑。

2.5.5 关键源程序代码段

最小化 DFA 核心存储结构：

```
struct dfaMinNode
{
    string flag;           // 是否包含终态 (+) 或初态 (-)
    int id;
    map<char, int> transitions; // 字符到下一状态的映射

    dfaMinNode() {
        flag = "";
    }
};
```

状态分割核心函数 `splitSet`，根据字符 `ch` 对状态集合进行分割，检查集合中每个状态在字符 `ch` 下的转移目标是否属于同一分组，如果不一致则将不一致的状态分割出来形成新的等价类：

```
// 辅助函数，状态分割函数
// 根据字符 ch 将状态集合 node 分成两个子集合
void GenerateDFA::splitSet(int i, char ch)
{
    set<int> result;

    auto& node = divideVector[i];

    int s = -2;

    for (auto state : node)
    {
        int thisNum;
```

```

        if (dfaTable[state - 1].transitions.find(ch) == dfaTable[state - 1].transitions.end())
            thisNum = -1; // 空集
        else {
            // 根据字符 ch 找到下一个状态
            int next_state = dfa2numberMap[dfaTable[state - 1].transitions[ch]];
            thisNum = dfaMinMap[next_state]; // 这个状态的下标是多少
        }

        if (s == -2) // 初始下标
            s = thisNum;

        else if (thisNum != s) // 如果下标不同，就是有问题，需要分出来
            result.insert(state);
    }

    // 删除要删除的元素
    for (int state : result)
        node.erase(state);

    // 都遍历完了，如果 result 不是空，证明有新的，加入 vector 中
    if (!result.empty()) {
        divideVector.push_back(result);
        // 同时更新下标
        for (auto a : result)
            dfaMinMap[a] = divideVector.size() - 1;
    }
}

```

最小化主函数 DFAminimize，实现将状态划分为终态和非终态两个初始等价类，然后通过循环不断分割等价类直至无法继续分割，最后构建最小化

的 DFA 状态表:

```
// DFA 最小化函数

void GenerateDFA::DFAminimize() {

    divideVector.clear();

    dfaMinMap.clear();

    dfaMinTable.clear();

    // 存入非终态、终态集合

    if (dfaNotEndStatusSet.size() != 0)

        divideVector.push_back(dfaNotEndStatusSet);

    // 初始化 map

    for (auto t : dfaNotEndStatusSet)

        dfaMinMap[t] = divideVector.size() - 1;

    divideVector.push_back(dfaEndStatusSet);

    for (auto t : dfaEndStatusSet)

        dfaMinMap[t] = divideVector.size() - 1;

    // 当 flag 为 1 时，一直循环

    int continueFlag = 1;

    while (continueFlag) {

        continueFlag = 0;

        int size1 = divideVector.size();

        for (int i = 0; i < size1; i++)

            // 逐个字符尝试分割状态集合
```

```

        for (char ch : dfaCharSet)

            splitSet(i, ch);

    int size2 = divideVector.size();

    if (size2 > size1)

        continueFlag = 1;
}

for (size_t dfaMinCount = 0; dfaMinCount < divideVector.size(); dfaMinCount++) {

    auto& v = divideVector[dfaMinCount];

    dfaMinNode d;

    d.flag = minSetHasStartOrEnd(v);

    d.id = dfaMinCount;

    // 逐个字符

    for (char ch : dfaCharSet) {

        if (v.size() == 0) {

            d.transitions[ch] = -1;    // 空集特殊判断

            continue;

        }

        int i = *(v.begin()); // 拿一个出来

        if (dfaTable[i - 1].transitions.find(ch) == dfaTable[i - 1].transitions.end()) {

            d.transitions[ch] = -1;    // 空集特殊判断

            continue;

        }

        int next_state = dfa2numberMap[dfaTable[i - 1].transitions[ch]];

        int thisNum = dfaMinMap[next_state];    // 这个状态下标

        d.transitions[ch] = thisNum;

    }

    dfaMinTable.push_back(d);
}

```

```

    }

    // 输出最小化 DFA 表
    printMinimizedDfaTable(dfaMinTable);

    qDebug() << "DFA 最小化完成! ";
}

```

状态属性判断函数 `minSetHasStartOrEnd`, 判断状态集合是否包含起始状态或终止状态, 生成相应的标记字符串用于标识最小化 DFA 状态的特殊属性:

```

// 辅助函数, DFA 最小化状态判断
// 判断是否含有初态终态, 含有则返回对应字符串
string GenerateDFA::minSetHasStartOrEnd(set<int>& statusSet)
{
    string result = "";

    if (statusSet.count(startStaus) > 0) {
        result += "-";
    }

    for (const int& element : dfaEndStatusSet) {
        if (statusSet.count(element) > 0) {
            result += "+";
            break; // 可能会有多个终态同时包含, 但只要一个
        }
    }

    return result;
}

```

2.6. 词法分析代码生成

本模块实现的是可执行词法分析器的完整生成，生成的词法分析代码可以直接编译运行，实现对输入源代码的词法分析。

2.6.1 存储结构

输入：最小化 DFA 状态表 `dfaMinTable`、关键词集合 `keyWords`、操作符映射 `opMap`、字符映射表 `m1`、文件路径 `filePath` 等

输出：完整的 C 语言词法分析器源代码 `lexCode` 和指定路径生成可编译的 C 源文件

主要存储结构如下表所示：

表 1-6 语法分析代码生成的主要存储结构

对象或变量名称	功能	存储结构
<code>dfaNode</code>	DFA 状态节点结构	<code>struct dfaNode</code>
<code>dfaTable</code>	存储完整的 DFA 状态表	<code>Vector<dfaNode></code>
<code>dfaStatusSet</code>	用于 DFA 状态去重，避免重复状态	<code>set<set<int>></code>
<code>dfaEndStatusSet</code>	存储所有 DFA 终态编号	<code>set<int></code>
<code>dfaNotEndStatusSet</code>	存储所有 DFA 非终态编号	<code>set<int></code>
<code>dfa2numberMap</code>	状态集合到 DFA 状态编号的映射	<code>map<set<int>, int></code>
<code>startStatus</code>	记录 DFA 起始状态编号	<code>int</code>
<code>dfaCharSet</code>	存储 DFA 字符集（从 NFA 继承）	<code>set<char></code>

2.6.2 算法实现过程文字描述

完成最小化 DFA 的构建后，可以直接利用这个 DFA 进行词法分析程序的构造，本项目中我采用了代码字符串段拼接的方式予以实现，具体如下：

拼接的代码字符串可以分为多个部分，首先第一部分是构建代码头部，包括必要的头文件引入和全局数据结构定义，而由于 C 语言没有 STL 库，无法使用 `map` 映射，故此处我通过定义结构数组并配合查找函数实现相似功能。

正则表达式中，数字和字符分别用 `digit` 和 `letter` 进行了简化替代，在词法分析程序中需要将其用 `(0|1|2|3|4|5|6|7|8|9)` 和 `(A|B|C|...|X|Y|Z)(a|b|c|...|x|y|z)`

进行替换还原，这将通过 genLexCodeCase 函数实现。

最后处理终态逻辑，生成输出代码，根据识别到的单词类型（关键词、标识符、数字、操作符等）输出对应的编码格式。最终生成 main 函数，完成文件操作和主循环逻辑，生成完整的可编译词法分析器程序。

考虑到该编码的源程序要用在任务二的项目中进行解码后的语义分析，其统一标准如下：

3.任务一的单词编码输出格式问题。

(2)例如，对于 int a=b+1 得到编码应该为这样：

1 101 "a" 200 101 "b" 201 100 1

假设：关键字int的编码是1，标识符的编码是101，整数的编码是100，= 的编码是200，+ 的编码是201

由于标识符以及整数均是大类编码，应该在编码文件中需要提供其对应具体数据。

任务一的词法分析程序是采用编译原理讲稿中的转换方法二方式的，对应示例如下图所示：

输出：生成对应的词法分析程序(方法二)

```
int Judgechar(char ch)
{ if (isalpha(ch)) return 1; else if (isdigit(ch)) return 0;
  return -1; }
char GetNext(){ return buffer[pos++]; }
void GetToken()
{ int state=1; bool done=false;
  while (!done)
  { switch(state)
    { case 1: if (Judgechar(ch)==1)
              { state=2; ch=GetNext(); }
            else { cout<<"Error!";return; }
          case 2: if ((Judgechar(ch)==1) || (Judgechar(ch)==0))
                  ch=GetNext();
                else done=true;
            }
  }
  cout<<"ID";
}
```

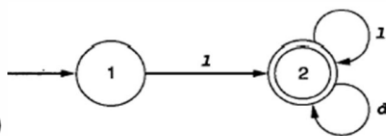
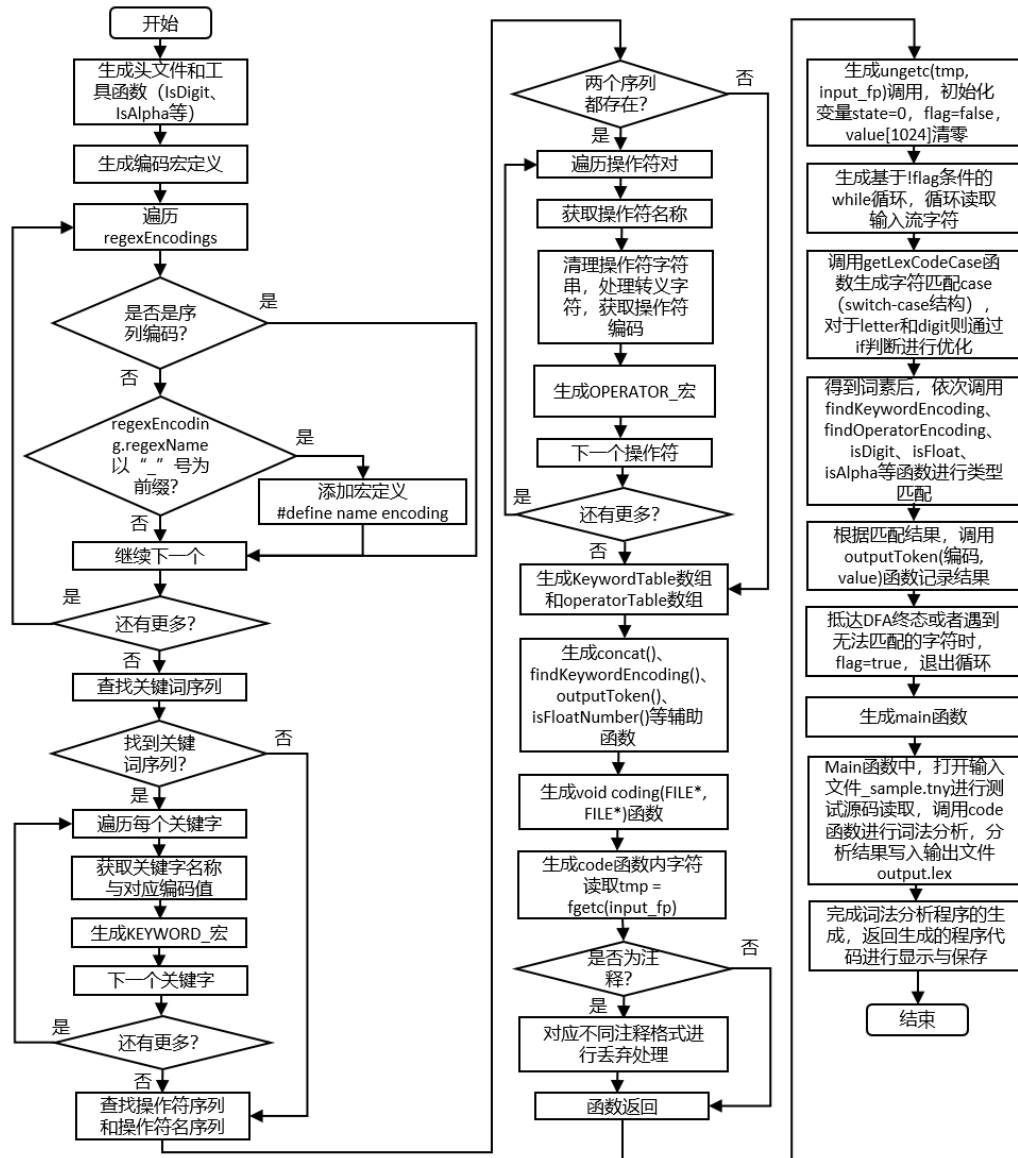


图 1-9 词法分析程序方法二示例图

综上，通过上述方法，我实现了 C 语言书写的词法分析程序的生成。

2.6.3 算法实现流程图



2.6.4 单元测试

在之前生成构建的最小化 DFA 基础上，点击“词法分析程序生成”按钮控件，即可生成展示词法分析程序并保存到指定路径下：



图 1-10 词法分析程序生成测试（简单正则表达式）

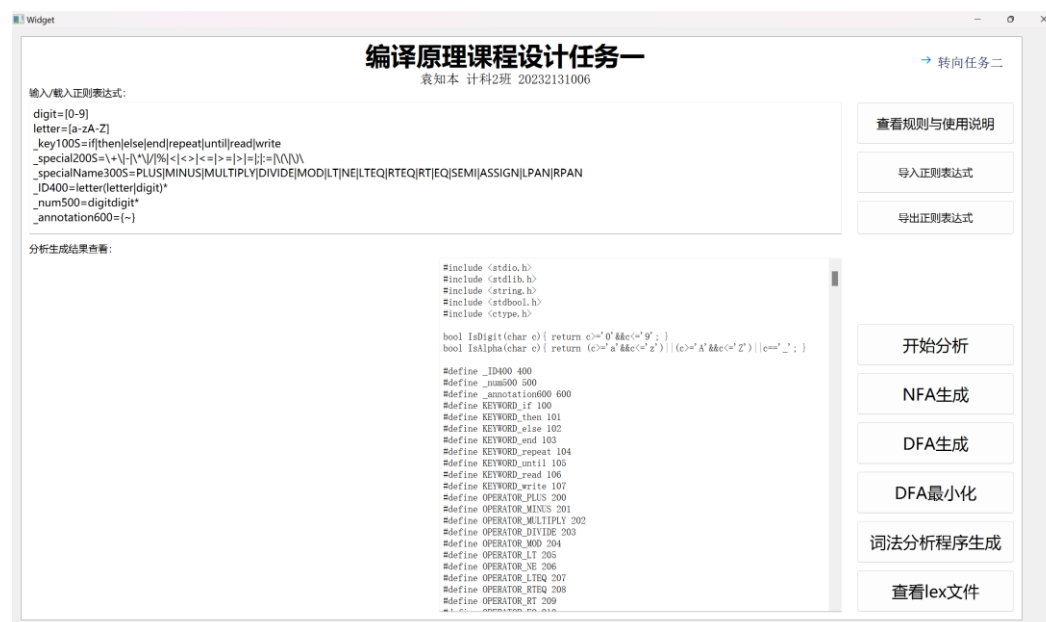


图 1-10 词法分析程序生成测试（TINY 语言）

得到该词法分析程序代码后，通过 cmd 命令行窗口输入命令【gcc -std=c99 _lexer.c -o lexer】对其进行编译，之后把测试用的 tny 文件放到同一文件夹里，运行该可执行文件，跑出 lex 文件，通过点击按钮控件“查看 lex 文件”即可进行查看：



图 1-11 lex 文件查看测试（TINY 语言）



图 1-12 lex 文件查看测试（TINY 语言）

2.6.5 关键源程序代码段

生成完整词法分析器的入口函数 `generateCode`，负责组织整个代码生成过程，包括头文件、数据结构和主函数：

```
// 生成完整词法分析程序  
// 简化只生成必要的宏定义
```

```

QString GenerateCode::generateCode(QString filePath) {

    QString lexCode;

    lexCode += "#include <stdio.h>\n#include <stdlib.h>\n#include <string.h>\n#include
<stdbool.h>\n#include <ctype.h>\n\n";

    lexCode += "bool IsDigit(char c){ return c>='0'&&c<='9'; }\n";

    lexCode += "bool IsAlpha(char c){ return
(c>='a'&&c<='z')||((c>='A'&&c<='Z')||c=='_'); }\n";

    // 输出正则编码宏 - 只输出非序列编码的宏定义

    for (const auto& reg : regexEncodings) {

        // 只输出非序列编码的宏定义（如_ID400, _num500 等）

        if (!reg.isSequence) {

            string cleanName = reg.regexName;

            // 确保宏名是有效的 C 标识符

            if (!cleanName.empty() && cleanName[0] == '_') {

                // 已经是有效的宏名

                lexCode += "#define " + QString::fromStdString(cleanName) + " " +
QString::number(reg.encoding) + "\n";

            }

        }

    }

    // 为关键字和操作符生成简洁的宏定义

    // 1. 关键字宏定义 - 从_key100S 序列获取

    auto key100S_it = sequenceEncodings.find("_key100S");

    if (key100S_it != sequenceEncodings.end()) {

        for (const auto& item : key100S_it->second) {

            string keyword = item.first;

```

```

        int encoding = item.second;

        string macroName = "KEYWORD_" + keyword;

        lexCode += "#define " + QString::fromStdString(macroName) + " " +
QString::number(encoding) + "\n";

    }

}

// 2. 操作符宏定义 - 使用_specialName300S 中的名称和_special200S 中的编码
auto special200S_it = sequenceEncodings.find("_special200S");
auto specialName300S_it = sequenceEncodings.find("_specialName300S");

if (special200S_it != sequenceEncodings.end() && specialName300S_it !=
sequenceEncodings.end()) {

    const auto& special200S_items = special200S_it->second;

    const auto& specialName300S_items = specialName300S_it->second;

    // 假设两个序列的顺序是一致的

    for (size_t i = 0; i < special200S_items.size() && i <
specialName300S_items.size(); i++) {

        string operatorName = specialName300S_items[i].first;

        int encoding = special200S_items[i].second;

        string macroName = "OPERATOR_" + operatorName;

        lexCode += "#define " + QString::fromStdString(macroName) + " " +
QString::number(encoding) + "\n";

    }

}

lexCode += "\n";

```

```

// 输出关键字表 - 直接从_key100S 序列获取，确保编码正确

lexCode += "static struct { const char* keyword; int encoding; } keywordTable[] =
{\n";

if (key100S_it != sequenceEncodings.end()) {

    for (const auto& item : key100S_it->second) {

        string keyword = item.first;

        int encoding = item.second;

        lexCode += "\t{\"\" + QString::fromStdString(keyword) + "\", \" +
QString::number(encoding) + \"},\n";

    }

}

lexCode += "};\n\n";


// 输出操作符表 - 直接从_special200S 序列获取并清理转义字符

lexCode += "static struct { const char* op; int encoding; } operatorTable[] = {\n";

if (special200S_it != sequenceEncodings.end()) {

    for (const auto& item : special200S_it->second) {

        string op = item.first;

        int encoding = item.second;


        // 彻底清理操作符字符串，移除所有转义反斜杠

        string cleanOp;

        for (size_t i = 0; i < op.length(); i++) {

            if (op[i] == '\\') {

                // 跳过转义反斜杠，取下一个字符

                if (i + 1 < op.length()) {

                    i++; // 跳过反斜杠

                    cleanOp += op[i]; // 取转义后的字符

                }

            }

        }

    }

}

}

```

```

        } else {

            cleanOp += op[i];

        }

    }

    // 如果清理后的操作符为空，跳过
    if (cleanOp.empty()) {

        continue;

    }

    // 在 C 字符串中转义特殊字符

    QString escapedOp;

    for (char c : cleanOp) {

        // 对 C 字符串中的特殊字符进行转义

        switch(c) {

            case '\\': escapedOp += "\\\""; break; // 反斜杠
            case '\"': escapedOp += "\\\""; break; // 双引号
            case '\': escapedOp += "\\\""; break; // 单引号
            case '\n': escapedOp += "\\n"; break; // 换行
            case '\t': escapedOp += "\\t"; break; // 制表
            case '\r': escapedOp += "\\r"; break; // 回车
            default: escapedOp += QChar(c); break;

        }

    }

    lexCode += "\t{\" + escapedOp + "\", " + QString::number(encoding) +

    "},\n";

    }

}

```

```

lexCode += "};\n\n";

// 辅助函数 - 添加调试信息

lexCode +=

    "void concat(char str[], char tmp){ size_t len=strlen(str); str[len]=tmp;
str[len+1]='\0'; }\n"

    "int findKeywordEncoding(const char* str){ \n"

    "    printf(\"查找关键字: %s\\n\", str);\n"

    "    for(int i=0;i<sizeof(keywordTable)/sizeof(keywordTable[0]);i++) { \n"

    "        printf(\"    比较: %s vs %s\\n\", str, keywordTable[i].keyword);\n"

    "        if(strcmp(str,keywordTable[i].keyword)==0) { \n"

    "            printf(\"        找到关键字: %s, 编码: %d\\n\", str,
keywordTable[i].encoding);\n"

    "            return keywordTable[i].encoding; \n"

    "        } \n"

    "    } \n"

    "    printf(\"    未找到关键字: %s\\n\", str);\n"

    "    return -1; \n"

    "}\n"

    "int findOperatorEncoding(const char* str){ \n"

    "    printf(\"查找操作符: %s\\n\", str);\n"

    "    for(int i=0;i<sizeof(operatorTable)/sizeof(operatorTable[0]);i++) { \n"

    "        printf(\"    比较: %s vs %s\\n\", str, operatorTable[i].op);\n"

    "        if(strcmp(str,operatorTable[i].op)==0) { \n"

    "            printf(\"        找到操作符: %s, 编码: %d\\n\", str,
operatorTable[i].encoding);\n"

    "            return operatorTable[i].encoding; \n"

    "        } \n"

    "    } \n"

```

```

"    printf("\n  未找到操作符: %s\n", str);\n"

"    return -1; \n"

"}\n"

"void outputToken(FILE* fp,int encoding,const char* val){ \n"

"    if(val&&strlen(val)>0){ \n"

"        fprintf(fp,"%d \\\\"%s\\\" \",encoding,val); \n"

"        printf("\\\"%d \\\\"%s\\\" \",encoding,val); \n"

"    } else { \n"

"        fprintf(fp,"%d \",encoding); \n"

"        printf("\\\"%d \",encoding); \n"

"    } \n"

"}\n"

"int isFloatNumber(const char* str){ for(int i=0;str[i];i++) if(str[i]=='.') return 1;

return 0; }\n\n";

```

// coding 函数 - 添加调试信息

```

lexCode += "void coding(FILE* input_fp, FILE* output_fp){\n"

"    char tmp=fgetc(input_fp);\n"

"if(tmp == EOF) return;\n"

"\n"

"    /* ----- 行注释处理补丁 ----- */\n"

"    if(tmp == '/'){\n"

"        char tmp2 = fgetc(input_fp);\n"

"        if(tmp2 == '/'){\n"

"            char c;\n"

"            while((c = fgetc(input_fp)) != EOF && c != '\n');\n"

"            return;\n"

"        }else{\n"

"            ungetc(tmp2, input_fp);\n"

```

```

        }\n"

    }\n"

"\n"

"    // 跳过空白字符\n"

"    if(tmp==' '||tmp=="\n"||tmp=="\t"||tmp=="\r") { return; }\n"

"    ungetc(tmp,input_fp);\n"

"    \n"

"    int state=0;\n"

"    bool flag=false;\n"

"    bool isIdentifier=false;\n"

"    bool isDigit=false;\n"

"    bool isFloat=false;\n"

"    char value[1024]; value[0]='\0';\n"

"    \n"

"    while(!flag){\n"

"        tmp=fgetc(input_fp);\n"

"        if(tmp == EOF) { flag=true; break; }\n"

"        \n"

"        // 检查字符类型\n"

"        if(tmp >= '0' && tmp <= '9') isDigit = true;\n"

"        else if((tmp >= 'a' && tmp <= 'z') || (tmp >= 'A' && tmp <=
'Z') || tmp == '_') isIdentifier = true;\n"

"        else if(tmp == '.') isFloat = true;\n"

"        \n"

"        printf("\n状态  %d, 读取字符  '%c' (ASCII %d)\n", state,
tmp, tmp);\n"

"        \n"

"        switch(state){\n";

```

```

// 为每个状态生成转移代码 - 修复状态转移逻辑

for (size_t i = 0; i < dfaMinTable.size(); i++) {

    if (!dfaMinTable[i].transitions.empty()) {

        lexCode += "\t\tcase " + QString::number(dfaMinTable[i].id) + ": {\n";

        // 首先处理特殊字符类（数字和字母）

        bool hasDigitTransition = false;

        bool hasLetterTransition = false;

        for (auto& pair : dfaMinTable[i].transitions) {

            char ch = pair.first;

            auto it = m1.find(ch);

            if (it != m1.end()) {

                if (it->second == "digit" || it->second == "num") {

                    hasDigitTransition = true;

                }

                else if (it->second == "letter") {

                    hasLetterTransition = true;

                }

            }

        }

        if (hasDigitTransition) {

            lexCode += "\t\t\t\tif(tmp >= '0' && tmp <= '9') {\n";

            // 找到对应的数字转移

            for (auto& pair : dfaMinTable[i].transitions) {

                char ch = pair.first;

                auto it = m1.find(ch);

                if (it != m1.end() && (it->second == "digit" || it->second ==

```

```

num")) {

    lexCode += "\t\t\t\tstate=" + QString::number(pair.second) +
";\n";

    break;

}

}

lexCode += "\t\t\t\tprintf(\" 数 字 转 移 : %d -> %d\\n\", " +
QString::number(dfaMinTable[i].id) + ", state);\n";

lexCode += "\t\t\t\tbreak;\n";

lexCode += "\t\t\t}\n";

}

if (hasLetterTransition) {

    lexCode += "\t\t\t\tif((tmp >= 'a' && tmp <= 'z') || (tmp >= 'A' && tmp
<= 'Z') || tmp == '_') {\n";

    // 找到对应的字母转移

    for (auto& pair : dfaMinTable[i].transitions) {

        char ch = pair.first;

        auto it = m1.find(ch);

        if (it != m1.end() && it->second == "letter") {

            lexCode += "\t\t\t\t\tstate=" + QString::number(pair.second) +
";\n";

            break;

        }

    }

    lexCode += "\t\t\t\tprintf(\" 字 母 转 移 : %d -> %d\\n\", " +
QString::number(dfaMinTable[i].id) + ", state);\n";

    lexCode += "\t\t\t\tbreak;\n";

    lexCode += "\t\t\t}\n";
}

```

```

    }

    // 处理普通字符 - 修复：包含转义的操作符
    lexCode += "\\t\\tswitch(tmp){\\n";

    QList<QString> tmpList;
    for (auto& pair : dfaMinTable[i].transitions) {
        char ch = pair.first;
        auto it = m1.find(ch);
        if (it != m1.end()) {
            const string& cls = it->second;

            if (cls == "\\n") {
                continue;
            }

            // 真正的字符类，交给 if(tmp>=...) 处理
            if (cls == "digit" || cls == "letter" ||
                cls == "num" || cls == "float") {
                continue;
            }

            // 转义的单字符操作符： \\+ \\* \\( \\)
            if (cls.size() >= 2 && cls[0] == '\\') {
                tmpList.append(QString(ch));
                continue;
            }

            // 其他情况忽略
            continue;
        }
    }

```

```

    }

    tmpList.append(QString(ch));

}

genLexCodeCase(tmpList, lexCode, i, true);

lexCode += "\\t\\tdefault: \\n";

lexCode += "\\t\\t\\tprintf(\"默认转移: 未匹配字符 '%c'\\n\", tmp);\\n";

lexCode += "\\t\\t\\tflag=true; ungetc(tmp,input_fp); break;\\n";

lexCode += "\\t\\t}\\n";

lexCode += "\\t\\tbreak;\\n\\t}\\n";

} else {

    // 没有转移的状态（通常是终态）

    lexCode += "\\t\\tcase " + QString::number(dfaMinTable[i].id) + ":\\n";

    lexCode += "\\t\\t\\tprintf(\"到达终态  %d, 回退字符  '%c'\\n\", state,

tmp);\\n";

    lexCode += "\\t\\t\\tungetc(tmp, input_fp);\\n";

    lexCode += "\\t\\t\\tflag = true;\\n";

    lexCode += "\\t\\t\\tbreak;\\n";

}

}

lexCode += "\\t\\tdefault:\\n";

lexCode += "\\t\\t\\tprintf(\"默认状态: 回退字符 '%c'\\n\", tmp);\\n";

lexCode += "\\t\\t\\tungetc(tmp, input_fp);\\n";

lexCode += "\\t\\t\\tflag = true;\\n";

lexCode += "\\t\\t\\tbreak;\\n";

lexCode += "\\t\\t}\\n"; // 结束 switch(state)

lexCode += "\\t\\tif(!flag) { \\n";

lexCode += "\\t\\t\\tconcat(value,tmp); \\n";

```

```

lexCode += "\t\t\tprintf(\"当前 value: %s\\n\", value);\\n";

lexCode += "\t\t}\\n";

lexCode += "\\t}\\n"; // 结束 while 循环


// 简化终态处理逻辑，先不进行前瞻检查

lexCode += "\t\tprintf(\"词法单元识别完成：value=%s, state=%d\\n\", value,
state);\\n";


lexCode += "\tif(strlen(value) > 0){\\n";

lexCode += "\t\tint enc = -1;\\n";


lexCode += "\t\t// 首先检查是否是关键字\\n";

lexCode += "\t\tint kEnc = findKeywordEncoding(value);\\n";

lexCode += "\t\tif(kEnc != -1) enc = kEnc;\\n";


lexCode += "\t\t// 其次检查是否是操作符\\n";

lexCode += "\t\tint opEnc = findOperatorEncoding(value);\\n";

lexCode += "\t\tif(opEnc != -1) enc = opEnc;\\n";


lexCode += "\t\t// 根据字符类型匹配正则表达式\\n";

lexCode += "\t\tif(enc == -1) {\\n";


lexCode += "\t\t\t// 标识符处理\\n";

lexCode += "\t\t\tif(isIdentifier) {\\n";

lexCode += "\t\t\t\tfor (const auto& reg : regexEncodings) {

\t\t\t\t\tif (!reg.isSequence &&

\t\t\t\t\t\t(reg.regexName.find("ID") != string::npos ||

\t\t\t\t\t\t\treg.regexName.find("id") != string::npos)) {

\t\t\t\t\t\t\tlexCode += "\t\t\t\t\t\t\tenc = " + QString::fromStdString(reg.regexName) +

```

[illegible]

```

lexCode += "\t\t\t\n"; // isDigit

lexCode += "\t\t\n";    // enc == -1


lexCode += "\t\toutputToken(output_fp, enc, value);\n";
lexCode += "\t}\n";
lexCode += "}\n\n";


// main 函数

lexCode += "int main(int argc,char* argv[]){\n"
           "    FILE* input_fp=fopen(\"\" + filePath + \"/_sample.tny\", \"r\");\n"
           "    if(input_fp==NULL){printf(\"Failed to open input\n"
file\\n\");return 1;}\n"
           "    FILE* output_fp=fopen(\"\" + filePath + \"/output.lex\", \"w\");\n"
           "    if(output_fp==NULL){printf(\"Failed to open output\n"
file\\n\");fclose(input_fp);return 1;}\n"
           "    \n"
           "    printf(\"开始词法分析...\\n\");\n"
           "    \n"
           "    char c;\n"
           "    while((c=fgetc(input_fp))!=EOF){ \n"
           "        ungetc(c,input_fp);\n"
           "        printf(\"--- 开始处理下一个词法单元 ---\\n\");\n"
           "        coding(input_fp,output_fp);\n"
           "    }\n"
           "    fprintf(output_fp,\"-1\"); printf(\"\\n 词法分析完成！ \\n\");\n"
fclose(input_fp); fclose(output_fp);\n"
           "    return 0;\n"

```

```
        "}\n";

    return lexCode;
}
```

状态转移代码生成函数 `genLexCodeCase`，负责生成 DFA 状态转移的具体 `case` 代码，处理不同类型的字符（字母、数字、操作符等）并生成相应的状态转移逻辑：

```
// 生成 DFA case 代码
bool GenerateCode::genLexCodeCase(QList<QString> tmpList,
                                   QString& codeStr,
                                   int idx,
                                   bool flag)
{
    for (auto& chStr : tmpList) {
        char ch = chStr.toUtf8().constData()[0];

        if (dfaMinTable[idx].transitions.count(ch) == 0)
            continue;

        int nextState = dfaMinTable[idx].transitions[ch];

        QString caseStr;

        char realChar = 0;

        auto it = m1.find(ch);
        if (it != m1.end()) {
            const string& cls = it->second;
```

```
// 跳过字符类

if (cls == "digit" || cls == "letter" ||

    cls == "num"    || cls == "float") {

    continue;

}


// == 核心修复点 ==

// 只要是转义形式: \+

if (!cls.empty() && cls[0] == '\\' && cls.size() >= 2) {

    realChar = cls[1];    // '+' '*' '(' ')'

}

// 普通单字符

else if (cls.size() == 1) {

    realChar = cls[0];

}

else {

    continue;

}

}

else {

    realChar = ch;

}


// 防御: 绝不生成 '\0'

if (realChar == 0)

    continue;


// 生成合法的 C 字符常量

switch (realChar) {
```

```

        case '\\': caseStr = "\\\"; break;

        case '\\\\': caseStr = "\\\"; break;

        case '\\n': caseStr = "\\n\"; break;

        case '\\t': caseStr = "\\t\"; break;

        case '\\r': caseStr = "\\r\"; break;

        default:

            caseStr = "" + QString(QChar(realChar)) + "";

            break;

    }

    codeStr += "\\t\\tcase " + caseStr + ": ";

    if (flag)

        codeStr += "state=" + QString::number(nextState) + "; ";

    codeStr += "break;\\n";

}

return false;

}

```

3. 系统测试：

详见上述各个模块中的单元测试以及课程设计测试报告。

4. 系统问题分析

- (1) 目前系统图形化窗口界面不够美观，可以进一步进行布局优化；
- (2) 当前的正则表达式输入格式处理的鲁棒性有待提升，非测试中列出的正则表达式可能会出现格式错误等问题，但测试用的正则表达式能全部通过。

5. 使用说明书

编译环境：

Qt Creator 4.11.1 (Community)

Desktop Qt 5.14.2 MinGW 32-bit

编译步骤说明：

- 1、打开 Qt Creator 代码编辑器
- 2、导入所需项目
- 3、点击左下角的“运行”按钮即可完成编译并运行

运行启动系统后，可以看到系统的图形化窗口界面，点击“查看规则与使用说明”按钮可以查看正则表达式输入格式与使用说明注意事项，先输入或通过右侧的“导入正则表达式”按钮，进行正则表达式的输入/导入，之后先点击“开始分析”按钮进行分析，再点击该按钮下面的各个按钮分别查看 NFA 图、DFA 图、最小化 DFA 图、语法分析程序、lex 文件。

具体使用说明参考“可执行程序”文件夹下的使用说明书。

(2) 任务二：

1. 系统总体设计：

在进行任务二的项目设计时，我同样将其分为两个大层次——图形化用户接口层和数据处理层。

图形化用户接口层包括两个模块：I/O 模块（2 型文法和测试源程序代码的输入输出、导入导出）、展示模块（显示 FIRST 集、FOLLOW 集、LR(0)DFA、LR(1)DFA、LR(1)分析表、源程序语法分析表、语法树、中间代码），其中展示模块中 FIRST 集、FOLLOW 集、LR(0)DFA、LR(1)DFA 均通过表格形式进行展示，语法树采用树状形式进行呈现。

数据处理层包含五个模块：文法处理模块、FIRST 集构建生成模块、FOLLOW 集构建生成模块、LR(0)DFA 处理构建模块、SLR(1)文法判断分析模块、LR(1)分析表构建生成模块、LR(1)DFA 处理构建模块、源程序语法分析处理模块、语法树构建生成模块、中间代码生成模块。

各层次和功能模块的如下图所示：

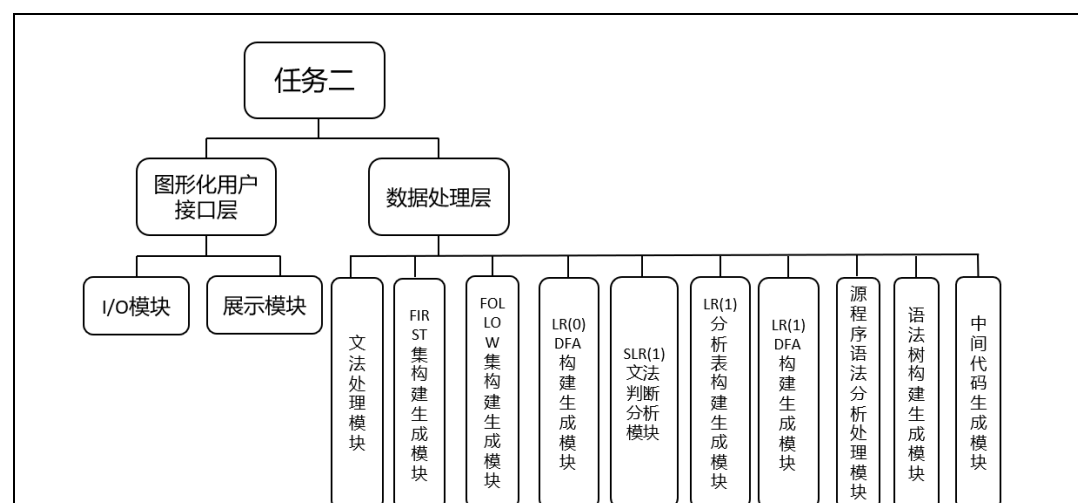


图 2-1 任务二项目的层次架构

2. 功能模块设计与实现：

2.1 图形化用户接口层各模块

(1) 图形化窗口实现

任务二项目通过 QT 实现了图形化窗口界面，通过控件 `plainTextEdit` 提供了文法文本的输入/导入框，通过控件 `plainTextEdit`、`tableWidget` 和 `treeWidget` 实现了文法分析后的相关展示功能（`plainTextEdit` 负责展示文法对照、SLR(1)文法分析结果和生成的中间代码；`tableWidget` 负责以表格形式展示 LR(1)DFA、LR(1)DFA、LR(1)分析表以及 LEX 源代码语法分析过程表；`treeWidget` 负责展示可视化语法树）。

对于用户的交互操作，我通过 QT 的 `pushButton` 按钮控件予以实现，整个系统共拥有 16 个按钮控件，分别实现了文法输入规则查看、分析结果说明、文法的导入、文法的导出、FIRST 集求解、FOLLOW 集求解、LR(0)DFA 生成、LR(1)DFA 生成、LR(1)分析表生成、LEX 输出文件语法分析、语法树生成以及中间代码生成这十二项功能，并将处理的结果通过上述的 `plainTextEdit`、`tableWidget` 以及 `treeWidget` 等控件进行呈现。第十六个按钮实现了任务一和任务二的操作界面切换。

(2) UI 设计

我通过 QT 自带的 UI 设计面板进行任务二项目的图形化窗口界面设计，具体的 UI 设计图如下图所示（由于设计界面的大小问题，未能完全展示整个 UI 设计图，但具体可参考系统运行界面图）：



图 2-2 任务二项目 UI 设计图

项目系统启动运行后，其界面如下图所示：



图 2-3 任务二项目系统运行界面图

(3) 各模块功能实现

图形化窗口界面的展示模块通过 `pushButton` 按钮控件的槽函数触发，并通过 `plainTextEdit` 和 `tableWidget` 控件进行展示，具体的各槽函数定义如下：

```

/*****

* @Copyright (c) 2025 LDL. All rights reserved.

*

* @FileName: widget.h

* @Brief: 应用程序窗口界面声明文件，包含主窗口类，负责交互操作控件定义

* @Module: 主界面模块

*

* @Current Version: 2.0.0

* @Author: 袁知本

* @Created Date: 2025/10/5

*

* @Revision History:

*   Version   Date       Author    Description
*   -----   -
*   1.0.0     2025/10/5   袁知本    初始版本创建
*   2.0.0     2026/1/16   袁知本    整合任务一和任务二的交互控件函数

*****/

#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
#include <QGraphicsScene>
#include <QGraphicsProxyWidget>

QT_BEGIN_NAMESPACE
```

```
namespace Ui { class Widget; }

QT_END_NAMESPACE

class Widget : public QWidget
{
    Q_OBJECT

public:
    Widget(QWidget *parent = nullptr);

    void wrapPage(QWidget *page, QGraphicsView *view);

    bool eventFilter(QObject *watched, QEvent *event);

    ~Widget();

private slots:

    // 查看正则表达式的输入规则说明
    void on_pushButton_Help_clicked();

    // 导入正则表达式
    void on_pushButton_LoadRegex_clicked();

    // 导出正则表达式
    void on_pushButton_SaveRegex_clicked();

    // 正则表达式分析
    void on_pushButton_Analyse_clicked();

    // 生成 NFA
    void on_pushButton_ShowNFA_clicked;
```

// 生成 DFA

void on_pushButton_ShowDFA_clicked();

// DFA 最小化

void on_pushButton_ShowMinDFA_clicked();

// 生成词法分析程序

void on_pushButton_GenerateCode_clicked();

// 查看 LEX 文件

void on_pushButton_LoadLEXFile_clicked();

// 查看文法输入规则说明

void on_pushButton_Help_1_clicked();

// 载入文法规则

void on_pushButton_LoadGrammar_clicked();

// 保存文法规则

void on_pushButton_SaveGrammar_clicked();

// 载入源程序

void on_pushButton_LoadProgram_clicked();

// 保存源程序

void on_pushButton_SaveProgram_clicked();

// 求 First 集

```
void on_pushButton_GetFirstSet_clicked();

// 求 Follow 集

void on_pushButton_GetFollowSet_clicked();

// 分析结果说明

void on_pushButton_Help_2_clicked();

// LR(0)DFA 生成

void on_pushButton_GenLR0DFA_clicked();

// SLR(1) 文法分析

void on_pushButton_SLR1Analyse_clicked();

// LR(1)DFA 生成

void on_pushButton_GenLR1DFA_clicked();

// LR(1) 分析表生成

void on_pushButton_LR1Analyse_clicked();

// 源程序分析

void on_pushButton_AnalyseSentence_clicked();

// 中间代码生成

void on_pushButton_Interlanguage_clicked();

// 可视化显示语法树

void on_pushButton_ShowSyntaxTree_clicked();
```

```
// 转到任务二界面

void on_commandLinkButton_clicked();

// 转到任务一界面

void on_commandLinkButton_2_clicked();

private:
    Ui::Widget *ui;
};

#endif // WIDGET_H
```

2.2. 文法输入/导入与处理

文法输入/导入与处理模块为后续的语法分析提供了规范化、结构化的文法数据基础，确保后续算法能够正确执行。

2.2.1 存储结构

输入：按照指定文法文本格式输入的完整某种文法文本

输出：非终结符集合 bigAlpha、终结符集合 smallAlpha、文法映射表 grammarMap、产生式队列 grammarDeque、产生式编号映射 grammarToInt

主要存储结构如下表所示：

表 2-1 文法输入/导入处理的主要存储结构

对象或变量名称	功能	存储结构
grammarUnit	文法产生式的基本单元，包含左部、右部和唯一标识	struct grammarUnit
bigAlpha	存储所有非终结符的集合	vector<string>
smallAlpha	存储所有终结符的集合	vector<string>
grammarStr	存储用户输入或导入的完整文法文本内容	string
grammarMap	以非终结符为键，对应产生式集合为值的映射表	unordered_map<string, set<string>>
grammarDeque	按顺序存储所有文法产生式的双端队列	deque<grammarUnit>

grammarToInt	产生式（左部+右部）到唯一编号的映射	map<pair<string, string>, int>
startSymbol	记录文法的原始开始符号	string
trueStartSymbol	记录增广处理后的实际开始符号	string
LR0Result	存储 LR(0)分析过程中的提示和结果信息	QString

2.2.2 算法实现过程文字描述

文法处理模块负责文法的载入、解析和预处理操作，为后续的语法分析提供基础的数据结构。其相关的算法实现阐述如下：

对于文法的输入，我有如下的输入格式定义来辅助我更好处理这些输入的文法：

- （1）用#号表示空串
- （2）第一行输入非终结符，用|号分隔
- （3）第二行输入终结符，用|号分隔
- （4）其他行数均为文法，输入文法时，单词间用空格进行分隔
- （5）默认左边出现的第一个字符串为文法的开始符号
- （6）当文法中含有符号|时，要分成两条进行输入

将基于这个输入格式输入的文法通过 `handleGrammar` 函数获取并进行解析，然后进行如下的操作步骤：

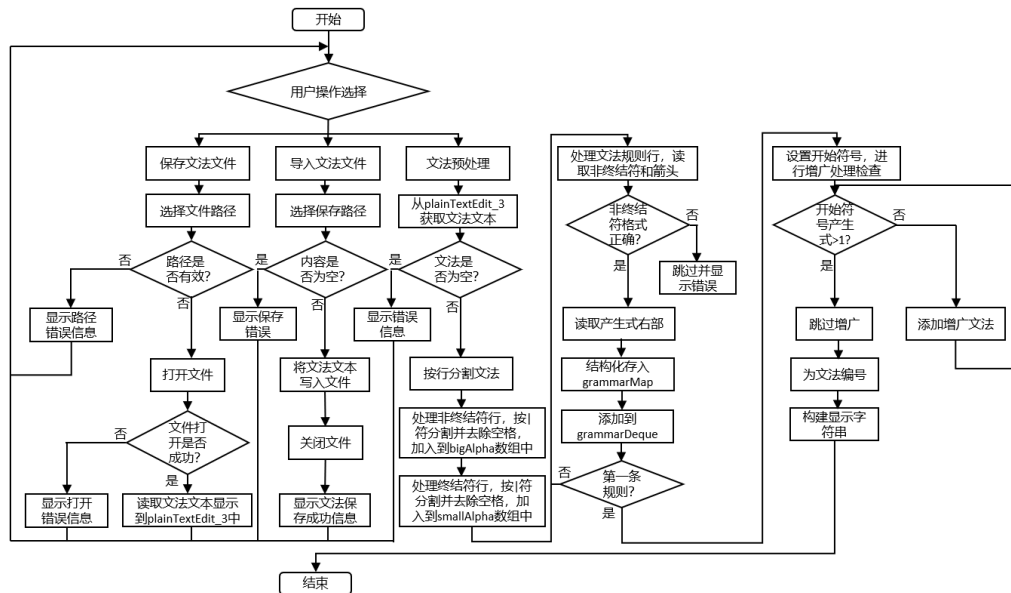
去除空格：第一行、第二行去除空格后分别存入 `bigAlpha` 数组和 `smallAlpha` 数组；

结构化处理：遍历到初始输入文法的第三行，开始处理文法产生式，每行格式均为“非终结符->右部字符串”，系统提取左部非终结符并验证其是否已存在于 `bigAlpha` 数组中，之后跳过箭头读取右部内容。在解析过程中，系统记录第一个出现的非终结符作为文法的开始符号 `startSymbol`，并将每个产生式分别存入 `grammarMap`（用于快速查找）和 `grammarDeque`（保持顺序），完成结构化处理。

增广处理：如果开始符号有多个产生式（大于 1 个），则进行增广处理，在 `grammarDeque` 前端添加新的产生式 `"zengguang -> startSymbol"`，并将 `trueStartSymbol` 设置为 `"zengguang"`。

最后，系统为每个产生式分配唯一编号 `gid`，并建立 `grammarToInt` 映射关系，同时生成 `LR0Result` 字符串记录处理过程和产生式列表，为后续的 `LR(0)`分析做好准备。

2.2.3 算法实现流程图



2.2.4 单元测试

使用对应任务一中简单正则表达式的对应文法进行测试，根据文法输入要求，可以得到对应的文法规则如下：

```

program | TOKEN
FLOAT | ID
program -> TOKEN
TOKEN -> FLOAT
TOKEN -> ID
  
```

使用 `TINY` 语言的文法进行测试，根据文法输入格式要求，可以得到 `TINY` 语言的文法如下：

```

program | stmt-sequence | statement | if-stmt | repeat-stmt | assign-stmt | read-stmt | write-
stmt | exp | simple-exp | comparison-op | addop | term | mulop | factor
SEMI | if | then | end | else | repeat | until | ASSIGN | read | ID | number | write | LT | EQ |
LTEQ | NE | RTEQ | RT | PLUS | MINUS | MULTIPLY | DIVIDE | MOD | LPAN | RPAN
| NUMBER
program -> stmt-sequence
  
```

```

stmt-sequence -> stmt-sequence SEMI statement
stmt-sequence -> statement
statement -> if-stmt
statement -> repeat-stmt
statement -> assign-stmt
statement -> read-stmt
statement -> write-stmt
if-stmt -> if exp then stmt-sequence end
if-stmt -> if exp then stmt-sequence else stmt-sequence end
repeat-stmt -> repeat stmt-sequence until exp
assign-stmt -> ID ASSIGN exp
read-stmt -> read ID
write-stmt -> write exp
exp -> simple-exp comparison-op simple-exp
exp -> simple-exp
comparison-op -> LT
comparison-op -> EQ
comparison-op -> LTEQ
comparison-op -> NE
comparison-op -> RTEQ
comparison-op -> RT
simple-exp -> simple-exp addop term
simple-exp -> term
addop -> PLUS
addop -> MINUS
term -> term mulop factor
term -> factor
mulop -> MULTIPLY
mulop -> DIVIDE
mulop -> MOD
factor -> LPAN exp RPAN
factor -> NUMBER
factor -> ID

```

将该这两个测试文法输入或通过文件导入到任务二系统对应文法输入框处，可以正常进行处理解析并为后续的 LR(0)分析等操作做好准备。

2.2.5 关键源程序代码段

文法解析核心函数 `handleGrammar`，负责将文本格式的文法转换为结构化的内部表示，包括符号分类、产生式解析和增广处理：

```
// 文法初始化处理主函数
```

// 文法处理说明如下:

/*

- * (1) 用 # 号表示空串
 - * (2) 第一行输入非终结符, 用 | 号分隔
 - * (3) 第二行输入终结符, 用 | 号分隔
 - * (4) 其他行均为文法, 输入文法时, 单词间用空格进行分隔
 - * (5) 默认左边出现的第一个字符串为文法的开始符号
 - * (6) 当文法中含有符号 | 时, 要分成两条进行输入
- */

```
void GrammarAnalyse::handleGrammar() {
    vector<string> lines;
    istringstream iss(grammarStr);
    string line;

    // 防止中间有换行符
    while(getline(iss, line))
        if (!line.empty())
            lines.push_back(line);

    // 非终结符
    QString line1 = QString::fromStdString(lines[0]);
    QStringList tokens1 = line1.split("|");

    for (const QString& token : tokens1) {
        // 去除空格
        QString trimmedToken = token.trimmed();
        if (!trimmedToken.isEmpty())
            bigAlpha.push_back(trimmedToken.toStdString());
    }

    // 非终结符
    QString line2 = QString::fromStdString(lines[1]);
    QStringList tokens2 = line2.split("|");

    for (const QString& token : tokens2) {
        // 去除空格
        QString trimmedToken = token.trimmed();
        if (!trimmedToken.isEmpty())
            smallAlpha.push_back(trimmedToken.toStdString());
    }

    for (size_t i = 2; i < lines.size(); i++) {
        string rule = lines[i];
        istringstream ruleStream(rule);
```

```

string nonTerminal;
// 读取非终结符
ruleStream >> nonTerminal;

// 验证非终结符的格式
if (!isBigAlpha(nonTerminal)) {
    QMessageBox::critical(nullptr, "Error", "文法开头必须是非终结符!");
    continue;
}

// 跳过箭头符号 “->”
string arrow;
ruleStream >> arrow;

string rightHandSide;
getline(ruleStream, rightHandSide);

// 去除开头的空格
rightHandSide = rightHandSide.substr(1);

// 若为第一条规则，则认为是开始符号
if (grammarMap.empty()) {
    startSymbol = nonTerminal;
    trueStartSymbol = startSymbol;
}

// 将文法结构化
grammarMap[nonTerminal].insert(QString::fromStdString(rightHandSide).trimmed().toStdString());

// 为 LR(0) 做准备
grammarDeque.push_back(grammarUnit(nonTerminal, rightHandSide));
}

// 增广处理
// 如果开始符号多于 2 个，说明需要增广，为了避免出现字母重复，采用 ^
// 作为增广后的字母，后期输出特殊处理
if (grammarMap[startSymbol].size() > 1) {
    grammarDeque.push_front(grammarUnit("zengguang", startSymbol));
    LR0Result += QString::fromStdString("进行了增广处理\n");

    trueStartSymbol = startSymbol;
}

```

```

// 开始编号
int gid = 0;
for (auto& g : grammarDeque) {
    g.gid = gid++;
    LR0Result += QString::number(g.gid) + QString::fromStdString(":") +
    QString::fromStdString(g.left == "zengguang" ? "E\" : g.left) +
    QString::fromStdString("->") + QString::fromStdString(g.right) + "\n";
    // 存入 map 中
    grammarToInt[make_pair(g.left, g.right)] = g.gid;
}
}

```

符号判断辅助函数 `isBigAlpha` 和 `isSmallAlpha`，分别实现对给定字符串是否属于非终结符集合（`bigAlpha`）或终结符集合（`smallAlpha`）的判断，用于进行文法格式的验证：

```

// 判断字符 c 是否是非终结符
bool GrammarAnalyse::isBigAlpha(string c) {
    for (const auto& symbol : bigAlpha)
        if (symbol == c)
            return true;
    return false;
}

// 判断字符 c 是否是终结符
bool GrammarAnalyse::isSmallAlpha(string c) {
    for (const auto& symbol : smallAlpha)
        if (symbol == c)
            return true;
    return false;
}

```

2.3. FIRST 集构建

2.3.1 存储结构

输入：从文法处理模块处继承获取的 `grammarMap`、`grammarDeque` 等文法数据结构、`bigAlpha` 和 `smallAlpha` 符号分类集合、`startSymbol` 标识文法的开始符号。

输出：FIRST 集合（`firstSets` 映射）

主要存储结构如下表所示：

表 2-2 构建 FIRST 集的主要存储结构

对象或变量名称	功能	存储结构
firstUnit	First 集合单元，包含符号集合和空串标志	struct firstUnit
fisrtSets	存储每个非终结符对应的 First 集合	map<string, firstUnit>
grammarMap	文法产生式映射（从文法模块继承）	unordered_map<string, set<string>>
grammarDeque	文法产生式队列（从文法模块继承）	deque<grammarUnit>
bigAlpha	非终结符集合（从文法模块继承）	vector<string>
smallAlpha	终结符集合（从文法模块继承）	vector<string>
startSymbol	文法的开始符号（从文法模块继承）	string

2.3.2 算法实现过程文字描述

对于规则 $X \rightarrow x_1x_2...x_n$ ， $first(x)$ 的计算算法如下：

```
First(x)={};  
K=1;  
While (k<=n)  
{ if (xk 为终结符号或  $\epsilon$ ) first(xk)=xk;  
  first(x)=first(x)  $\cup$  first(xk) - { $\epsilon$ }  
  If (  $\epsilon \notin first(xk)$  ) break;  
  k++;  
}  
If (k==n+1) first(x)=first(x)  $\cup$   $\epsilon$ 
```

图 2-3 讲稿中 FIRST 集的计算算法

在任务二的项目系统中，我采用了上图中讲稿的算法来实现 FIRST 集的构建，具体阐述如下：

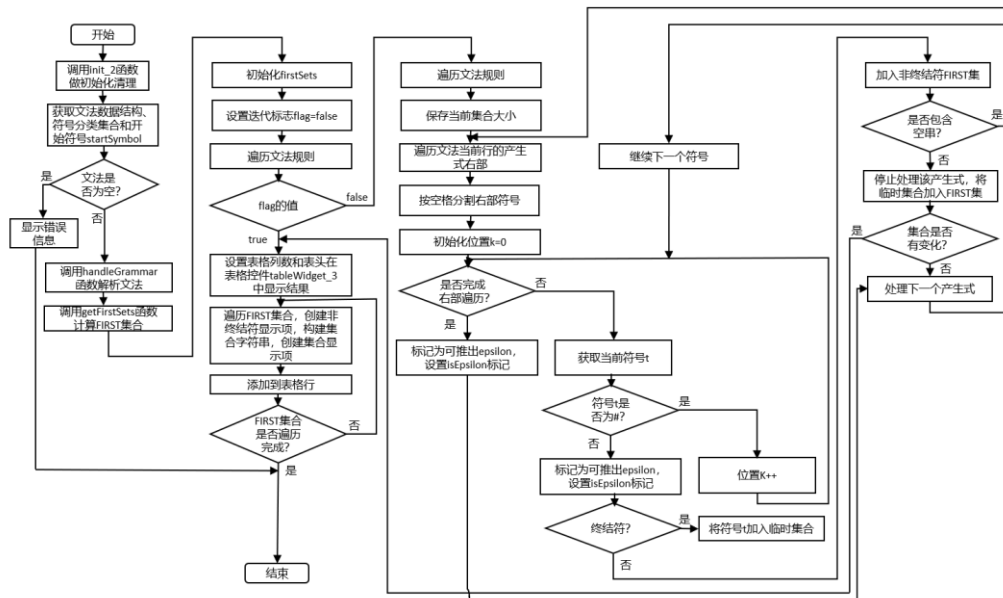
通过递归算法，遍历文法中的每个非终结符，对每个非终结符，遍历其

产生式，而对于每个产生式，找到其每个符号的 FIRST 集合，并将其加入到非终结符的 FIRST 集合中，若发现某个符号的 FIRST 集合包含空串且其后面还有其他符号，则继续处理下一个符号，若某个产生式的所有符号的 FIRST 集合都包含空串，则将空串也加入到该非终结符的 FIRST 集合当中。

在完成上述的递归后，检查原始的 FIRST 集合的大小以及检查其是否包含空串，若与原始的相比发生了变化，则将 flag 设置为 true。

以此类推，遍历直至所有文法遍历完成为止，此时即可完成该文法的 FIRST 集合的构建了。

2.3.3 算法实现流程图



2.3.4 单元测试

基于此前输入使用的两个文法（简单测试文法和 TINY 语言文法），在启动项目系统后，点击图形化窗口界面中的 First 集合求解旁的“求解”按钮，即可进行 FIRST 集的构建并展示在图形化窗口界面对应区域中，供用户查看，具体如下图所示：

First集合求解

求解

	非终结符	First集合
1	TOKEN	FLOAT,ID
2	program	FLOAT,ID

图 2-4 FIRST 集构建测试（简单测试文法）

First集合求解

求解

	非终结符	First集合
1	addop	MINUS,PLUS
2	assign-stmt	ID
3	comparison-op	EQ,LT,LTEQ,N...
4	exp	ID P Δ N N M

图 2-5 FIRST 集构建测试（TINY）

由于显示区域大小问题，构建的 FIRST 集无法显示完全，但是可以通过滑动条实现完整的查看。

2.3.5 关键源程序代码段

FIRST 集合迭代计算核心函数 `calculateFirstSets`，进行单次 First 集合迭代计算，遍历所有产生式，根据符号类型逐步计算 First 集合：

```
// 计算 First 集合，用于单轮 FIRST 推导
// 若任何 FIRST 集发生变化，返回 true，否则返回 false
bool AnalyseFIRST::calculateFirstSets() {
    bool flag = false;
    for (auto& grammar : grammarMap) {
        string nonTerminal = grammar.first;
        // 保存当前 FIRST 集合的大小以用于检查是否发生变化
        size_t originalSize = firstSets[nonTerminal].s.size();
        bool originalE = firstSets[nonTerminal].isEpsilon;
        for (auto& g : grammar.second) {
            QStringList gList = QString::fromStdString(g).split(" ");
            int k = 0;
            while (k <= gList.size() - 1) {
                string t = gList[k].toStdString();
                set<string> first_k;
                if (t == "#") {
                    k++;
                    continue;
                }
                else if (GrammarAnalyse::isSmallAlpha(t))
                    first_k.insert(t);
                else
                    first_k = firstSets[t].s;

                firstSets[nonTerminal].s.insert(first_k.begin(), first_k.end());
                // 若是终结符或没有空串再非终结符中，则直接跳出
                if (GrammarAnalyse::isSmallAlpha(t) || !firstSets[t].isEpsilon)
                    break;
                k++;
            }
            if ((size_t)k == g.size())
                firstSets[nonTerminal].isEpsilon = true;
        }
        // 看原始大小和是否变化 epsilon，如果变化说明要重新再来一次
        if (originalSize != firstSets[nonTerminal].s.size() || originalE !=
            firstSets[nonTerminal].isEpsilon)
            flag = true;
    }
    return flag;
}
```

First 集主计算函数 `getFirstSets`，控制 First 集合的迭代计算过程，直到集合稳定：

```
// First 集合核心计算函数入口
void AnalyseFIRST::getFirstSets() {
    // 不停迭代，直到 FIRST 集合不再发生变化
    bool flag = false;
    do
        flag = calculateFirstSets();
    while (flag);
}
```

2.4. FOLLOW 集构建

2.4.1 存储结构

输入：从文法处理模块处继承获取的 `grammarMap`、`grammarDeque` 等文法数据结构、`bigAlpha` 和 `smallAlpha` 符号分类集合、`startSymbol` 标识文法的开始符号。

输出：FOLLOW 集合（`followSets` 映射）

主要存储结构如下表所示：

表 2-3 构建 FOLLOW 集的主要存储结构

对象或变量名称	功能	存储结构
<code>followUnit</code>	Follow 集合单元，包含符号集合	<code>struct followUnit</code>
<code>followSets</code>	存储每个非终结符对应的 Follow 集合	<code>map<string, followUnit></code>
<code>grammarMap</code>	文法产生式映射（从文法模块继承）	<code>unordered_map<string, set<string>></code>
<code>grammarDeque</code>	文法产生式队列（从文法模块继承）	<code>deque<grammarUnit></code>
<code>bigAlpha</code>	非终结符集合（从文法模块继承）	<code>vector<string></code>
<code>smallAlpha</code>	终结符集合（从文法模块继承）	<code>vector<string></code>
<code>startSymbol</code>	文法的开始符号（从文法模块继承）	<code>string</code>

2.4.2 算法实现过程文字描述

计算Follow集合的算法

1.初始化:

1.1 Follow(开始符号)={ \$ }

1.2 其他任何一个非终结符号A, 则执行 Follow(A)={ }

2.循环: 反复执行

2.1 循环: 对于文法中的每条规则 $A \rightarrow X_1 X_2 \dots X_n$ 都执行

2.1.1 对于该规则中的每个属于非终结符号的 X_i , 都执行

2.1.1.1 把 $\text{First}(X_{i+1} X_{i+2} \dots X_n) - \{\epsilon\}$ 添加到 Follow(X_i)

2.1.1.2 if ϵ in $\text{First}(X_{i+1} X_{i+2} \dots X_n)$, 则把 Follow(A) 添加到 Follow(X_i)

直到任何一个Follow集合的值都没有发生变化为止。

$$A \rightarrow X_1 X_2 \dots X_i X_{i+1} \dots X_n$$

图 2-6 讲稿中 FOLLOW 集的计算算法

在任务二的项目系统中, 我采用了上图中讲稿的算法来实现 FOLLOW 集的构建, 具体阐述如下:

同样使用递归算法进行, 通过递归遍历文法中的每个非终结符, 对每个非终结符, 遍历其产生式, 而对于每个产生式, 遍历其中的每个符号, 若符号为终结符或空串, 则跳过; 若符号是非终结符, 则根据产生式的位置和后续符号来计算 FOLLOW 集合, 此时可以划分为如下的两种情况:

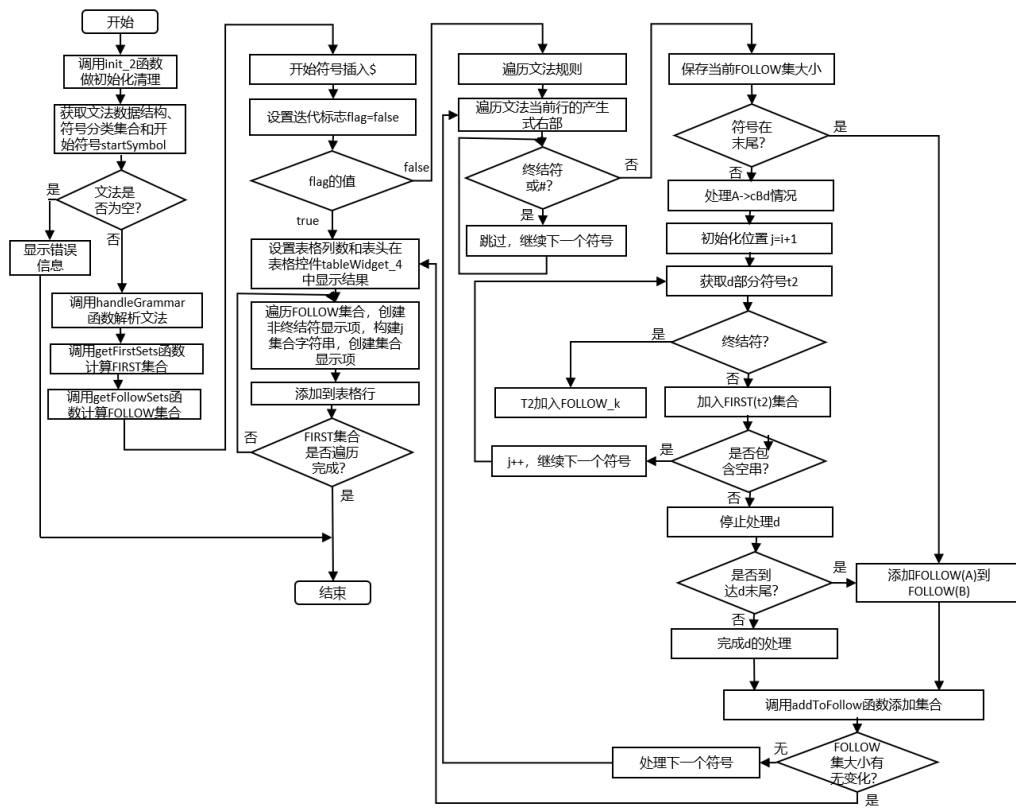
(1) 情况一, $A \rightarrow cBd$, 此时需考虑空串是否包含在 d 的 FIRST 集合中, 若包含则 FOLLOW 集合为 $\text{FIRST}(A) - \{\epsilon\}$; 若不包含则 FOLLOW 集合为 FOLLOW(A);

(2) 情况二, $A \rightarrow cB$, 此时 B 直接继承 A 的 FOLLOW 集, 有 $\text{FOLLOW}(A) = \text{FOLLOW}(B)$ 。

根据对应情况处理完 FOLLOW 集合的计算后, 更新 FOLLOW 集合内容并检查是否发生变化, 变化则设置 $\text{flag} = \text{true}$ 。

以此类推, 遍历直至所有文法遍历完成为止, 此时即可完成该文法的 FOLLOW 集合的构建了。

2.4.3 算法实现流程图



2.4.4 单元测试

基于此前输入使用的两个文法，在启动项目系统后，点击图形化窗口界面中的 Follow 集合求解旁的“求解”按钮，即可进行 FOLLOW 集的构建并展示在图形化窗口界面对应区域中，供用户查看，具体如下图所示：

Follow集合求解

求解

	非终结符	Follow集合
1	TOKEN	\$
2	program	\$

图 2-7 FOLLOW 集构建测试（简单测试文法）

Follow集合求解

求解

	非终结符	Follow集合
1	addop	ID,LPAN,NUM...
2	assign-stmt	\$,SEMI,else,en...
3	comparison-op	ID,LPAN,NUM...
4	exp	\$(LPANSEMI

图 2-8 FOLLOW 集构建测试（TINY）

由于显示区域大小问题，构建的 FOLLOW 集无法显示完全，但是可以通过滑动条实现完整的查看。

2.4.5 关键源程序代码段

FOLLOW 集迭代计算核心函数 `calculateFollowSets`, 单次 Follow 集合迭代计算, 处理两种基本情况, 并考虑后续符号的可空性:

```
// 计算 Follow 集合
bool AnalyseFOLLOW::calculateFollowSets() {
    bool flag = false;
    for (auto& grammar : grammarMap) {
        string nonTerminal = grammar.first;

        for (auto& g : grammar.second) {
            QStringList gList = QString::fromStdString(g).split(" ");
            for (int i = 0; i < gList.size(); ++i) {
                string t = gList[i].toStdString();
                if (GrammarAnalyse::isSmallAlpha(t) || t == "#")
                    continue;    // 跳过终结符
                set<string> follow_k;
                size_t originalSize = followSets[t].s.size();

                if (i == gList.size() - 1)
                    // Case A: A ->  $\alpha$ B, add Follow(A) to Follow(B)
                    follow_k.insert(followSets[nonTerminal].s.begin(),
followSets[nonTerminal].s.end());
                else {
                    // Case B: A ->  $\alpha$ B $\beta$ 
                    int j = i + 1;
                    while (j < gList.size()) {
                        string t2 = gList[j].toStdString();
                        if (GrammarAnalyse::isSmallAlpha(t2)) {
                            // 若为终结符则直接加入并跳出循环
                            follow_k.insert(t2);
                            break;
                        }
                    }
                    else {
                        // 若为非终结符则加入 FIRST 集合
                        set<string> first_beta = firstSets[t2].s;
                        follow_k.insert(first_beta.begin(), first_beta.end());

                        // 若没有空串在 FIRST 集合中, 则停止
                        if (!firstSets[t2].isEpsilon)
                            break;
                        ++j;
                    }
                }
            }
        }
    }
}
```

```

        // 若  $\beta$  是空串 ( $\epsilon$ ) 或者  $\beta$  是 all nullable, add Follow(A)
        to Follow(B)
        if (j == gList.size())
            follow_k.insert(followSets[nonTerminal].s.begin(),
followSets[nonTerminal].s.end());
        }

        addToFollow(t, follow_k);
        // 检查是否发生变化
        if (originalSize != followSets[t].s.size())
            flag = true;
    }
}

return flag;
}

```

FOLLOW 集主计算函数 `getFollowSets`, 初始化开始符号的 FOLLOW 集, 并控制迭代计算过程:

```

// Follow 集合核心计算函数入口
void AnalyseFOLLOW::getFollowSets() {
    // 开始符号加入 $ 符号
    addToFollow(startSymbol, { "$" });

    // 不停迭代, 直到 FOLLOW 集合不再发生变化
    bool flag = false;
    do
        flag = calculateFollowSets();
    while (flag);
}

```

2.5. LR(0)DFA 图生成

LR(0)DFA 构建模块实现了完整的 LR(0)自动机构建算法, 为后续的语法分析表构建和冲突检测等提供了基础数据结构。

2.5.1 存储结构

输入: 文法结构 (`grammarMap`、`grammarDeque`、`grammarToInt`)、符号分类 (`bigAlpha` 和 `smallAlpha`)、开始符号 (`startSymbol` 和 `trueStartSymbol`)

输出：DFA 状态集合（dfaStateVector）、项目集合（dfaCellVector 包含所有 LR(0)项目）、符号集合（VN 和 VT 记录 DFA 中出现的非终结符和终结符）

主要存储结构如下表所示：

表 2-4 构建 LR(0)DFA 图的主要存储结构

对象或变量名称	功能	存储结构
dfaCell	DFA 项目结构，表示一个 LR(0)项（产生式+点的位置）	struct dfaCell
dfaState	DFA 状态结构，包含项目集合和状态转移信息	struct dfaState
nextStateUnit	状态转移单元，记录字符和转移目标	struct nextStateUnit
scnt	状态编号计数器，确保每个状态有唯一 ID	int
ccnt	项目编号计数器，确保每个项目有唯一 ID	int
dfaCellVector	存储所有 DFA 项目的容器	vector<dfaCell>
dfaStateVector	存储所有 DFA 状态的容器	vector<dfaState>
visitedStates	DFS 遍历时记录已访问的状态 ID	set<int>
VN	在 DFA 构建过程中收集到的非终结符	set<string>
VT	在 DFA 构建过程中收集到的终结符	set<string>
grammarMap	文法产生式映射（继承）	unordered_map<string, set<string>>
grammarDeque	文法产生式队列（继承）	deque<grammarUnit>
grammarToInt	产生式到编号的映射（继承）	map<pair<string, string>, int>

2.5.2 算法实现过程文字描述

构建 LR(0)DFA 的整体算法实现详细阐述如下：

生成过程中，首先要对第一条文法生成 LR(0)状态，而由于增广文法的存在，即可知只会有一个入口，也即有唯一确定真初始状态。此步操作将通过 createFirstState 函数实现。

LR(0)DFA 的初始状态建立完成后，可以通过递归函数生成其他各状态，

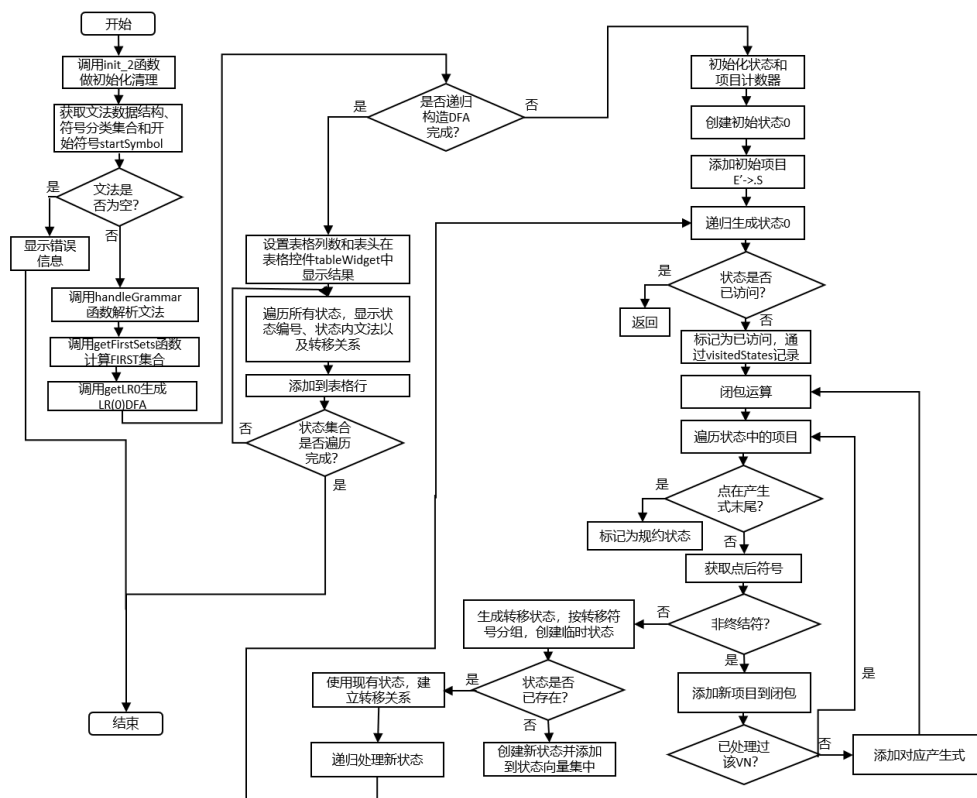
该递归算法类似于 DFS 遍历，需要对状态是否到达过进行判断（通过 `visitedStates` 进行记录），若到过则不必再递归下去，执行回退。

到达某个未到达过的状态后，需要借助闭包计算函数 `closure` 和 `GOTO` 运算函数，前者确定 LR(0) 每个项目集中的项目，将所有可通过闭包运算扩展的项目都加入到项目集中；后者计算 LR(0) 项目在某个符号 `X` 上的 `GOTO`，获取通过某个符号到达的新项目集，得到所有项目集后，将其转换为 DFA 图。

完成一个状态的所有求解后，重新遍历该状态的所有文法，生成新的状态，之后要通过 `isNewState` 函数检验是否是重复的状态（项目检测同理），若是重复的则当前状态在该项目中前进转移的是回到 `dfaStateVector` 中的某个已存在的状态上，而非新状态，只有确定是新状态后，才进行对 `dfaStateVector` 的 `push` 的更新操作。

以此类推，通过整体的递归算法，完成所有状态的构建和转移连接，即可完成 LR(0) DFA 图的构建。

2.5.3 算法实现流程图



2.5.4 单元测试

基于此前使用的两个文法（简单测试文法和 TINY 文法），启动项目系统后，输入文法后，点击“LR(0)DFA 生成”按钮，即可在下方的“分析结果表”内生成 LR(0)DFA 图（表格形式），同时可以查看上方的“对应文法”，来核对用户输入的文法是否一致，具体如下图所示：

查看分析说明

LR(0)DFA生成

SLR(1)文法分析

文法分析

LR(1)DFA生成

LR(1)分析表生成

源代码分析

0:program->TOKEN
1:TOKEN->FLOAT
2:TOKEN->ID

分析结果表

	状态	状态内文法	FLOAT	ID	TOKEN
1	0	program->.TOKEN TOKEN->.FLOAT TOKEN->.ID	1	2	3
2	1	TOKEN->FLOAT.			
3	2	TOKEN->ID.			
4	3	program->TOKEN.			

图 2-9 LR(0)DFA 图构建测试（简单测试文法）

[查看分析说明](#)
[LR\(0\) DFA生成](#)
[SLR\(1\) 文法分析](#)

文法分析

[LR\(1\) DFA生成](#)
[LR\(1\) 分析表生成](#)
[源代码分析](#)

```

0:program->stmt-sequence
1:stmt-sequence->stmt-sequence SEMI statement
2:stmt-sequence->statement
3:statement->if-stmt
4:statement->repeat-stmt
5:statement->assign-stmt
6:statement->read-stmt
7:statement->write-stmt
8:if-stmt->if exp then stmt-sequence end
9:if-stmt->if exp then stmt-sequence else stmt-sequence end
10:repeat-stmt->repeat-stmt until condition
    
```

分析结果表

	状态	状态内文法	ASSIGN	VII	EQ	ID	LP
1	0	program->.stmt-...				1	
2	1	assign-stmt-...	13				
3	2	statement->assign-stmt.					
4	3	if-stmt->if.expthenstmt-...				14	15
5	4	statement->if-stmt.					
6	5	read-stmt->read.ID				43	
7	6	statement->read-stmt.					
8	7	repeat-stmt-...				1	
9	8	statement->repeat-stmt.					
10	9	stmt-sequence-...					
11	10	program->stmt-...					

由于显示区域大小问题，构建的 LR(0)DFA 图（表格形式）无法显示完全，但是可以通过滑动条实现完整的查看。

初始状态创建函数 `createFirstState`，用于创建 LR(0)DFA 的初始状态，包含增广文法的起始项目：

```

// utils: 创建 LR(0)初始状态
void createFirstState()
{
    // 由于增广，一定会只有一个入口
    dfaState zero = dfaState();
    zero.sid = scnt++; // 给他一个 id
    dfaStateVector.push_back(zero); // 放入数组中

    // 添加初始的 LR0 项，即 E' -> .S
    dfaCell startCell;
    startCell.gid = 0; // 这里假设增广文法的编号为 0
    startCell.index = 0;
    startCell.cellid = ccnt++;

    dfaCellVector.push_back(startCell);

    // 把初始 LR0 项放入初始状态
    dfaStateVector[0].cellV.push_back(startCell.cellid);
    dfaStateVector[0].originV.push_back(startCell.cellid);
}

```

LR(0)状态生成的核心函数 `generateLR0State`，实现闭包计算、状态转移和递归构建：

```

// 递归生成 LR(0) 状态
void GenerateLR0DFA::generateLR0State(int stateId) {
    // 采用 DFS 算法
    if (visitedStates.count(stateId) > 0)
        return ;

    // 若标记已走过
    visitedStates.insert(stateId);

    qDebug() << stateId << endl;

    // 求闭包
    for (size_t i = 0; i < dfaStateVector[stateId].cellV.size(); ++ i) {
        dfaCell& currentCell = dfaCellVector[dfaStateVector[stateId].cellV[i]];

        qDebug() << QString::fromStdString(grammarDeque[currentCell.gid].left) <<
        QString::fromStdString("->") <<
        QString::fromStdString(grammarDeque[currentCell.gid].right) << endl;
        qDebug() << "current index:" << currentCell.index << endl;
    }
}

```

```

        QStringList rightList =
QString::fromStdString(grammarDeque[currentCell.gid].right).split(" ");

        // 若点号在产生式末尾或者空串，则跳过（LR0 不需要结束）
        if (currentCell.index == rightList.size() || grammarDeque[currentCell.gid].right
== "#") {
            dfaStateVector[stateId].isEnd = true;
            continue;
        }

        string nextSymbol = rightList[currentCell.index].toStdString();

        // 如果 nextSymbol 是非终结符，则将新项添加到状态中
        if (GrammarAnalyse::isBigAlpha(nextSymbol) &&
dfaStateVector[stateId].right_VNs.find(nextSymbol) ==
dfaStateVector[stateId].right_VNs.end()) {
            dfaStateVector[stateId].right_VNs.insert(nextSymbol);
            for (auto& grammar : grammarMap[nextSymbol]) {
                // 获取通过 nextSymbol 转移的新 LR0 项
                dfaCell nextCell = dfaCell();
                nextCell.gid = grammarToInt[make_pair(nextSymbol, grammar)];
                nextCell.index = 0;
                int nextcellid = isNewCell(nextCell.gid, nextCell.index);
                if (nextcellid == -1) {
                    nextCell.cellid = ccnt++;
                    dfaCellVector.push_back(nextCell);
                    dfaStateVector[stateId].cellV.push_back(nextCell.cellid);
                }
                else
                    dfaStateVector[stateId].cellV.push_back(nextcellid);
            }
        }
    }

    // 暂存新状态
    map<string, dfaState> tempSave;
    // 生成新状态，但还不能直接存到 dfaStateVector 中，我们要校验他是否和之
    前的状态一样
    for (size_t i = 0; i < dfaStateVector[stateId].cellV.size(); ++i) {
        dfaCell& currentCell = dfaCellVector[dfaStateVector[stateId].cellV[i]];
        QStringList rightList =
QString::fromStdString(grammarDeque[currentCell.gid].right).split(" ");

        // 如果点号在产生式末尾，则跳过（LR0 不需要结束）

```

```

    if (currentCell.index == rightList.size() || grammarDeque[currentCell.gid].right
== "#")
        continue;

    // 下一个字符
    string nextSymbol = rightList[currentCell.index].toString();

    // 创建下一个状态（临时的）
    dfaState& nextState = tempSave[nextSymbol];
    dfaCell nextStateCell = dfaCell();
    nextStateCell.gid = currentCell.gid;
    nextStateCell.index = currentCell.index + 1;

    // 看项目是否有重复的，若重复则取之前的，不重复生成
    int nextStateCellid = isNewCell(nextStateCell.gid, nextStateCell.index);
    if (nextStateCellid == -1) {
        nextStateCell.cellid = cnt ++;
        dfaCellVector.push_back(nextStateCell);
    }
    else
        nextStateCell.cellid = nextStateCellid;
    nextState.cellV.push_back(nextStateCell.cellid);
    nextState.originV.push_back(nextStateCell.cellid);

    // 收集后方便后续的通过表格形式构建 LR(0)DFA
    if (GrammarAnalyse::isBigAlpha(nextSymbol))
        VN.insert(nextSymbol);
    else if (GrammarAnalyse::isSmallAlpha(nextSymbol))
        VT.insert(nextSymbol);
}

// 校验是否存在重复状态
for (auto& t : tempSave) {
    dfaState nextState = dfaState();
    int newStateId = isNewState(t.second.originV);

    // 不重复就新开一个状态
    if (newStateId == -1) {
        nextState.sid = cnt++;
        nextState.cellV = t.second.cellV;
        nextState.originV = t.second.originV;
        dfaStateVector.push_back(nextState);
    }
    else

```

```

        nextState.sid = newStateId;

        // 存入现在这个状态的 nextStateVector
        nextStateUnit n = nextStateUnit();
        n.sid = nextState.sid;
        n.c = t.first;
        dfaStateVector[stateId].nextStateVector.push_back(n);
    }

    // 对每个下一状态进行递归
    int nsize = dfaStateVector[stateId].nextStateVector.size();
    for (int i = 0; i < nsize; i++) {
        auto& nextunit = dfaStateVector[stateId].nextStateVector[i];
        generateLR0State(nextunit.sid);
    }
}

```

状态去重检查函数 `isNewState`，检查状态的项目集合是否已存在，用于状态去重：

```

// 判断是否为新的状态
int GenerateLR0DFA::isNewState(const vector<int>& cellIds) {
    for (const dfaState& state : dfaStateVector)
        // 检查状态中的 originV 是否相同
        if (state.originV.size() == cellIds.size() && equal(state.originV.begin(),
state.originV.end(), cellIds.begin()))
            // 若不是新状态
            return state.sid;
    // 若是新状态
    return -1;
}

```

项目去重检查函数 `isNewCell`，用于检查 LR(0)项目是否已经存在，避免项目的重复创建：

```

// 判断是否为新的项目
int GenerateLR0DFA::isNewCell(int gid, int index) {
    for (const dfaCell& cell : dfaCellVector)
        // 检查 dfaCellVector 中是否存在相同的 gid 和 index 的 dfaCell
        if (cell.gid == gid && cell.index == index)
            // 若不是新项目
            return cell.cellid;
    // 若是新项目
}

```

```
return -1;
}
```

2.6. SLR(1)文法判断

SLR(1)文法判断分析模块为编译器设计提供了重要的文法性质判断能力，帮助用户识别其输入的文法的分析表冲突，为后续的语法分析器生成提供基础。

2.6.1 存储结构

输入：LR(0)DFA（dfaStateVector 和 dfaCellVector）、FOLLOW 集（followSets）、文法结构（grammarMap、grammarDeque 等）

输出：SLR(1)文法判断分析结果（纯文本，包含判断结果和原因分析）

主要存储结构如下表所示：

表 2-5 进行 SLR(1)文法判断的主要存储结构

对象或变量名称	功能	存储结构
dfaStateVector	存储 LR(0)自动机的所有状态（从 LR(0)模块继承）	vector<dfaState>
dfaCellVector	存储 LR(0)自动机的所有项目（从 LR(0)模块继承）	vector<dfaCell>
firstSets	存储非终结符的 First 集合（从 First/Follow 模块继承）	map<string, firstUnit>
followSets	存储非终结符的 Follow 集合（从 First/Follow 模块继承）	map<string, followUnit>
grammarMap	文法产生式映射（从文法模块继承）	unordered_map<string, set<string>>
grammarDeque	文法产生式队列（从文法模块继承）	deque<grammarUnit>
isSpecial	冲突标记，通过 dfaState 的 isSpecial 字段标记存在冲突的状态	bool isSpecial

2.6.2 算法实现过程文字描述

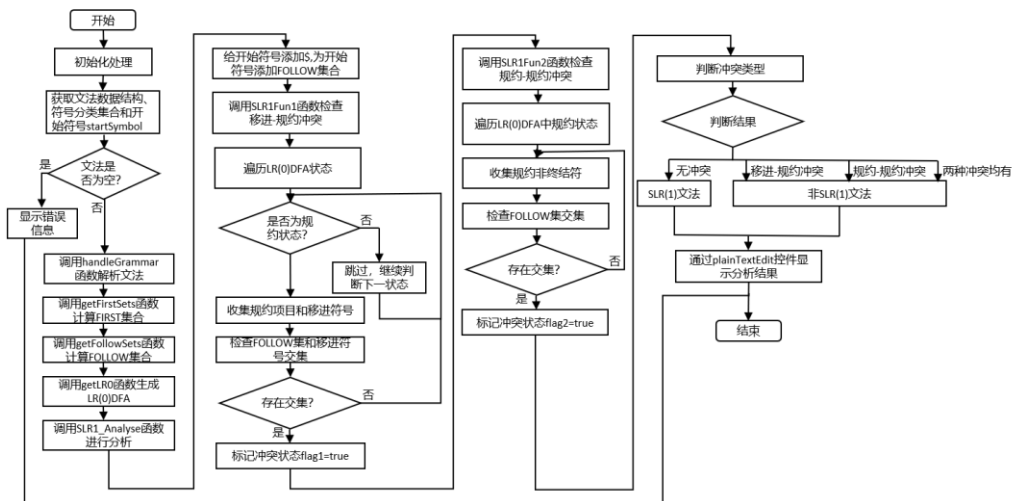
SLR(1)文法分析算法基于已构建的 LR(0)自动机和计算得到的 Follow 集合，检查文法是否存在移进-规约冲突和规约-规约冲突。首先明确两种冲突的判断方式：

(1) 移进-规约冲突：当状态中某个终结符 v 既出现在移进项目的点后，又出现在某个规约项目左部非终结符的 Follow 集合中；

(2) 规约-规约冲突：当状态中存在多个规约项目，且它们左部非终结符的 Follow 集合有交集。

若出现上述的任何一个情况或均出现，则具有冲突，否则符合 SLR(1) 文法。

2.6.3 算法实现流程图



2.6.4 单元测试

依旧使用 TINY 语言的文法继续测试，启动任务二项目系统后，输入 TINY 文法，点击项目界面上方的“SLR(1)文法分析”按钮，即可开始 SLR(1) 文法分析。

本次测试中，TINY 语言的文法是 SLR(1) 文法，故能通过 SLR(1) 文法判断，同时我也通过局部修改 TINY 语言的文法来进行具有冲突情况的测试，具体操作如下：

(1) 新增了非终结符 `primary`，并让 `term -> primary` 与已有的 `term -> factor` 并存，而 `primary` 与 `factor` 都能直接归约为 `NUMBER` 或 `ID`，由此构造了具有“规约-规约冲突”的文法；

(2) 引入了经典的“悬挂 else”二义性，构造具有“移进-规约冲突”的文法；

(3) 把(1)和(2)的内容相结合，构造出同时存在“移进-规约冲突”和“规约-规约冲突”的文法。

将这四个文法进行测试，具体测试结果如下：

查看分析说明

LR(0) DFA生成

SLR(1) 文法分析

文法分析

LR(1) DFA生成

LR(1) 分析表生成

源代码分析

该文法不是SLR(1) 文法
原因：存在规约-规约冲突
在部分DFA状态中，存在多个可用的规约项目，且它们的Follow集合有交集

查看分析说明

LR(0) DFA生成

SLR(1) 文法分析

文法分析

LR(1) DFA生成

LR(1) 分析表生成

源代码分析

该文法不是SLR(1) 文法
原因：存在移进-规约冲突
在部分DFA状态中，同一个终极符既可以移进也可以规约

查看分析说明

LR(0) DFA生成

SLR(1) 文法分析

文法分析

LR(1) DFA生成

LR(1) 分析表生成

源代码分析

该文法不是SLR(1) 文法
原因：同时存在移进-规约冲突和规约-规约冲突



图 2-11、2-12、2-13、2-14 SLR(1)文法判断测试

2.6.5 关键源程序代码段

移进-规约冲突检查函数 SLR1Fun1，检查 SLR(1)文法中的移进-规约冲突，即同一状态下既可移进又可规约的冲突情况：

```
// 检查“移进-规约”冲突
bool SLR1Analyse::SLR1Fun1() {
    bool flag = false;
    for (dfaState& state : dfaStateVector) {
        // 规约项目的左边集合
        set<string> a;
        // 终结符
        set<string> rVT;
        // 不是规约状态不考虑
        if (!state.isEnd) continue;
        // 规约状态
        for (int cellid : state.cellV) {
            // 拿到这个 cell
            const dfaCell& cell = dfaCellVector[cellid];
            // 获取文法
            const grammarUnit gm = grammarDeque[cell.gid];

            QStringList rightList =
                QString::fromStdString(grammarDeque[cell.gid].right).split(" ");
            // 判断是不是规约项目
            if (cell.index == rightList.size() || gm.right == "#")
                a.insert(gm.left);
            // 判断是不是终结符
            else
                if
                    (GrammarAnalyse::isSmallAlpha(rightList[cell.index].toStdString()))
                        rVT.insert(rightList[cell.index].toStdString());
```

```

    }
    for (string c : a)
        for (string v : rVT)
            if (followSets[c].s.find(v) != followSets[c].s.end()) {
                flag = true;
                state.isSpecial = true;
            }
    }
    return flag;
}

```

规约-规约冲突检查函数 SLR1Fun2，检查 SLR(1)文法中的规约-规约冲突，即同一状态下有多个可用的规约产生式：

```

// 检查“规约-规约”冲突
bool SLR1Analyse::SLR1Fun2() {
    for (const auto& state : dfaStateVector) {
        // 规约项目的左边集合
        set<string> a;
        // 不是规约状态不考虑
        if (!state.isEnd) continue;

        // 规约状态
        for (int cellid : state.cellV) {
            // 拿到这个 cell
            const dfaCell& cell = dfaCellVector[cellid];
            // 获取文法
            const grammarUnit gm = grammarDeque[cell.gid];

            QStringList rightList =
                QString::fromStdString(grammarDeque[cell.gid].right).split(" ");
            // 判断是不是规约项目
            if (cell.index == rightList.size() || gm.right == "#")
                a.insert(gm.left);
        }

        for (string c1 : a)
            for (string c2 : a)
                if (c1 != c2) {
                    // 判断 followSets[c1]和 followSets[c2]是否有交集
                    set<string> followSetC1 = followSets[c1].s;
                    set<string> followSetC2 = followSets[c2].s;
                    set<string> intersection;
                }
    }
}

```

```
        // 利用 STL 算法求交集
        set_intersection(
            followSetC1.begin(), followSetC1.end(),
            followSetC2.begin(), followSetC2.end(),
            inserter(intersection, intersection.begin())
        );

        // 如果交集非空，说明存在规约-规约冲突
        if (!intersection.empty())
            return true;
    }

    return false;
}
```

2.7 LR(1)DFA 图生成

LR(1)DFA 生成模块实现了规范的 LR(1)分析表构建算法，为后续的语法分析提供最精确的自动机基础，能够处理更复杂的文法冲突情况。

2.7.1 存储结构

输入：文法结构（grammarMap、grammarDeque、grammarToInt）、First 集合（从 First/Follow 模块继承的 firstSets 映射）、符号分类（bigAlpha 和 smallAlpha）

输出：LR(1)DFA 图（以表格形式展示）

主要存储结构如下表所示：

表 2-6 构建 LR(1)DFA 图的主要存储结构

对象或变量名称	功能	存储结构
lr1Item	LR(1)项结构，包含产生式 ID、点位置和展望符	struct lr1Item
lr1State	LR(1)状态结构，包含项集合和状态转移信息	struct lr1State
lr1Items	存储所有 LR(1)项的全局容器	vector<lr1Item>
lr1States	存储所有 LR(1)状态的全局容器	vector<lr1State>
lr1VN	记录 LR(1)DFA 中出现的非终结符	set<string>

lr1VT	记录 LR(1)DFA 中出现的终结符	set<string>
lr1ItemCount	LR(1)项 ID 计数器，确保唯一性	int
lr1StateCount	LR(1)状态 ID 计数器，确保唯一性	int
lr1VisitedStates	深度优先搜索的访问标记集合	set<int>
grammarMap	文法产生式映射（继承）	unordered_map<string, set<string>>
grammarDeque	文法产生式队列（继承）	deque<grammarUnit>
grammarToInt	产生式到编号的映射（继承）	map<pair<string, string>, int>
firstSets	非终结符的 First 集合（继承）	map<string, firstUnit>

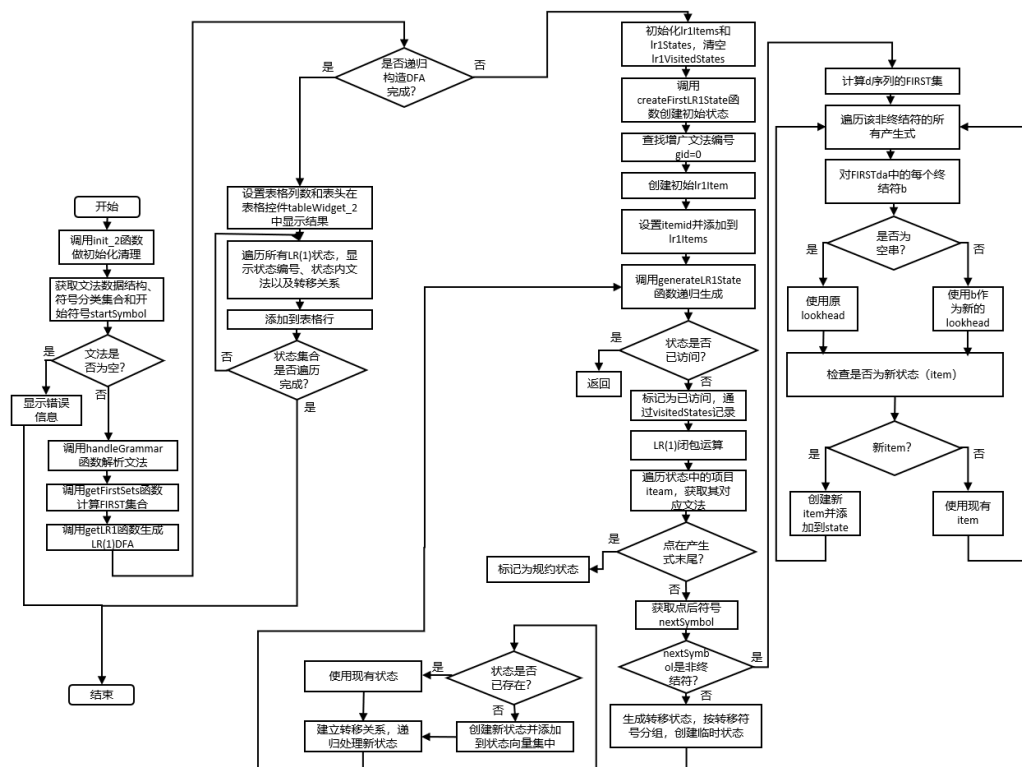
2.7.2 算法实现过程文字描述

LR(1)DFA 生成算法采用规范的 **LR(1)项集族**构建方法，通过**递归下降策略**生成完整的 DFA 图（本系统采用表格形式展示）。

算法首先通过 `getLR1` 函数进行初始化，清空所有全局容器和计数器，然后调用 `createFirstLR1State` 创建初始状态。初始状态包含增广文法的起始项目（如 $E' \rightarrow .S$ ，。接着算法调用 `generateLR1State` 函数以深度优先方式递归生成状态，该函数首先检查当前状态是否已被访问以避免重复处理，然后进行 LR(1)闭包计算：遍历状态中的每个 `lr1Item`，如果点在产生式末尾则标记该状态为规约状态，否则获取点后符号，如果该符号是非终结符且尚未处理，则计算该非终结符所有产生式的新 LR(1)项，其中展望符通过 `firstOfSequence` 函数计算当前项点后符号序列 β 与原始展望符 a 连接的 FIRST 集合得到。闭包计算完成后，算法进行状态转移生成：再次遍历状态中的所有项，对于点不在末尾的项，创建新项（点后移一位，展望符不变），按点后符号分组存入临时状态映射 `tempSave` 中。对于每个临时状态，使用 `isNewLR1State` 检查是否与已有状态重复，不重复则创建新状态并分配 ID，同时记录状态转移关系。

最后算法递归处理每个新生成的状态，直到所有状态都被处理完毕，形成完整的 LR(1)DFA。

2.7.3 算法实现流程图



2.7.4 单元测试

继续基于刚才的简单测试文法和 TINY 语言文法进行测试，点击上方的“LR(1)DFA 生成”按钮，即可在下方的分析结果表中查看到对应文法的 LR(0)DFA（表格形式），测试结果如下图所示：

分析结果表

	状态	状态内文法	FLOAT	ID	TOKEN
1	0	program->.TOKEN , \$...	1	2	3
2	1	TOKEN->FLOAT. , \$			
3	2	TOKEN->ID. , \$			
4	3	program->TOKEN. , \$...			

图 2-15 LR(1)DFA 图生成测试（简单测试文法）

分析结果表					
	状态	状态内文法	ASSIGN	DIVIDE	EC
1	0	program-...			
2	1	assign-stmt-...	13		
3	2	statement-...			
4	3	if-stmt-...			
5	4	statement->if...			
6	5	read-stmt-...			
7	6	statement-...			
8	7	repeat-stmt-...			
9	8	statement-...			
10	9	stmt-sequenc...			
11	10	program-...			
	11	...			

图 2-16 LR(1)DFA 图生成测试（TINY 文法）

由于显示区域大小问题，构建的 LR(1)DFA 图（表格形式）无法显示完全，但是可以通过滑动条实现完整的查看。

2.7.5 关键源程序代码段

序列 FIRST 集合计算函数 `firstOfSequence`，计算符号序列的 FIRST 集合，处理空串传播和终结符终止条件：

```
// 计算符号序列的 FIRST 集合
set<string> GenerateLR1DFA::firstOfSequence(const vector<string>& seq) {
    set<string> result;
    bool allNullable = true;
    for (size_t i = 0; i < seq.size(); ++i) {
        string sym = seq[i];
        if (sym == "#")
            // 若为空（#），则跳过继续查看下一符号
            continue;
        if (GrammarAnalyse::isSmallAlpha(sym) || sym == "$") {
            // 终结符或 $ 直接加入
```

```

        result.insert(sym);
        allNullable = false;
        break;
    }
    // 非终结符
    // 将非终结符的 first（仅包含终结符）加入
    for (const string &t : firstSets[sym].s)
        result.insert(t);

    // 若该非终结符不能推出 epsilon，则停止
    if (!firstSets[sym].isEpsilon) {
        allNullable = false;
        break;
    }
    // else 继续下一个符号
}
if (allNullable)
    // 用内部标志表示可以推出 epsilon
    // 内部使用，表示 nullable
    result.insert("#");

return result;
}

```

LR(1)状态生成核心函数 `generateLR1State`，实现闭包计算和状态转移：

```

// LR(1) 状态生成（递归），与 LR0 实现类似但使用 lr1Items/lr1States，处理 lookahead
// 递归生成 LR(1) 状态
void GenerateLR1DFA::generateLR1State(int stateId) {
    if (lr1VisitedStates.count(stateId) > 0)
        return;

    lr1VisitedStates.insert(stateId);

    lr1State &currentState = lr1States[stateId];

    // 确保状态 0 包含起始产生式
    if (stateId == 0) {
        // 查找起始符号的产生式
        for (size_t i = 0; i < grammarDeque.size(); ++i)
            if (grammarDeque[i].left == startSymbol) {
                // 添加初始项目 [S' -> . program, $]
                lr1Item initialItem;
            }
        }
    }
}

```

```

        initialItem.gid = i;
        initialItem.index = 0;
        initialItem.lookahead = "$";
        initialItem.itemid = lr1ItemCount++;
        lr1Items.push_back(initialItem);
        currentState.itemV.push_back(initialItem.itemid);
        break;
    }
}

// 先做闭包：对于状态中每个 item，如果点后是非终结符 B，则 for each
B-> $\gamma$ , 对应 lookahead 由 FIRST( $\beta$  a) 决定
for (size_t i = 0; i < lr1States[stateId].itemV.size(); ++i) {
    lr1Item &it = lr1Items[lr1States[stateId].itemV[i]];
    grammarUnit &g = grammarDeque[it.gid];
    QStringList rightListQ = QStringList::fromStdString(g.right).split(" ",
Qt::SkipEmptyParts);
    vector<string> rightList;
    for (int k = 0; k < rightListQ.size(); ++k)
        rightList.push_back(rightListQ[k].toStdString());

    // 如果 dot 在产生式末尾或产生式为 # 则标记为规约项 (isEnd)
    if (it.index == (int)rightList.size() || g.right == "#") {
        lr1States[stateId].isEnd = true;
        continue;
    }

    string nextSymbol = rightList[it.index];

    // 若 nextSymbol 是非终结符，则对其进行闭包扩展
    if (GrammarAnalyse::isBigAlpha(nextSymbol)) {
        //  $\beta$  是点后剩余符号 (不包括第一个被处理的 nextSymbol)
        vector<string> beta;
        for (size_t p = it.index + 1; p < rightList.size(); ++p)
            beta.push_back(rightList[p]);

        // 对于当前 item 的 lookahead, 计算 FIRST( $\beta$  + lookahead)
        vector<string> seq = beta;
        seq.push_back(it.lookahead);

        set<string> firstSetSeq = firstOfSequence(seq);

        // 遍历 grammarMap[nextSymbol] 的所有产生式, 按 FIRST 集合创
建新 item

```

```

        for (const auto &prod : grammarMap[nextSymbol]) {
            int gid_new = grammarToInt[make_pair(nextSymbol, prod)];
            for (const string &b : firstSetSeq) {
                string la = (b == "#") ? it.lookahead : b; // 若能推出  $\epsilon$ , 用原
lookahead

                int existing = isNewLR1Item(gid_new, 0, la);
                if (existing == -1) {
                    lr1Item newItem(gid_new, 0, la);
                    newItem.itemid = lr1ItemCount++;
                    lr1Items.push_back(newItem);
                    lr1States[stateId].itemV.push_back(newItem.itemid);
                }
                else
                    // 即使 item 已存在, 也保证放入 state.itemV
                    if (find(lr1States[stateId].itemV.begin(),
lr1States[stateId].itemV.end(), existing) == lr1States[stateId].itemV.end())
                        lr1States[stateId].itemV.push_back(existing);
            }
        }
    }

    // 生成从当前状态出发的转移 (按点后符号分组)
    map<string, lr1State> tempSave; // key: symbol -> 临时 lr1State
    for (size_t i = 0; i < lr1States[stateId].itemV.size(); ++i) {
        lr1Item &it = lr1Items[lr1States[stateId].itemV[i]];
        grammarUnit &g = grammarDeque[it.gid];
        QStringList rightListQ = QString::fromStdString(g.right).split(" ",
Qt::SkipEmptyParts);
        vector<string> rightList;
        for (int k = 0; k < rightListQ.size(); ++k)
rightList.push_back(rightListQ[k].toStdString());

        if (it.index == (int)rightList.size() || g.right == "#")
            // 点在末尾
            continue;

        string nextSymbol = rightList[it.index];
        // 准备下一个 item: gid 不变, index + 1, lookahead 保持不变
        lr1Item nextIt(it.gid, it.index + 1, it.lookahead);
        int nextItemId = isNewLR1Item(nextIt.gid, nextIt.index, nextIt.lookahead);
        if (nextItemId == -1) {
            nextIt.itemid = lr1ItemCount++;
            lr1Items.push_back(nextIt);

```

```

    }
    else
        nextIt.itemid = nextItemId;

    // 将 nextIt 放到临时状态 tempSave[nextSymbol] 中
    lr1State &tempSt = tempSave[nextSymbol];
    tempSt.sid = -1; // 临时
    tempSt.itemV.push_back(nextIt.itemid);
    tempSt.originV.push_back(nextIt.itemid);

    // record symbol sets
    if (GrammarAnalyse::isBigAlpha(nextSymbol))
        lr1VN.insert(nextSymbol);
    else if (GrammarAnalyse::isSmallAlpha(nextSymbol))
        lr1VT.insert(nextSymbol);
}

// 将每个临时状态归一化（闭包会在递归进入时补完）；检测是否与已有状态
// 重复，若不重复则新建
for (auto &p : tempSave) {
    string symbol = p.first;
    lr1State tmp = p.second;
    // 检查是否和已有 lr1States 相同（通过 originV）
    int existSid = isNewLR1State(tmp.originV);
    lr1State newState;
    if (existSid == -1) {
        newState.sid = lr1StateCount++;
        newState.itemV = tmp.itemV;
        newState.originV = tmp.originV;
        lr1States.push_back(newState);
    }
    else
        newState.sid = existSid;

    nextStateUnit n;
    n.c = symbol;
    n.sid = newState.sid;
    lr1States[stateId].nextStateVector.push_back(n);
}

// 递归处理下级状态
int nsize = lr1States[stateId].nextStateVector.size();
for (int i = 0; i < nsize; ++i) {
    auto &nextunit = lr1States[stateId].nextStateVector[i];

```

```
        generateLR1State(nextunit.sid);
    }
}
```

初始状态创建函数 `createFirstLR1State`，创建 LR(1)DFA 的初始状态，包含增广文法的起始项目：

```
// 创建 LR(1) 初始状态（增广项 E' -> .S, lookahead = $）
void GenerateLR1DFA::createFirstLR1State() {
    lr1States.clear();
    lr1Items.clear();
    lr1ItemCount = 0;
    lr1StateCount = 0;

    lr1State start;
    start.sid = lr1StateCount++;
    lr1States.push_back(start);

    // 找到增广文法的编号
    // 在 handleGrammar() 中把增广规则 push_front 到 grammarDeque
    // 与 LR0 一致，取 gid = 0 作为 E' -> S
    lr1Item firstItem;
    firstItem.gid = 0;
    firstItem.index = 0;
    firstItem.lookahead = "$";
    firstItem.itemid = lr1ItemCount++;
    lr1Items.push_back(firstItem);

    lr1States[0].itemV.push_back(firstItem.itemid);
    lr1States[0].originV.push_back(firstItem.itemid);
}
```

2.8 LR(1)分析表生成

LR(1)分析表构建模块将抽象的 LR(1)DFA 转换为具体可用的分析表，为语法分析器的实现提供了核心数据结构，是整个编译器前端的关键组成部分之一。

2.8.1 存储结构

输入：文法文本、LR(1)DFA、文法结构、FIRST 集

输出：ACTION 表、GOTO 表和整体可视化表格

主要存储结构如下表所示：

表 2-7 构建 LR(1)分析表的主要存储结构

对象或变量名称	功能	存储结构
ACTION	动作表，存储状态-符号对应的分析动作	map<pair<int, string>, string>
GOTO	转移表，存储状态-非终结符对应的状态转移	map<pair<int, string>, int>
allTerminals	所有终结符集合（包括\$结束符）	set<string>
allNonTerminals	所有非终结符集合	set<string>
symbolToColumn	符号到表格列索引的映射	map<string, int>
lr1States	LR(1)自动机状态集合（从 LR(1)DFA 模块继承）	vector<lr1State>
lr1Items	LR(1)项目集合（从 LR(1)DFA 模块继承）	vector<lr1Item>
grammarDeque	文法产生式队列（从文法模块继承）	deque<grammarUnit>
grammarToInt	产生式到编号的映射（从文法模块继承）	map<pair<string, string>, int>
smallAlpha	终结符集合（从文法模块继承）	vector<string>
bigAlpha	非终结符集合（从文法模块继承）	vector<string>
firstSets	First 集合（从 First/Follow 模块继承）	map<string, firstUnit>

2.8.2 算法实现过程文字描述

LR(1)分析表构建算法基于已生成的 LR(1)DFA 构建完整的 ACTION 和 GOTO 分析表，具体阐述如下：

基于已经完成的 LR(1)DFA 构建，遍历 lr1States 中的每个状态及其 nextStateVector 中的转移关系，对于其中每个转移，有如下的区分操作：

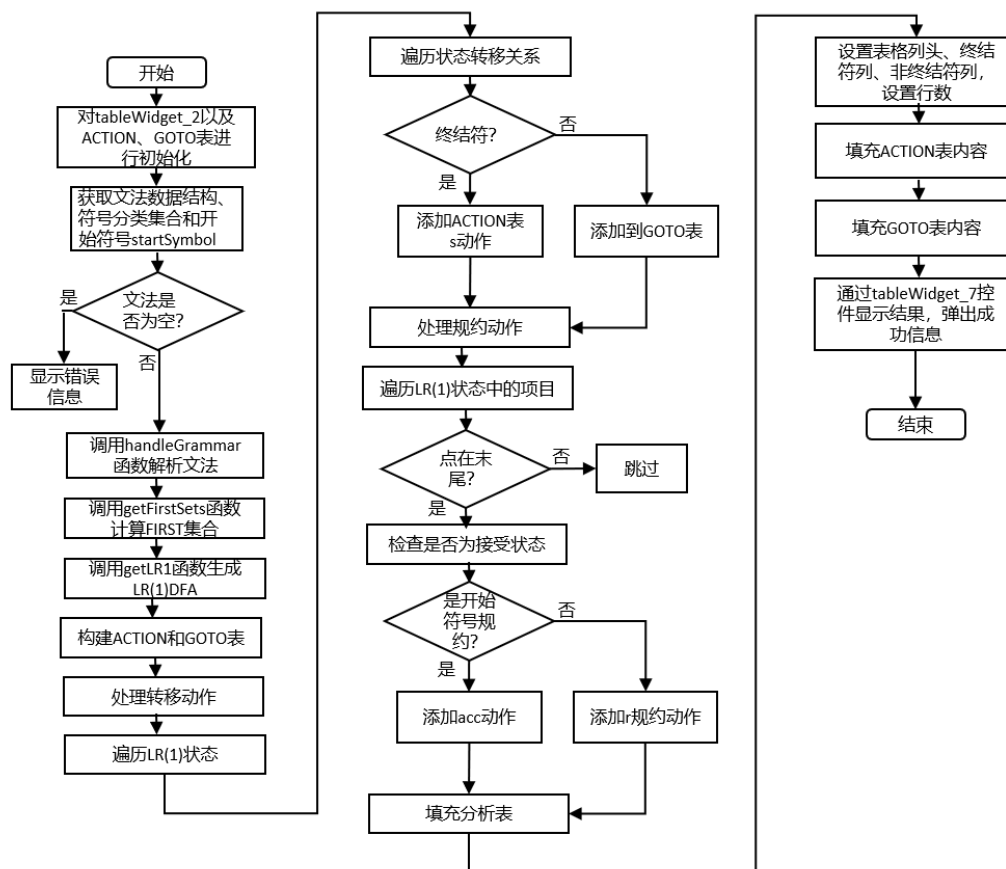
- （1）如果转移符号是终结符，在 ACTION 表中添加"s 目标状态 ID"的移进动作；
- （2）如果转移符号是非终结符，在 GOTO 表中添加目标状态 ID 的转移关系。

之后算法处理规约动作，遍历每个状态中的 LR(1)项目，检查是否为规

约项目（点在产生式末尾或空产生式），如果是规约项目，检查是否为接受状态（增广文法的起始产生式且展望符为），是则添加"acc"接受动作，否则添加"r 产生式编号"的规约动作。

最后，收集所有终结符和非终结符，设置表格行列结构，填充 ACTION 和 GOTO 表内容，完成 LR(1)分析表的构建，并通过对对应控件实现 LR(1)分析表的可视化展示。

2.8.3 算法实现流程图



2.8.4 单元测试

沿用 TINY 文法，点击“LR(1)分析表”旁的“开始生成”按钮，即可查看 LR(1)分析表如下图所示：

	状态	\$	FLOAT	ID	TOKEN	program
1	0		s1	s2	3	
2	1	r1				
3	2	r2				
4	3	acc				

图 2-17 LR(1)分析表生成测试

分析结果表

	状态	\$	ASSIGN	DIVIDE	EQ	ID	LPAN	LT	LTEQ	MINUS	MOD	MU
1	0					s1						
2	1		s13									
3	2	r5										
4	3					s76	s77					
5	4	r3										
6	5					s264						
7	6	r6										
8	7					s129						
9	8	r4										
10	9	r2										
11	10	acc										

图 2-18 LR(1)分析表生成测试

由于显示区域大小问题，构建的 LR(1)分析表无法显示完全，但是可以通过滑动条实现完整的查看。

2.8.5 关键源程序代码段

移进动作处理代码，处理 LR(1)自动机中的状态转移，生成移进动作和 GOTO 转移：

```
// 处理状态转移（移进动作）
for (const lr1State &st : lr1States) {
    int sid = st.sid;
    for (const nextStateUnit &n : st.nextStateVector) {
        string sym = n.c;
        int toSid = n.sid;
        if (isSmallAlpha(sym) || sym == "$") {
            ACTION[make_pair(sid, sym)] = "s" + to_string(toSid);
        } else {
            GOTO[make_pair(sid, sym)] = toSid;
        }
    }
}
```

```
}  
}
```

规约动作处理代码，处理规约项目，生成规约动作和接受动作：

```
// 添加规约动作  
for (const lr1State &st : lr1States) {  
    int sid = st.sid;  
    for (int itemid : st.itemV) {  
        const lr1Item &it = lr1Items[itemid];  
        const grammarUnit &g = grammarDeque[it.gid];  
        QStringList rightListQ = QStringList::fromStdString(g.right).split(" ",  
Qt::SkipEmptyParts);  
        int rightSize = rightListQ.size();  
  
        // 检查是否为规约项目  
        if (it.index == rightSize || g.right == "#") {  
            // 检查是否是接受状态  
            if (g.left == trueStartSymbol && it.lookahead == "$") {  
                ACTION[make_pair(sid, it.lookahead)] = "acc";  
            } else {  
                string rLabel = "r" + to_string(g.gid);  
                ACTION[make_pair(sid, it.lookahead)] = rLabel;  
            }  
        }  
    }  
}
```

2.9 源程序语法分析表生成

LR(1)源程序语法分析模块实现了完整的语法分析流程，将词法分析、语法分析、语义分析和中间代码生成有机结合，为编译器前端提供了完整的解决方案。该算法能够处理复杂的程序设计语言结构，并为生成高质量的中间代码表示做准备工作。

2.9.1 存储结构

输入：文法文本、测试源代码、LR(1)DFA（从 getLR1()获取生成的LR(1)DFA 图数据）、分析表（ACTION 和 GOTO 表）

输出：逐步显示的分析状态表格

主要存储结构如下表所示：

表 2-8 构建源程序语法分析表的主要存储结构

对象或变量名称	功能	存储结构
stateStack	存储分析过程中的状态序列	vector<int>
symbolStack	存储已识别的文法符号序列	vector<string>
nodeStack	存储语法树节点，用于构建语法树	vector<TreeNode*>
semanticStack	存储语义值，用于中间代码生成	vector<string>
inputTokens	存储词法分析后的输入Token 序列	vector<string>
ACTION	LR(1)动作分析表（状态×符号→动作）	map<pair<int, string>, string>
GOTO	LR(1)转移分析表（状态×非终结符→状态）	map<pair<int, string>, int>
intermediateCodes	生成的中间代码指令序列	vector<IntermediateCode>
syntaxTreeRoot	语法树的根节点指针	TreeNode*
lr1States	LR(1)自动机状态集合（继承）	vector<lr1State>
lr1Items	LR(1)项目集合（继承）	vector<lr1Item>
grammarDeque	文法产生式队列（继承）	deque<grammarUnit>
stepCounter	分析步骤计数器	int
tableRow	表格行计数器	int

2.9.2 算法实现过程文字描述

LR(1)源程序语法分析算法基于已构建的 LR(1)分析表对输入的源代码进行语法分析，具体阐述如下：

准备阶段：

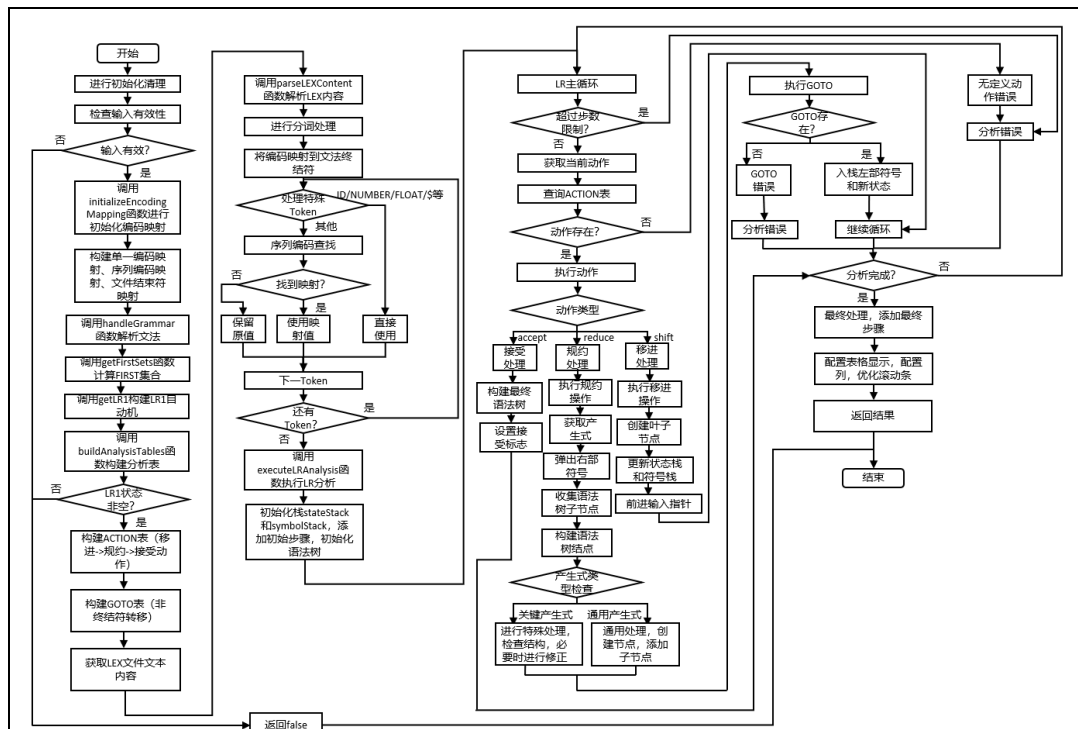
首先通过 init_2 函数进行数据结构的清空与初始化，接着通过 GrammarAnalyse::handleGrammar 函数处理输入的程序文法，随后调用 AnalyseFIRST::getFirstSets 计算文法的 FIRST 集，最后通过 GenerateLR1DFA::getLR1 函数构建完整的 LR(1)DFA（规范 LR(1)项集族）。在此之后，算法根据 DFA 的状态转移构建 ACTION 表和 GOTO 表：遍历所有 LR(1)状态，根据其 nextStateVector 记录移进和 GOTO 动作；并遍历所有 LR(1)项，当点的位置位于产生式末尾时，在 ACTION 表中添加规约动作（若为开始符号 program 且向前看符号为\$，则添加接受动作 acc）。

LR(1)分析驱动与语法树构建阶段:

该阶段首先处理从 LEX 分析器得到的编码序列。算法利用预定义的 `encodingToTerminalMap` (该映射由 `regexEncodings` 和 `sequenceEncodings` 等信息初始化) 将数字编码映射为文法定义的终结符 (如 ID、NUMBER、PLUS 等), 形成规范的输入 Token 序列并以 \$ 结尾。分析引擎初始化状态栈为 [0], 符号栈和语法树节点栈 (`nodeStack`) 为空, 然后进入主循环。每次循环取当前栈顶状态和输入符号, 查询 ACTION 表获取动作。如果是移进动作 `sX`, 则将输入符号压入符号栈, 将目标状态 `X` 压入状态栈, 并为该终结符创建一个语法树叶节点压入 `nodeStack`, 然后前进输入指针。如果是规约动作 `rX`, 则根据产生式右部长度 `rcount` 弹出栈顶符号和状态, 同时从 `nodeStack` 中弹出相应数量的子节点。**此处是关键:** 算法根据具体的产生式左部执行专门的语法树构建逻辑, 以创建正确的父节点并组织子节点结构, 然后将新节点压回 `nodeStack`。接着, 将产生式左部符号压入符号栈, 并查询 GOTO 表得到新状态压入状态栈。如果是接受动作 `acc`, 则从 `nodeStack` 中取出最终的根节点作为语法树 (分析树) 根 `parseTreeRoot`, 并完成分析。如果 ACTION 表查询失败, 则报告语法错误。整个过程的所有步骤 (包括状态栈、符号栈、剩余输入和执行的动做说明) 均被记录并添加到 `QTableWidget` 表格控件中, 实现分析过程的逐步可视化。

经过以上两阶段的处理, 算法在构建完整的 LR(1)分析表后, 驱动分析过程生成源程序的语法分析树, 并通过表格控件实现分析过程的动态展示供用户查看。

2.9.3 算法实现流程图



2.9.4 单元测试

沿用此前的 TINY 文法以及简单测试文法，在单元测试中我使用简单的测试代码，复杂完整的测试将放到项目整体测试中实现（此部分详见测试文档），lex 文件内容分别如下：

简单测试文法的：

```
100 "3.14159" -1
```

TINY 的：

```
400 "azusa" 212 400 "a" 204 500 "123" 200 500 "1" -1
```

分别在指定输入框输入/导入对应的文法文本和对应的测试代码的 lex 文件后，点击上方的“源代码分析”按钮，即可实现源程序语法分析表的构建，测试结果如下图所示：



图 2-19 LR(1)源程序语法分析表生成测试

分析结果表					
	步骤	状态栈	序号	余数	说明
1	0	0		1...	初始
2	1	0		1...	移进
3	2	0 1	ID	...	移进: 状态1遇到'ASSIGN', 移进到状态13
4	3	0 1 13	1...	1...	移进: 状态13遇到'ID', 移进到状态14
5	4	0 1 13 14	1...	...	规约: 使用产生式 factor -> ID
6	5	0 1 13 18	1...	...	规约完成, 执行GOTO
7	6	0 1 13 18	1...	...	规约: 使用产生式 term -> factor
8	7	0 1 13 20	1...	...	规约完成, 执行GOTO
9	8	0 1 13 20	1...	...	移进: 状态20遇到'MOD', 移进到状态42
10	9	0 1 13 20 42	1...	...	规约: 使用产生式 mulop -> MOD
11	10	0 1 13 20 62	1...	...	规约完成, 执行GOTO
12	11	0 1 13 20 62	1...	...	移进: 状态62遇到'ID', 移进到状态14

图 2-20 LR(1)源程序语法分析表生成测试

由于显示区域大小问题, 构建的 LR(1)源程序语法分析表无法显示完全, 但是可以通过滑动条实现完整的查看。

2.9.5 关键源程序代码段

词法分析核心代码, 文法解析和 LR(1)自动机构建代码, 处理文法字符串构建分析表:

```
// 1. 清空初始化
init_2();

// 2. 文法处理
GrammarAnalyse::handleGrammar();

// 3. 计算 FIRST 集
AnalyseFIRST::getFirstSets();

// 4. 构建 LR(1)自动机
GenerateLR1DFA::getLR1();
```

```

// 5. 构建 ACTION/GOTO 分析表
map<pair<int, string>, string> ACTION;
map<pair<int, string>, int> GOTO;

// 处理状态转移
for (const lr1State &st : lr1States) {
    int sid = st.sid;
    for (const nextStateUnit &n : st.nextStateVector) {
        string sym = n.c;
        int toSid = n.sid;
        if (GrammarAnalyse::isSmallAlpha(sym) || sym == "$") {
            ACTION[make_pair(sid, sym)] = "s" + to_string(toSid);
        } else {
            GOTO[make_pair(sid, sym)] = toSid;
        }
    }
}

// 处理归约动作
for (const lr1State &st : lr1States) {
    int sid = st.sid;
    for (int itemid : st.itemV) {
        const lr1Item &it = lr1Items[itemid];
        const grammarUnit &g = grammarDeque[it.gid];
        QStringList rightList = QString::fromStdString(g.right).split(" ",
Qt::SkipEmptyParts);
        int rightSize = rightList.size();

        if (it.index == rightSize || g.right == "#") {
            if (g.left == "program" && it.lookahead == "$") {
                ACTION[make_pair(sid, it.lookahead)] = "acc";
            } else {
                string rLabel = "r" + to_string(g.gid);
                ACTION[make_pair(sid, it.lookahead)] = rLabel;
            }
        }
    }
}
}

```

移进动作处理代码，处理移进动作，更新分析栈和语法树：

```

if (act[0] == 's') {
    int toState = stoi(act.substr(1));

```

```

// 显示移进前状态
QString remaining = tokensToString(
    vector<string>(inputTokens.begin() + ip, inputTokens.end()));

addAnalysisStep(tableWidget, stepCounter++,
    stateStackToString(stateStack),
    symbolStackToString(symbolStack),
    remaining,
    QString("移进: 状态%1 遇到'%2', 移进到状态%3")
    .arg(curState)
    .arg(QString::fromStdString(a))
    .arg(toState),
    tableRow);

// 执行移进操作
symbolStack.push_back(a);
stateStack.push_back(toState);

// 创建语法树叶节点
TreeNode* leaf = new TreeNode(a);
nodeStack.push_back(leaf);

ip++; // 前进输入指针
}

```

规约动作处理代码，处理规约动作，构建语法树并生成中间代码

```

if (act[0] == 'r') {
    int gid = stoi(act.substr(1));
    const grammarUnit &prod = grammarDeque[gid];

    // 计算右部符号数
    QStringList rightList =
        QString::fromStdString(prod.right).split(" ", Qt::SkipEmptyParts);
    int rcount = (prod.right == "#") ? 0 : rightList.size();

    // 显示规约前状态
    QString remaining = tokensToString(
        vector<string>(inputTokens.begin() + ip, inputTokens.end()));

    addAnalysisStep(tableWidget, stepCounter++,
        stateStackToString(stateStack),
        symbolStackToString(symbolStack),

```

```

        remaining,
        QString("规约： 使用产生式 %1 -> %2")
        .arg(QString::fromStdString(prod.left))
        .arg(QString::fromStdString(prod.right)),
        tableRow);

// 规约操作：弹出栈顶的右部符号
for (int k = 0; k < rcount; k++) {
    if (!symbolStack.empty()) symbolStack.pop_back();
    if (!stateStack.empty()) stateStack.pop_back();
}

// Reduce：构造语法树子树
vector<TreeNode*> children;
for (int k = 0; k < rcount; ++k) {
    if (!nodeStack.empty()) {
        children.push_back(nodeStack.back());
        nodeStack.pop_back();
    }
}
reverse(children.begin(), children.end());

// 特殊处理 program 节点
if (prod.left == "program" && children.size() == 1) {
    TreeNode* programNode = new TreeNode("program");
    programNode->addChild(children[0]);
    nodeStack.push_back(programNode);
}

// 处理 definition-list 递归定义
else if (prod.left == "definition-list" && children.size() == 2) {
    TreeNode* listNode = children[0];
    TreeNode* defNode = children[1];

    if (listNode->symbol != "definition-list") {
        TreeNode* newListNode = new TreeNode("definition-list");
        if (listNode->symbol == "definition") {
            newListNode->addChild(listNode);
        }
        newListNode->addChild(defNode);
        nodeStack.push_back(newListNode);
    } else {
        if (defNode->symbol != "definition") {
            TreeNode* wrapDef = new TreeNode("definition");
            wrapDef->addChild(defNode);

```

```
        listNode->addChild(wrapDef);
    } else {
        listNode->addChild(defNode);
    }
    nodeStack.push_back(listNode);
}
}
// 通用处理
else {
    TreeNode* parent = new TreeNode(prod.left);
    for (TreeNode* ch : children) {
        parent->addChild(ch);
    }
    nodeStack.push_back(parent);
}

// 执行 GOTO
int cur = stateStack.back();
string A = prod.left;
auto gotoIt = GOTO.find(make_pair(cur, A));
int gotoState = gotoIt->second;

symbolStack.push_back(A);
stateStack.push_back(gotoState);

addAnalysisStep(tableWidget, stepCounter++,
                stateStackToString(stateStack),
                symbolStackToString(symbolStack),
                remaining,
                "规约完成，执行 GOTO",
                tableRow);
}
```

2.10 语法树构建与展示

2.10.1 存储结构

输入：文法文本、测试源代码、LR(1)DFA（从 getLR1()获取生成的 LR(1)DFA 图数据）、分析表（ACTION 和 GOTO 表）

输出：语法树（可视化）

主要存储结构如下表所示：

表 2-9 构建语法树的主要存储结构

对象或变量名称	功能	存储结构
TreeNode	语法树的节点结构，存储语法树节点的信息	struct TreeNode
parseTreeRoot	分析树根节点（为简化的原始分析树）	TreeNode* parseTreeRoot
syntaxTreeRoot	语法树根节点（简化后的语法树）	TreeNode* syntaxTreeRoot
nodeStack	存储语法树节点，用于构建语法树	vector<TreeNode*>
stateStack	存储分析过程中的状态序列	vector<int>
symbolStack	存储已识别的文法符号序列	vector<string>
semanticStack	存储语义值，用于中间代码生成	vector<string>
inputTokens	存储词法分析后的输入Token 序列	vector<string>
ACTION	LR(1)动作分析表（状态×符号→动作）	map<pair<int, string>, string>
GOTO	LR(1)转移分析表（状态×非终结符→状态）	map<pair<int, string>, int>
intermediateCodes	生成的中间代码指令序列	vector<IntermediateCode>
syntaxTreeRoot	语法树的根节点指针	TreeNode*
lr1States	LR(1)自动机状态集合（继承）	vector<lr1State>
lr1Items	LR(1)项目集合（继承）	vector<lr1Item>
grammarDeque	文法产生式队列（继承）	deque<grammarUnit>
stepCounter	分析步骤计数器	int
tableRow	表格行计数器	int

2.10.2 算法实现过程文字描述

语法树的生成是通过移进-规约过程中的节点栈管理实现的。语法树的构建与 LR 分析过程紧密耦合，采用自底向上的方式逐步构建完整的抽象语法树：

- **移进动作**创建叶子节点，对应终结符；
- **规约动作**合并子树，构建层次结构；
- **接受动作**完成整棵语法树。

通过节点栈管理构建过程中的临时子树，最终完成完整的语法树生成，具体的阐述如下：

LR(1)源程序语法分析算法基于已构建的 LR(1)分析表对输入的源代码进行语法分析，具体阐述如下：

语法树构建模块基于 LR(1)分析过程中生成的原始分析树构建简化的抽象语法树。首先在 LR(1)语法分析完成后获取生成的原始分析树根节点，该分析树包含完整的语法推导过程但存在大量冗余节点和辅助符号，需要进行精简和重构。算法通过 `GenerateSyntaxTree::buildSyntaxTree` 函数启动转换过程，首先对现有语法树进行清理，确保从干净的初始状态开始。

语法树简化与重构阶段通过深度优先遍历递归处理原始分析树的每个节点。算法首先对节点进行分类和特殊处理，通过 `isPunctuation` 函数识别并过滤分号、逗号、括号等标点符号节点。对于包含多个子节点的标点节点创建顺序节点重新组织子节点结构。表达式相关节点如 `exp`、`simple-exp`、`term`、`factor` 等进行特殊处理，检查子节点是否包含运算符来决定保留完整表达式结构或进行扁平化简化。运算符节点如 `addop`、`mulop`、`relop` 等被保留，其具体运算符值从子节点提升到当前节点的 `value` 属性中。核心程序结构节点包括 `program`、`function-definition`、`compound-stmt` 以及控制流节点和 `return-stmt` 等被保留原结构，仅对其子节点进行递归简化。列表型节点如 `definition-list`、`statement-list` 等进行扁平化处理，展开嵌套的同名节点。赋值语句节点被特殊处理，保留 ID 和表达式子节点并移除中间的标点符号。

冗余节点简化通过 `isRedundantNode` 函数识别冗余中间节点，当这些节点只有一个有效子节点时用子节点替换当前节点以实现树高度压缩，但核心表达式节点、运算符节点和程序结构节点等不被视为冗余以保持关键语法结构。空节点清理阶段遍历处理后的树结构，删除无子节点且非保留类型的节点以确保树的简洁性。

语法树优化与修正阶段检查简化后的语法树根节点，如果根节点为 `program` 且只有一个 `definition-list` 子节点，则用该子节点替换根节点以消除不必要的包装层次。同时确保生成的语法树节点保持正确的父子关系，并保留行号信息和值信息。

语法树输出与展示阶段通过 `printSyntaxTree` 函数以缩进形式在控制台打印完整的语法树结构，包括节点符号和附加值信息以便调试和验证转换结



图 2-21 语法树生成测试

2.10.5 关键源程序代码段

语法树的核心数据结构如下：

```
// 语法树节点结构
struct TreeNode {
    string symbol;           // 符号名称（终结符或非终结符）
    string value;           // 值（对于终结符，如 ID 的名称、
    NUMBER 的值）
    vector<TreeNode*> children; // 子节点
    int line;               // 行号（可选）

    TreeNode(const string& sym, const string& val = "")
        : symbol(sym), value(val), line(0) {}

    ~TreeNode() {
        for (TreeNode* child : children) {
            delete child;
        }
    }

    // 添加子节点
    void addChild(TreeNode* child) {
        children.push_back(child);
    }
};
```

```
// 分析过程中的语法树节点栈（用于 LR 分析）
```

```
extern vector<TreeNode*> nodeStack;
```

```
// 语法树根节点（简化后的语法树）
```

```
extern TreeNode* syntaxTreeRoot;
```

```
// 分析树根节点（未简化的原始分析树）
```

```
extern TreeNode* parseTreeRoot;
```

移进动作时的语法树处理，当分析器执行移进动作时，为每个终结符创建叶子节点，新建的叶子节点被压入 nodeStack，等待后续的规约操作：

```
// 移进动作处理
```

```
if (act[0] == 's') {
```

```
...
```

```
// Shift: 构造语法树叶子结点
```

```
symbolStack.push_back(a);
```

```
stateStack.push_back(toState);
```

```
...
```

```
}
```

规约动作时语法树构建的核心环节，通过子树合并逐步构建完整的语法树：

```
// 规约动作处理
```

```
if (act[0] == 'r') {
```

```
...
```

```
// Reduce: 构造语法树子树
```

```
// 1. 收集右部对应的子树
```

```
vector<TreeNode*> children;
```

```
for (int k = 0; k < rcount; ++k) {
```

```
    if (!nodeStack.empty()) {
```

```
        children.push_back(nodeStack.back());
```

```
        nodeStack.pop_back();
```

```
    }
```

```
}
```

```
reverse(children.begin(), children.end());
```

```
// 2.  $\epsilon$  产生式:  $A \rightarrow \#$ 
```

```
if (rcount == 0) {
```

```
    // 不产生任何节点
```

```
...
```

```
    }

    // 3
    else if (prod.left == "program" && children.size() == 1) {
        TreeNode* programNode = new TreeNode("program");
        programNode->addChild(children[0]);
        nodeStack.push_back(programNode);
    }
    ...
    // 其他产生式：通用处理
    else {
        TreeNode* parent = new TreeNode(prod.left);
        for (TreeNode* ch : children) {
            parent->addChild(ch);
        }
        nodeStack.push_back(parent);
    }
    ...
}
```

2.11 中间代码生成（选做）

2.11.1 存储结构

输入：语法分析生成的语法树节点

输出：中间代码序列（三地址码形式）

主要存储结构如下表所示：

表 2-10 生成中间代码（三元组）的主要存储结构

对象或变量名称	功能	存储结构
IntermediateCodeType	中间代码指令类型枚举，定义所有支持的运算类型	enum
IntermediateCode	中间代码指令结构，表示一条三地址码指令	struct IntermediateCode
intermediateCodes	存储所有生成的中间代码指令序列	vector<IntermediateCode>
tempVarCounter	临时变量计数器，用于生成唯一的临时变量名	int
labelCounter	标签计数器，用于生成唯一的跳转标签名	int

2.11.2 算法实现过程文字描述

中间代码生成模块采用**语法制导翻译方法**，将语法分析得到的语法树转换为三地址码形式的中间代码。算法通过 `generateFromSyntaxTree` 函数递归遍历语法树节点，该函数接收语法树节点指针作为输入，根据节点符号类型选择相应的代码生成策略。

对于赋值语句节点 (`assign-stmt`)，生成 `IC_ASSIGN` 指令将右侧表达式结果赋给左侧标识符；对于输入输出语句节点 (`read-stmt` / `write-stmt`)，分别生成 `IC_READ` 和 `IC_WRITE` 指令。

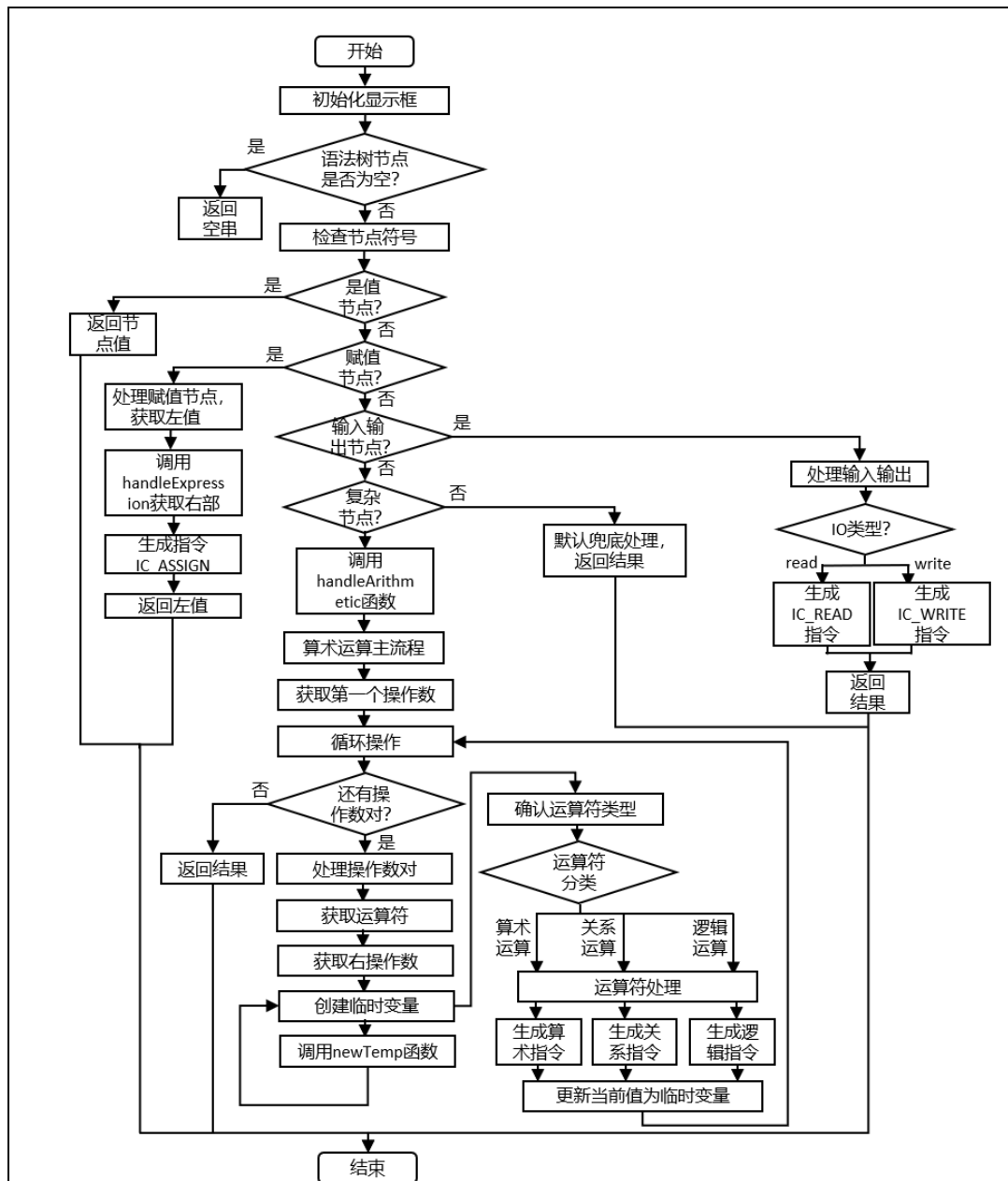
对于算术和逻辑表达式节点，`handleExpression` 和 `handleArithmetic` 函数根据运算符类型生成相应的算术或比较指令，并使用 `newTemp` 函数创建临时变量存储中间结果。

对于控制流语句，如条件语句和循环语句，分别由 `handleConditional` 和 `handleLoop` 函数生成条件跳转和无条件跳转指令，并使用 `newLabel` 函数创建标签标记跳转目标位置。

在生成过程中，算法维护临时变量计数器和标签计数器确保名称唯一性，所有生成的中间代码指令通过 `addCode` 函数按顺序存储在全局的 `intermediateCodes` 向量中。

最终，模块提供了 `outputToFile` 和 `outputToString` 函数，将中间代码序列格式化为可读文本输出到文件或字符串，供用户查看和使用。

2.11.3 算法实现流程图



2.11.4 单元测试

沿用之前的 TINY 文法，采用如下的简单 TINY 语言源程序赋值语句进行测试：

400 "azusa" 212 400 "a" 204 500 "123" 200 500 "1" -1

点击“代码生成”按钮，即可观察到测试结果如下图所示：

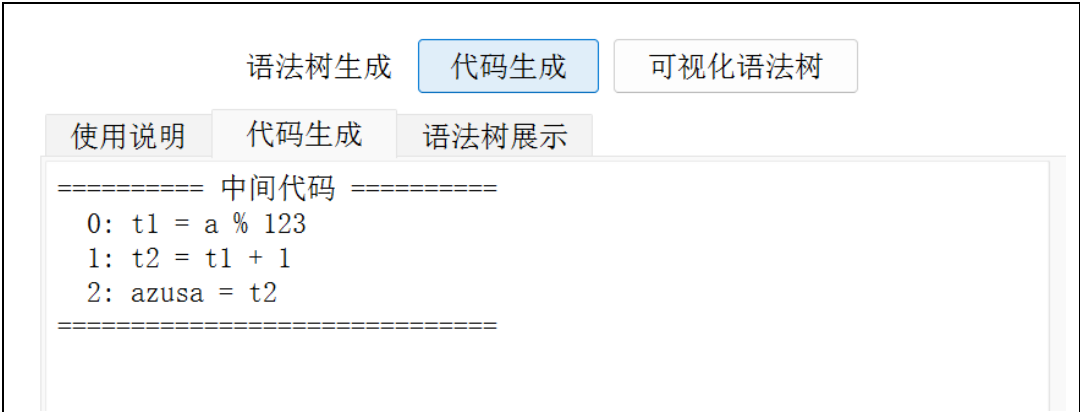


图 2-22 中间代码（三元组）的单元测试结果

2.11.5 关键源程序代码段

中间代码指令结构定义，通过定义终极那代码的指令类型和指令结构，支持三地址码格式：

```
// 中间代码类型枚举
enum IntermediateCodeType {
    IC_ASSIGN,      // 赋值
    IC_READ,        // 读
    IC_WRITE,       // 写
    IC_ADD,         // 加法
    IC_SUB,         // 减法
    IC_MUL,         // 乘法
    IC_DIV,         // 除法
    IC_MOD,         // 取模
    IC_LT,          // 小于
    IC_LE,          // 小于等于
    IC_GT,          // 大于
    IC_GE,          // 大于等于
    IC_EQ,          // 等于
    IC_NE,          // 不等于
    IC_JUMP,        // 无条件跳转
    IC_JUMP_TRUE,   // 为真跳转
    IC_JUMP_FALSE,  // 为假跳转
    IC_LABEL,       // 标签
    IC_FUNC_DECL,   // 函数声明
    IC_FUNC_END,    // 函数结束
    IC_PARAM,       // 参数
    IC_CALL,        // 函数调用
    IC_RETURN,      // 返回
    IC_ARRAY_DECL,  // 数组声明
    IC_ARRAY_LOAD,  // 数组加载
```

```

    IC_ARRAY_STORE // 数组存储
};

// 中间代码结构体
struct IntermediateCode {
    IntermediateCodeType type; // 指令类型
    string result;             // 结果
    string op1;                // 操作数 1
    string op2;                // 操作数 2
    string label;              // 标签

    IntermediateCode(IntermediateCodeType t = IC_ASSIGN,
                    const string& r = "",
                    const string& o1 = "",
                    const string& o2 = "",
                    const string& l = "")
        : type(t), result(r), op1(o1), op2(o2), label(l) {}

    // 转换为字符串表示
    string toString() const;
};

// 全局中间代码列表
extern vector<IntermediateCode> intermediateCodes;
extern int tempVarCounter;
extern int labelCounter;

```

中间代码生成的核心函数 `generateFromSyntaxTree`，根据生成的语法树生成相应的中间代码：

```

// 根据语法树节点生成中间代码
string GenerateMidCode::generateFromSyntaxTree(TreeNode* node) {
    if (!node) return "";

    const string& sym = node->symbol;

    // 值节点
    if (sym == "ID" || sym == "NUMBER") {
        return node->value;
    }

    // 语句序列
    if (sym == "stmt-sequence" || sym == "statement" || sym == "program") {
        for (TreeNode* ch : node->children)

```

```

        generateFromSyntaxTree(ch);
        return "";
    }

    // 赋值
    if (sym == "assign-stmt") {
        string lhs = generateFromSyntaxTree(node->children[0]);
        string rhs = handleExpression(node->children[2]);
        addCode(IC_ASSIGN, lhs, rhs);
        return lhs;
    }

    // 读写
    if (sym == "read-stmt") {
        string id = generateFromSyntaxTree(node->children[1]);
        addCode(IC_READ, id);
        return "";
    }

    if (sym == "write-stmt") {
        string v = handleExpression(node->children[1]);
        addCode(IC_WRITE, "", v);
        return "";
    }

    // 表达式
    if (sym == "expression" || sym == "simple-exp" || sym == "term" || sym == "factor")
    {
        return handleExpression(node);
    }

    // 兜底处理
    for (TreeNode* ch : node->children)
        generateFromSyntaxTree(ch);

    return "";
}

```

3. 系统测试

详见上述各个模块中的单元测试以及课程设计测试报告。

4. 系统问题分析

- (1) 当前系统的代码健壮性有待提升，部分错误处理不够完善；
- (2) 系统的图像化窗口界面的美观性有待提高，可引入更复杂精细的布局实现窗口界面的美化。

5. 使用说明书

编译环境：

Qt Creator 4.11.1 (Community)

Desktop Qt 5.14.2 MinGW 32-bit

编译步骤说明：

- 1、打开 Qt Creator 代码编辑器
- 2、导入所需项目
- 3、点击左下角的“运行”按钮即可完成编译并运行

启动任务二项目的系统后，可以看到系统的图形化窗口界面，点击“查看输入规则”按钮可以查看文法和文法对应的测试程序源代码的输入格式与使用说明注意事项，在右侧语法树、中间代码生成模块的“使用说明”处可以查看语法树、中间代码生成的说明。

先输入或通过右侧的“打开”按钮，进行文法的输入/导入，之后对测试源程序句子的输入也同理。按照模块划分可以分别实现 FIRST 集合求解、FOLLOW 集合求解、LR(0)DFA 图构建（表格形式展示）、SLR(1)文法判断、LR(1)DFA 图构建（表格形式展示）、LR(1)分析表生成、源程序语法分析表生成、语法树可视化展示以及中间代码的生成等各项文法分析的功能。

具体使用说明参考“可执行程序”文件夹下的使用说明书。

(3) 任务三

任务三是使用 mini-c 语言的正则表达式和文法对任务一、任务二的项目分别进行测试，具体如下：

3.1. 任务一测试

3.1.1 数据准备

构建 mini-c 语言的正则表达式如下：

```
digit=[0-9]
letter=[a-zA-Z]
_key100S=else|if|int|float|double|return|void|do|while
_special200S=\+\\-
|\\*\\|/|%|<|<=|>|=|==|!=|=|;|,|\\(\\|\\)\\|\\[\\|\\]\\|\\{|\\}
_specialName200S=PLUS|MINUS|MULTIPLY|DIVIDE|MOD|LT|LTEQ|RTEQ|RT
|EQ|NE|ASSIGN|SEMI|DOU|LLM|RLM|LMM|RMM|LBM|RBM
_ID400=(|letter)(|letter|digit)*
_NUMBER500=digit+(.digit)?
_annotation600=//~\\n
```

构建的_simple.tny 代码如下：

```
int x ;
int fact ;
double y ;
int arr [ 100 ] ;
void test ( int a , int b )
{
    a = 1 ;
    b = 2 ;
    return ;
}
void main ( void )
{
    x = 2 ;
    y = 3 ;
    if ( x > 1 )
        do
            fact = fact * x ;
            while ( fact >= 1 ) ;
    else
        x = 1 ;
    return ;
}
```

3.1.2 测试过程

启动项目后，输入上面准备的 mini-c 语言的正则表达式，点击“开始分析”按钮进行分析，之后再依次点击“NFA 生成”、“DFA 生成”、“DFA 最小化”、“词法分析程序生成”对各项结果进行查看：

通过指令【gcc -std=c99 _lexer.c -o lexer】编译该保存下来的_lexer.c 文件后，生成了可执行程序_lexer.exe，此时将测试用的 sample.tny 放到同一文件夹下，运行可执行程序，即可得对应的 output.lex 编码程序文件，再通过任务一项目系统的“查看 lex 文件”按钮实现查看 output.lex 文件内容：



图 3-5 lex 文件查看

3.1.3 测试结果

经过测试，mini-c 正则表达式输入后测试所有的功能均通过，系统能实现对 mini-c 正则表达式的分析并生成 NFA、DFA 图（以表格形式展示），进行 DFA 图最小化并展示（表格形式），生成对应的语法分析程序，并能对 lex 文件进行查看，全部测试都能通过。

3.2. 任务二测试

3.2.1 数据准备

由于需要对任务二系统的文法输入格式进行适配，故我对 mini-c 语言的文法格式做了少量的修改，但不影响 mini-c 文法的功能，具体最终使用的 mini-c 文法如下：

```
program|definition-list|definition|global-variable-definition|function-  
definition|type-indicator|parameters|compound-stmt|parameter-  
list|parameter|local-definition-list|local-definition|statement-
```

```

list|statement|matched-stmt|unmatched-stmt|expression-stmt|condition-
stmt|return-stmt|expression|assignment-expression|simple-
expression|variable|additive-
expression|relop|term|addop|mulop|factor|call|arguments|argument-
list|while-stmt|do-while-stmt|for-stmt|for-init|for-condition|for-update
ID|SEMI|LMM|RMM|LBM|RBM|int|float|double|void|LLM|RLM|if|else|do|while|f
or|return|LTEQ|LT|RT|RTEQ|EQ|NE|PLUS|MINUS|MULTIPLY|DIVIDE|MOD|NUMBER|FL
OAT_NUMBER|DOU|ASSIGN
program -> definition-list
definition-list -> definition-list definition
definition-list -> definition
definition -> global-variable-definition
definition -> function-definition
global-variable-definition -> type-indicator ID ASSIGN NUMBER SEMI
global-variable-definition -> type-indicator ID ASSIGN FLOAT_NUMBER SEMI
global-variable-definition -> type-indicator ID SEMI
global-variable-definition -> type-indicator ID LMM NUMBER RMM SEMI
global-variable-definition -> type-indicator ID ASSIGN simple-expression
SEMI
type-indicator -> int
type-indicator -> float
type-indicator -> double
type-indicator -> void
function-definition -> type-indicator ID LLM parameters RLM compound-stmt
parameters -> parameter-list
parameters -> void
parameters -> #
parameter-list -> parameter-list DOU parameter
parameter-list -> parameter
parameter -> type-indicator ID
parameter -> type-indicator ID LMM RMM
compound-stmt -> LBM local-definition-list statement-list RBM
local-definition-list -> local-definition-list local-definition
local-definition-list -> #
local-definition -> type-indicator ID ASSIGN NUMBER SEMI
local-definition -> type-indicator ID ASSIGN FLOAT_NUMBER SEMI
local-definition -> type-indicator ID SEMI
local-definition -> type-indicator ID LMM NUMBER RMM SEMI
local-definition -> type-indicator ID ASSIGN simple-expression SEMI
statement-list -> statement-list statement
statement-list -> #
statement -> matched-stmt
statement -> unmatched-stmt
statement -> while-stmt

```

```

statement -> for-stmt
statement -> do-while-stmt
matched-stmt -> expression-stmt
matched-stmt -> compound-stmt
matched-stmt -> return-stmt
matched-stmt -> while-stmt
matched-stmt -> for-stmt
matched-stmt -> do-while-stmt
matched-stmt -> if LLM expression RLM matched-stmt else matched-stmt
unmatched-stmt -> if LLM expression RLM statement
unmatched-stmt -> if LLM expression RLM matched-stmt else unmatched-stmt
expression-stmt -> expression SEMI
expression-stmt -> SEMI
return-stmt -> return SEMI
return-stmt -> return expression SEMI
while-stmt -> while LLM expression RLM statement
do-while-stmt -> do statement while LLM expression RLM SEMI
for-stmt -> for LLM for-init SEMI for-condition SEMI for-update RLM
statement
for-init -> expression
for-init -> #
for-condition -> expression
for-condition -> #
for-update -> expression
for-update -> #
expression -> assignment-expression
expression -> simple-expression
assignment-expression -> variable ASSIGN expression
variable -> ID
variable -> ID LMM expression RMM
simple-expression -> additive-expression relop additive-expression
simple-expression -> additive-expression
relop -> LTEQ
relop -> LT
relop -> RT
relop -> RTEQ
relop -> EQ
relop -> NE
additive-expression -> additive-expression addop term
additive-expression -> term
addop -> PLUS
addop -> MINUS
term -> term mulop factor
term -> factor

```

```
mulop -> MULTIPLY
mulop -> DIVIDE
mulop -> MOD
factor -> LLM expression RLM
factor -> variable
factor -> call
factor -> NUMBER
factor -> FLOAT_NUMBER
call -> ID LLM arguments RLM
arguments -> argument-list
arguments -> #
argument-list -> argument-list DOU expression
argument-list -> expression
```

用于测试的 **mini-c** 代码如下所示：

```
// 条件判断程序
int max ( int x, int y ) {
    if ( x > y ) {
        return x ;
    } else {
        return y ;
    }
}

int x ;
int fact ;
double y ;
int arr [ 100 ] ;
void test ( int a , int b )
{
    a = 1 ;
    b = 2 ;
    return ;
}
void main ( void )
{
    x = 2 ;
    y = 3 ;
    if ( x > 1 )
        do
            fact = fact * x ;
        while ( fact >= 1 ) ;
    else
```

```
x = 1 ;
return ;
}
```

对应得到的 LEX 文件内容如下：

```
102 400 "max" 214 102 400 "x" 213 102 400 "y" 215 218 101 214 400 "x" 208
400 "y" 215 218 105 400 "x" 212 219 100 218 105 400 "y" 212 219 219 102
400 "x" 212 102 400 "fact" 212 104 400 "y" 212 102 400 "arr" 216 500 "100"
217 212 106 400 "test" 214 102 400 "a" 213 102 400 "b" 215 218 400 "a"
211 500 "1" 212 400 "b" 211 500 "2" 212 105 212 219 106 400 "main" 214
106 215 218 400 "x" 211 500 "2" 212 400 "y" 211 500 "3" 212 101 214 400
"x" 208 500 "1" 215 107 400 "fact" 211 400 "fact" 202 400 "x" 212 108 214
400 "fact" 207 500 "1" 215 212 100 400 "x" 211 500 "1" 212 105 212 219 -
1
```

3.2.2 测试过程



图 3-6 FIRST、FOLLOW 集构建、LR(0)DFA 构建

如图 3-6 所示，在“文法载入编辑”框内输入/导入此前得到的 mini-c 文法规则文本，点击下方“First 集合求解”即可查看到基于该文法构建的 First 集合（受限于显示框大小，无法将 First 集完整显示，但可以通过滚动条拉动实现完整的查看），点击下方的“Follow 集合求解”则可查看到基于该文法构建的 Follow 集合（受限于显示框大小，无法将 Follow 集完整显示，但可

以通过滚动条拉动实现完整的查看)

再点击系统界面正上方的“LR(0)DFA 生成”按钮,即可在下方的“分析结果表”中查看到基于当前文法构建的表格形式展示的 LR(0)DFA (同样受制于显示框无法完整展示,但可以通过滚动条实现完整查看)。同时,在“文法分析”栏处看到文法的输出进行对比参照。

分析结果表

	状态	状态内文法	ASSIGN	DIVIDE	DOU	EQ	FLC
19	18	factor->LLM.expressionRLM , SEMI ...					46
20	19	global-variable-definition->type-...					
21	20	simple-expression->additive-...				61	
22	21	factor->call. , SEMI factor->call. , E...					
23	22	term->factor. , SEMI term->factor....					
24	23	global-variable-definition->type-...					
25	24	additive-expression->term. , SEMI ...		92			
26	25	factor->variable. , SEMI factor-...					
27	26	global-variable-definition->type-...					
28	27	call->IDLLM.argumentsRLM , SEMI ...					29
29	28	variable->IDLMM.expressionRMM , ...					75
30	29	factor->FLOAT.NUMBER.RLM					

图 3-7 LR(1)DFA 构建

点击图 3-6 中的“LR(1)DFA 生成”,即可对当前输入的 mini-c 文法进行表格形式的 LR(1)DFA 生成,如图 3-7 所示 (同样受制于显示框无法完整展示,但可以通过滚动条实现完整查看)。

分析结果表										
	状态	\$	ASSIGN	DIVIDE	DOU	E			LLM	L
1	0									
2	1	r2								
3	2	acc								
4	3							r12		
5	4							r11		
6	5	r4								
7	6	r3								
8	7							r10		
9	8							s11		
10	9							r13		
11	10	r1								

提示

LR(1)分析表生成完成!

OK

图 3-8 LR(1)分析表构建

点击图 3-6 中的“LR(1)分析表生成”，即可对当前输入的 mini-c 文法进行对应的 LR(1)分析表构建，具体构建结果如图 3-8 所示（同样受制于显示框无法完整展示，但可以通过滚动条实现完整查看）



图 3-9 载入 LEX，进行源程序语法分析

如图 3-9 所示，在项目系统界面左侧“lex 载入编辑”框内输入/载入 mini-c 测试源代码通过任务一生成的词法分析程序分析得到的 output.lex 文件内容，之后点击系统界面正上方的“源代码分析”按钮，即可在下方的“分析结果表”中查看到 mini-c 测试源代码的语法分析逐步过程。

化语法树”按钮，即可生成语法树并通过下方的“语法树展示”区进行展示，如 3-10 所示。

据上述测试流程可知，基于数据准备中所提到的 mini-c 文法和 mini-c 测试程序源代码，把此二者输入后，依次点击按钮控件进行 FIRST 集合求解、FOLLOW 集合求解、LR(0)DFA 构建、SLR(1)文法判断、LR(1)DFA 生成、LR(1)分析表构建、源代码分析、语法树可视化以及中间代码生成（见图 3-7）。

经过测试，FIRST 集、FOLLOW 集、LR(0)DFA、LR(1)DFA、LR(1)分析表、源程序语法分析表、可视化语法树均能正确生成，由于显示区域的大小问题，这几项功能都无法在初始情况下将结果完整展示出来，但可以通过上下、左右的滚动条实现完整查看。

对于 SLR(1)语法判断分析，出现了规约-规约冲突、移进-规约冲突，对于移进-规约冲突，采取只做移进不做规约处理；对于规约-规约冲突，采用选择第一个匹配的产生式处理。

使用另一 mini-c 测试样例，生成的中间代码如下图所示：

使用说明 代码生成 语法树展示

中间代码（三地址码）：
=====

0: a = 1
1: b = 2
2: t0 = global_var + 10
3: a = t0
4: t1 = a * 2
5: b = t1
6: t2 = a + b
7: t3 = t2 / 2
8: c = t3
9: if a <= b goto L0
10: t4 = a - b
11: c = t4
12: goto L1
13: L0:
14: t5 = b - a
15: c = t5
16: L1:
17: i = 0
18: L2:
19: if i >= 5 goto L3
20: t6 = i * i
21: my_array[i] = t6
22: t7 = i + 1
23: i = t7
24: goto L2
25: L3:
26: j = 0
27: sum = 0
28: L4:
29: if j >= 5 goto L5
30: t8 = my_array[j]
31: t9 = sum + t8
32: sum = t9
33: t10 = j + 1
34: j = t10
35: goto L4
36: L5:
37: return

图 3-11 中间代码生成

3.2.3 测试结果

经过测试，mini-c 文法和 mini-c 测试源码输入后测试所有的功能均通过，系统能实现对 mini-c 文法和测试源码进行分析并构建 FIRST 集、FOLLOW 集、LR(0)DFA(表格形式展示)、LR(1)DFA(表格形式展示)、SLR(1)文法判断分析、LR(1)分析表、源程序语法分析表、可视化语法树、中间代码，全部测试均可通过。

四、实验总结（心得体会）

通过本次课程设计项目开发，我加深了对编译原理相关知识的理解，并

通过实践对上学期所学的理论知识有了切实的体会，具体总结如下：

(1) 任务一项目心得体会：

在任务一中，我实现了正则表达式的输入/载入并预处理后进行 NFA、DFA 的转换生成，再对 DFA 最小化处理，并生成相应的语法分析代码，可以导入查看 lex 文件，综合实现一个简单的词法分析器。

在 NFA 图构建过程中我采用**双栈递归下降的解析策略**，通过优先级处理运算符；转换 DFA 时使用 **DFS 算法**生成状态转移表，通过**子集构造法** (ϵ 闭包计算&字符转移闭包) 实现 NFA 到 DFA 的转换，DFA 最小化的过程中，通过**等价类划分方法**，不断分割状态集合直至无法分割实现最小化 DFA 的构建；通过使用**讲稿中的语法分析代码方法二**，实现了语法分析代码的生成。

总而言之，任务一的项目将上学期所学的理论知识通过实践实现，让我学到了如何将理论知识转为实际应用，让我完整地体会了编译原理词法分析的流程，为我更好地学习、理解和应用自动机相关提供了宝贵的机会。

(2) 任务二项目心得体会：

在任务二中，我实现了一个用于处理文法分析并对该文法对应的源程序代码进行文法符合性分析与合规性检查的文法处理器应用，该应用综合实现了 FIRST 集和 FOLLOW 集的构建、

LR(0)DFA 构建、SLR(1)文法分析判断、LR(1)DFA 构建、LR(1)分析表生成、源程序语法分析表生成、语法树构建与可视化、中间代码生成等多项重要的文法处理分析功能。

在实现任务二项目的过程中，我采用**递归迭代算法**遍历文法实现 FIRST 集合和 FOLLOW 集合的构建；使用**闭包计算**和**GOTO 函数算法**，实现了 LR(0)DFA 图（表格形式）的生成；基于**FOLLOW 集与移进符号交集**和**FOLLOW 集交集**实现 SLR(1)文法冲突判断；通过**闭包计算**和**展望符传播**实现规范 LR(1)项族构建，实现 LR(1)DFA（表格形式）的生成；基于 LR(1)DFA 的**ACTION 和 GOTO 表构建算法**，通过状态-符号映射生成移进、规约和接受动作，实现了 LR(1)分析表的生成；通过**LR(1)分析算法+词法分析**（Token 序列生成）和**栈式移进-规约过程**，实现了句子语法分析表的构建；通过**自底向上构建算法**，使用节点栈再移进-规约过程中组合出语法树；通过**语法制导翻译算法**，生成三地址码（三元组形式），基于产生语义动作，实现了中间代码的生成。

总而言之，完整完成任务二项目让我对文法处理与分析以及代码合规性判断等有了更深一层的理解，让我把上学期所学通过实践加以熟悉。

(3) 任务三心得体会：

任务三为综合测试，是用 mini-c 语言对任务一和任务二的

项目进行整体的测试工作，通过任务三的测试，我深刻理解了代码健壮性的重要，要同时适配多种语言的文法分析和正则表达式处理，需要对文法有详细的了解和多方面的考虑，这提升了我对编译原理知识的应用能力和项目排错等能力。

总而言之，本次课程设计不仅巩固了我的编译原理知识，还培养了我的项目管理和独立开发能力。我认识到编译器设计是一个系统工程，需要平衡理论严谨性和实践可行性。这为我后续的学习打下了坚实基础，我将继续探索编译技术，提升自我，拓展更多收获。

五、参考文献：

- [1] 编译原理课程设计--讲稿
- [2] 编译原理课程设计任务书
- [3] 《编译原理》课件
- [4] 《编译原理（第2版）》[M]. 赵建华等译. 2009.
- [5] Qt Group. 《Qt 5.15 框架参考手册》[EB/OL]. <https://doc.qt.io/qt-5/>, 2020.

六、课程设计自评

我对本次课程设计的自评分数为 80 分，理由如下：

（1）优点方面：

1. 任务一项目的全部功能均已实现，所有要求都已完成，为用户提供了一个交互友好的图形化窗口界面，用户可以输入或通过文件导入指定格式的正则表达式，本项目系统能够正确地将用户的正则表达式处理构建出 NFA 图，并实现 DFA 图的转化和 DFA 最小化处理，同时能生成对应的词法分析程序，还提供了查看单词编码的功能。使用 TINY 语言

和 mini-c 语言的正则表达式进行测试，所有功能都符合预期，结果均正确。

2. 任务二项目的全部功能均已实现，并完成了选做功能的实现，该项目为用户提供了一个友好的图形化窗口交互界面，用户可以根据使用说明按格式要求输入或从文件导入对应语言的文法和测试用源程序，通过与界面上的控件交互，系统实现了 FIRST 集、FOLLOW 集的构建呈现，能正确构建 LR(0)DFA 和 LR(1)DFA，能针对文法生成 LR(1)分析表，能对用户提供的文法进行 SLR(1)文法判断，能对用户提供的文法对应的源程序代码进行源程序语法分析并构建成表，并生成可视化语法树、中间代码供用户查看。本项目对 TINY 语言和 mini-c 语言的文法和示例代码的测试均通过，所有分析结果均正确。
3. 任务三的 mini-c 语言对任务一和任务二的项目进行综合测试，其结果均正确，通过了所有自组织的测试用例。
4. 任务一与任务二的系统通过一个系统两个界面实现合并，方便用户使用，同时对任务一、任务二中的每个子任务独立设置了类模块进行处理，实现了本课设项目的规范化、模块化，提高了可维护性，并且利于后续的功能扩充。

(2) 不足方面：

1. 中间代码生成功能对于一些复杂的函数内嵌套语句仍有出现部分错误的情况未能得到很好的解决，文法和正则表达

式的输入格式有一定的要求，有进一步优化的空间，语法树生成的相关代码对部分结点做了特殊处理，有待进一步优化。

2. 代码健壮性尚且不足，系统鲁棒性有待提升，部分错误处理不够完善，对于文法规则、正则表达式的输入有一定的要求且对用户提供的文法和正则表达式尚未能做好完善的输入格式检查，有待优化和进一步完善。
3. 系统的图像化窗口界面的美观性有待提高，可引入更复杂精细的布局实现窗口界面的美化。